Robustness and Typed Hypernet Contexts

A mathematical commentary on the Lean development

Companion to `danghica/hypernets-lean`

24 October 2026

Source snapshot: `b850e7a`; Lean `4.35.0`

Abstract

This document explains every definition and proved statement in the Lean development, including its regression examples, in mathematical English. Each declaration links to its complete source fragment at a fixed Git revision. The abstract counting-simulation soundness theorem is proved outright; the forward and converse robustness arguments remain conditional on an explicit specimen interface.

The concrete development supplies finite typed graphs, well-formed substitution, operation-path uniqueness, finite boxed hierarchies, exact hole inventories and freshness criteria, recursive isomorphism, transported sites, and nested substitution laws. Globally focus-free and focussed hypernets, actual states, focus insertion and erasure, both round trips, and recursive renaming invariance are formalized. The five coordinate search rules induce deterministic search on recursive state classes. Projection from literal-label classes preserves and lifts executions with exact lengths and reflects search termination. Concrete rootedness and refocusing are defined; search preserves rootedness. Rooted focus sources have operation paths from the unique input, proving the concrete Lemma 23 and the non-operation-path obstruction of Lemma 26(1).

One-way hypernets have their term-to-term path definition, renaming invariance, and filling closure. Path extraction proves the single-hole obstruction of Lemma 26(2), including self-looping holes. Simultaneous shallow substitution now has proved incidence, finiteness, and typing invariants. A concrete pre-template and shallow-specimen representation supplies actual filled states. Output-closure yields the multi-hole obstruction of Lemma 26(3) for these shallow specimens, allowing the one-way side to vary at different holes. Their initial/final agreement also connects the boundary conclusion of Lemma 39(1) directly to concrete specimen states.

The dependency map distinguishes these checked results from the remaining obligations. Deep specimen substitution, compatibility of simultaneous expansion with addressed substitution, contextual-rewrite adequacy, the full UAM, safety and robustness, and the complete specimen-interface interpretation remain open. This is therefore not yet a formalization of the paper's full Theorem 6.

Contents

1 Scope, notation, and the status of the formalization **6**
1.1 Paper-to-Lean proof dependency map . . . 7
1.2 Conventions . . . 8
1.3 Dependency order . . . 9

2 Machines and finite executions **9**

3 Cost relations and state refinement **12**

4 Counting simulation and its soundness **14**

5 Typed observations and contextual lifting **16**

6 The abstract specimen interface **17**

7 The forward robustness argument **20**

8 Opposite specimens and the converse argument **21**

9 State and contextual consequences **23**

10 Semantic examples and regression proofs **25**
10.1 A countdown machine . . . 25
10.2 A crash and an infinite loop . . . 26
10.3 The zero-step converse case . . . 27

11 Finite carriers and concrete interfaced graphs **29**
11.1 Enumerations and bijections . . . 29
11.2 Ports, signatures, and graph invariants . . . 30
11.3 Elementary graph constructors . . . 32

12 Shallow plugging, including self-loop holes **33**
12.1 Carriers and boundary identification . . . 33
12.2 Preservation of incidence invariants . . . 34
12.3 Labels, typing, and the certified result . . . 35

13 Paths, renaming, and isomorphism **36**
13.1 Paths and their behaviour under plugging . . . 36
13.2 Transport along carrier bijections . . . 38
13.3 Graph isomorphism and shallow context congruence . . . 39

14 Concrete uniqueness of operation paths **40**

15 Typed boxes, deep occurrences, and contexts **42**
15.1 Relabelling without changing incidence . . . 42
15.2 The linear wire and label discipline . . . 42
15.3 Box construction and promotion of the depth bound . . . 44
15.4 Finite occurrence addresses at arbitrary depth . . . 44
15.5 Hole lookup and the context condition . . . 45

16 Addressed deep replacement and concrete locality **46**
16.1 Changing one box body . . . 46
16.2 Typed sites and replacement at a site . . . 46

17 Concrete graph examples and regression proofs **47**
17.1 Self-loop gluing and the necessary reflection restriction . . . 47
17.2 Values, boxes, and distinguished occurrence addresses . . . 48
17.3 An input-rooted operation path . . . 49
17.4 Filling a hole inside a certified context . . . 50

18 Occurrence fibres and transport of hole observations **51**
18.1 Coordinates local to an outer edge . . . 51
18.2 Transport along equality and edge maps . . . 52

19 Hole inventories and sharp shallow context closure **53**
19.1 Hole inventories, absence, and isomorphism . . . 53
19.2 Provenance and the abstract collision argument . . . 54
19.3 Concrete shallow provenance and a first context operation . . . 55
19.4 Exact inventory and the sharp freshness condition . . . 55
19.5 Consumption of a selected hole and certified composition . . . 56

20 Provenance and context closure at arbitrary box depth **57**
20.1 Local provenance for a changed label . . . 57
20.2 Lifting a body's provenance through one box . . . 58
20.3 The deep context closure theorem . . . 59

21 Context composition: absence, reuse, and a collision **60**
21.1 Hole-free filling inside a box . . . 60
21.2 Two distinct shallow holes in series . . . 61
21.3 A checked failure without freshness . . . 62

22 Exact inventories and sharp context closure at every depth **62**
22.1 Separating one outer edge from the rest of a graph . . . 62
22.2 A syntactic inventory of retained holes . . . 64
22.3 Necessary and sufficient context closure . . . 65
22.4 Consumption of the selected label . . . 66
22.5 Concrete regressions inside boxes . . . 67

23 Recursive isomorphism and substitution modulo renaming **68**
23.1 Isomorphisms with related labels . . . 68
23.2 Recursive comparison of box contents . . . 69
23.3 Operation traces and paths . . . 71
23.4 Transporting occurrences and holes through all boxes . . . 71
23.5 Congruence of shallow and deep replacement . . . 73
23.6 Graphs and replacement modulo recursive renaming . . . 74
23.7 A strict extension of literal-label isomorphism . . . 75

24 Transporting replacement sites through isomorphisms **76**
24.1 Transport of boundary indices and shallow hosts . . . 76
24.2 Structural correspondence of addresses . . . 78
24.3 Substitution and freshness under transport . . . 79
24.4 A moved address and a wrong-address counterexample . . . 80

25 Associativity, units, and exact context composition **81**
25.1 Shallow associativity with explicit carrier bijections . . . 81
25.2 Composition at arbitrary box depth . . . 82
25.3 Exact inventories and context validity of composites . . . 84
25.4 Corolla units . . . 85
25.5 Restoration and unit equations at deep sites . . . 87
25.6 Regression proofs: self-loops, mixed depth, and repaired collisions . . . 87

26 Concrete focuses and the boundary argument of Lemma 39(1) **89**
26.1 Global focus freedom and a unique shallow focus . . . 89
26.2 Preservation through one filling . . . 90
26.3 Finite fillings and the state space . . . 91
26.4 Nonvacuous checks . . . 92

27 The five search rules on underlying graph coordinates **93**
27.1 Operation kinds and certified incidence views . . . 93
27.2 Local outcomes and the search relation . . . 94
27.3 Determinism and initial/final boundaries . . . 95
27.4 Preservation under context filling . . . 96
27.5 Concrete checks of rule selection . . . 97

28 Explicit focus insertion and realization of typed search positions **98**
28.1 Splitting a vertex . . . 98
28.2 Focus certification, contraction, and holes . . . 100
28.3 Search preserves term typing . . . 101
28.4 Actual states from typed coordinates . . . 101
28.5 Boundary and interior regression checks . . . 103

29 Focus, search, and rooted-state paths **104**
29.1 Contraction maps and the retained carriers . . . 104
29.2 A valid focus-free underlying graph . . . 106
29.3 Reinsertion recovers an arbitrary focussed graph . . . 107
29.4 Focus-preserving isomorphisms and state recovery . . . 108
29.5 Search between actual states . . . 109
29.6 Concrete execution and interior recovery . . . 110
29.7 Insertion followed by erasure . . . 111
29.8 Search coordinates respect renaming . . . 112
29.9 Insertion respects renaming . . . 113
29.10 Exact characterization of search from realized states . . . 113
29.11 Erasure and state search respect renaming . . . 114
29.12 Search on isomorphism classes of states . . . 116
29.13 Concrete quotient-search regression . . . 117
29.14 Recursive renaming of focussed states . . . 117
29.15 Search and insertion under recursive renaming . . . 118
29.16 Erasure under recursive renaming . . . 120
29.17 State search modulo recursive endpoint renaming . . . 121
29.18 Recursive state classes and their search relation . . . 122
29.19 Projection and lifting between the two state quotients . . . 123
29.20 Exact-length executions across the quotients . . . 124
29.21 Recursive-renaming and execution regression proofs . . . 125
29.22 Canonical roots of concrete states . . . 126
29.23 Concrete rootedness and its search invariant . . . 127
29.24 Operation reachability under coordinate search . . . 128
29.25 Incoming paths and insertion of the focus . . . 129
29.26 Rooted states have actual operation paths: Lemma 23 . . . 130
29.27 The non-operation-path obstruction: Lemma 26(1) . . . 131
29.28 Concrete refocusing and the remaining rewrite obligation . . . 132
29.29 Rooted and non-rooted concrete examples . . . 133
29.30 One-way hypernets and the term-boundary restriction . . . 133
29.31 Extracting an operation path from a filling . . . 134
29.32 Reflecting host reachability through a one-way filling . . . 135
29.33 The concrete one-way obstruction: Lemma 26(2) . . . 136
29.34 One-way examples and a filled-state obstruction . . . 137
29.35 Simultaneous shallow substitution: carriers and incidence . . . 138
29.36 Well-formedness of simultaneous expansion . . . 139
29.37 Retained corollas, identity expansion, and hole freedom . . . 140
29.38 Paired path reflection for many fillings . . . 141
29.39 Actual focussed expansions and their obstruction . . . 142
29.40 Concrete pre-templates and shallow specimens . . . 143
29.41 Lemma 26(3) and the boundary bridge for shallow specimens . . . 144
29.42 A constructed two-hole focussed specimen . . . 145
29.43 Alternating one-way choices: a necessary regression . . . 146

30 Proof checking and remaining obligations **147**
30.1 The two axiom audits . . . 147
30.2 Reproducing the Lean and document checks . . . 149
30.3 What remains for the paper's concrete theorem . . . 150

A Technical terms **151**

1 Scope, notation, and the status of the formalization

The abstract part of the development follows the proof in Appendix F of [1]. Its completed mathematical core is the soundness of counting simulation, corresponding to Proposition 37. The remaining argument is expressed through a structure of abstract specimens. Given that structure and its laws, Lean proves both directions of the simulation construction and their contextual consequences. Constructing this structure from the paper's actual hypernets is still a separate task. The concrete graph part now supplies finite interfaced graphs, certified shallow plugging, renaming and path invariance, a typed boxed hierarchy, and addressed deep replacement. These constructions and their regression examples are explained after the abstract development. The context calculus now also proves exact hole inventories, necessary-and-sufficient freshness, and exact consumption of selected hole labels at arbitrary box depth. Recursive isomorphism now identifies independently renamed box contents and respects paths, hole observations, and deep replacement. Sites now transport across isomorphic hosts, and substitution respects simultaneous host and replacement renaming, including the exact freshness condition. Nested substitutions now associate at arbitrary box depth, with corolla unit laws and a sharp final context-validity criterion even when an intermediate collision is later removed.

The concrete development now also defines globally focus-free hypernets, focussed hypernets, and the hole-free state space. The boundary argument of Lemma 39(1) is proved for finite sequences of addressed focus-free fillings; see Section 26. The five search rules are now defined on underlying graph coordinates, with determinism, boundary laws, and preservation under shallow filling proved (Section 27). Explicit focus insertion now realizes typed coordinates as actual states, preserving holes and boundary observations, and contracting each inserted focus recovers original incidences (Section 28). General erasure and the focus-preserving erasure-then-insertion isomorphism are now proved for arbitrary closed focussed states. Erasure-based search on actual states is deterministic up to focussed isomorphism and has the expected search boundary laws (Section 29). Both focus round trips and source/target renaming invariance are proved; search and boundary observations descend to literal-label state isomorphism classes, with equality-based search determinism. Search also respects recursive renaming inside boxes and descends to recursive state classes. The map from literal to recursive classes preserves and lifts search executions with exact lengths, including successful search termination. Canonical roots and concrete rootedness are now defined; search preserves rootedness, and concrete rooted focus sources have operation paths from the unique input. An incoming path containing a non-operation edge rules out rootedness (Section 29.27). Concrete refocusing is defined and proved for search. One-way hypernets and the single-hole obstruction of Lemma 26(2) are now proved, using extraction of actual operation paths through shallow substitution (Section 29.33). Simultaneous shallow expansion now has proved graph invariants. Concrete pre-templates, output-closure, and shallow specimens provide actual filled states and a multi-hole proof of Lemma 26(3) for this representation (Section 29.40). Initial/final agreement now applies directly to these specimen states. Deep specimens, compatibility with addressed substitution, graph-rewrite adequacy, substitution and compute transitions, and the full specimen interpretation remain open.

1.1 Paper-to-Lean proof dependency map

The target is the proof of Theorem 6 through Propositions 40, 37, and 38. The following map separates checked arguments from their missing concrete interpretations, rather than treating graph infrastructure as progress on every part of that proof.

1. **Proposition 37: checked.**

   `Machine.CountingSimulation.sound` proves counting-simulation soundness for a deterministic machine and a reasonable cost triple. Its proof follows the quantitative argument, including the eventual-stuck alternative.

2. **Proposition 40: checked conditionally.**

   `SpecimenSystem.countingSimulation` and its converse construct counting simulations from the abstract specimen laws, safety, and robustness. These laws are parameters, not established concrete UAM facts.

3. **Lemma 39(1), Proof F.4: boundary argument checked.**

   `Focussed.fillings_boundary_agreement` proves initial/final agreement for two finite filling sequences of one focussed hypernet. The proof preserves the focus mode and input status of its source, as in the paper. The remaining connection is to construct such witnesses for full deep specimen and tuple substitution. The simultaneous shallow specimen representation now also has a direct proof of this boundary conclusion.

4. **Lemma 39(2) and (3): open.**

   Point (2) uses the contrapositive of Lemma 26(3), requiring concrete rooted states and output-closure. Concrete rootedness, its search invariant, the operation-path conclusion of Lemma 23, and the obstruction in Lemma 26(1) are now checked. Lemma 26(2) is also checked for one-way shallow fillings, including self-looping holes. Lemma 26(3) is now proved for concrete simultaneous shallow specimens, with pointwise output-closure and actual filled states. Extension to deep holes and compatibility with addressed substitution remain open. Point (3) requires the entering/exiting classifications and context-only traversal argument. The Fig. 9 search rules are now checked in underlying-graph coordinates, and existing context steps survive shallow hole filling. Extracting a context step from a filled specimen and proving adequacy for local graph rewriting remain open. Typed coordinates have explicit state realizations, and arbitrary closed states now have erasure-based search with proved boundary laws, invariant under renaming and deterministic on literal-label isomorphism classes. The abstract field `context_step` currently assumes their needed consequence. It has not been discharged by the graph path results.

5. **Proposition 38: logical lifting checked; initial bridge open.**

   `Observations.contextual_of_lifting` proves the logical implication once initial pairs belong to the lifting. Concrete initial focus attachment is now constructed. Allowed contexts, the observation interface, and specimen witnesses must still discharge the `initial_lift` parameter.

6. **Theorem 6: conditional, not complete.**

   The two `contextual_sufficiency` theorems combine these abstract pieces. The paper itself assumes determinism and refocusing of the UAM. Those should remain the theorem's hypotheses: the missing work is to define their concrete meaning and prove the required structural consequences under them, not to prove arbitrary extrinsic rewrite systems deterministic.

The typed finite-depth hierarchy and explicit renaming bijections are representation choices. Relating them to the paper's set-based hypernets and across depth bounds is a separate adequacy obligation. The existing plugging, path, inventory, and isomorphism lemmas support that representation; they do not replace the operational proofs in Appendix F. The next proof dependency is adequacy of erasure-based search for local graph rewriting, together with the remaining transitions of §6.4. The deep-specimen extension of Lemma 26(3), full-UAM refocusing, and Lemma 39(2–3) remain subsequent obligations. The active/passive operation partition and all five coordinate search rules are now defined. These then support the concrete Proposition 40 bridge; initial focus construction now supplies its graph-level component for the separate Proposition 38 bridge, whose specimen witnesses remain open.

As a planning estimate, approximately 60% of the mathematical effort toward the concrete general robustness theorem is complete (a plausible range is 50–70%). This revises the preceding estimate of about 55% after the concrete shallow multi-hole specimen milestone. It is a subjective estimate, not a ratio of declarations or discharged checklist entries. The remaining operational adequacy, full UAM, and specimen locality arguments are substantial and may reveal further missing infrastructure.

There are three distinct kinds of assertion below. A *definition* specifies a mathematical object or predicate. A *proved statement* is established by a Lean proof term. An *interface law* is a hypothesis carried by a structure; the current code uses that hypothesis but does not prove it for concrete hypernets. In particular, the laws of a specimen system must not be read as established theorems about the universal abstract machine.

All code links point to revision `b850e7a`, rather than to the moving `main` branch. Each link highlights the declaration together with its proof or defining fields. Namespace prefixes in link labels disambiguate declarations with the same short name. Anonymous Lean `example` declarations are explained alongside the named theorems.

1.2 Conventions

Fix a set of states X and a machine M on X. Write $s \to t$ for its one-step transition relation, $s \longrightarrow^n t$ for exactly n transitions, and $\mathsf{Final}(s)$ for successful finality. Natural numbers include zero. A cost preorder Q is written infix: $n\,Q\,m$. The source count is always the left argument and the target count the right argument. Write $s \preceq_Q t$ for state refinement, and $s \simeq_Q t$ for mutual state refinement.

The paper's three cost parameters (Q, Q', Q'') are named (Q, P, S) in the code and in this commentary. Thus P is the auxiliary relation on the *left* of a simulation candidate, and S is the one on its *right*. We spell out witnesses rather than relying on a convention for composition of relations.

No result assumes that all executions terminate. An arbitrary state can be final, stuck, or capable of further computation. Successful termination is the only qualitative observation in the present state-refinement definition. Neither equality of returned values nor equality of final states across two different source states is required.

1.3 Dependency order

The files are organized as follows. Finite-execution facts in `Transition.lean` support the quantitative relations in `Quantitative.lean` and the soundness argument in `Simulation.lean`. The observation interface in `Contextual.lean` converts state results into contextual ones. The specimen argument in `LocalReasoning.lean` constructs the simulation to which soundness applies. Finally, `Examples.lean` checks substantive edge cases, and `Audit.lean` reports theorem dependencies. The root file `Hypernets.lean` imports the development and its examples; it adds no mathematical declaration.

The concrete branch begins with `Graph/Finite.lean` and `Graph/Basic.lean`. Plugging, paths, and isomorphism support the hierarchy and deep replacement modules. Operation-path uniqueness imports the counted-execution results, providing a concrete use of the abstract machine lemmas. Occurrence fibres and provenance then support shallow and deep context closure in the new context modules. Both branches are imported by the root file.

The exposition follows these dependency orders. Section numbers and theorem numbers here are local to this document; references to the paper are explicitly identified as such.

2 Machines and finite executions

Definition 2.1 (Machine)**.** A machine on X consists of a relation $\to \subseteq X \times X$, a predicate $\mathsf{Final} \subseteq X$, and the law

$$\mathsf{Final}(s) \implies \neg(s \to t) \qquad (s, t \in X).$$

Thus every successful final state has no outgoing transition. No converse is assumed: an unsuccessful normal form is permitted.

Lean: `Machine`

The fields `step` and `final` supply the relation and predicate. The field `final_normal` supplies the displayed law. A machine may have any state type; finiteness or a graphical representation is not required.

Definition 2.2 (Determinism, normality, and stuckness)**.** The machine is *deterministic* if $s \to t_1$ and $s \to t_2$ imply $t_1 = t_2$. A state s is *normal* if $s \to t$ is false for every t. It is *stuck* if it is normal and is not final:

$$\mathsf{Normal}(s) \iff \forall t\, \neg(s \to t), \qquad \mathsf{Stuck}(s) \iff \mathsf{Normal}(s) \wedge \neg\mathsf{Final}(s).$$

Lean: `Machine.Deterministic`

Lean: `Machine.Normal`

Lean: `Machine.Stuck`

Determinism is a separate hypothesis, not a field of every machine. Stuckness distinguishes failure from successful finality even though both are normal forms.

Definition 2.3 (Counted execution)**.** The relation $s \longrightarrow^n t$ is generated inductively by

$$\frac{}{s \longrightarrow^0 s}, \qquad \frac{s \to u \quad u \longrightarrow^n t}{s \longrightarrow^{n+1} t}.$$

Lean: `Machine.Steps`

These are precisely the constructors `zero` and `cons`. The count includes every transition of the machine. There is no separate cost weight for searching or rewriting. In particular, zero transitions cannot change a state.

Lemma 2.4 (Zero-step equality)**.** *If $s \longrightarrow^0 t$, then $s = t$.*

Lean: `Machine.Steps.zero_eq`

Proof. Only the zero constructor can produce an execution with count zero. Its two endpoints are the same state. The step constructor has a positive count and cannot occur. □

Lemma 2.5 (Concatenation)**.** *If $s \longrightarrow^m t$ and $t \longrightarrow^n v$, then $s \longrightarrow^{m+n} v$.*

Lean: `Machine.Steps.append`

Proof. Induct on the first execution. For a zero-step execution, $s = t$, so the second execution is the required one. Otherwise its first transition is $s \to u$ and its remaining execution has some length k. By induction, that remainder followed by the second execution has length $k + n$. Prepending $s \to u$ gives length $(k + n) + 1 = (k + 1) + n$. No determinism is used. □

Lemma 2.6 (An execution from a normal state)**.** *If $s \longrightarrow^n t$ and* $\mathsf{Normal}(s)$*, then $n = 0$ and $s = t$.*

Lean: `Machine.Steps.normal_eq`

Proof. The zero-step case has both conclusions by definition. A positive execution would begin with a transition out of s, contradicting normality. □

For the following three lemmas, assume determinism.

Lemma 2.7 (Every execution is a prefix of a final execution)**.** *Suppose $s \longrightarrow^m t$, $s \longrightarrow^n z$, and* $\mathsf{Final}(z)$*. Then there is a natural number r such that*

$$n = m + r \quad \text{and} \quad t \longrightarrow^r z.$$

In particular $m \leq n$.

Lean: `Machine.Steps.prefix_of_final`

Proof. Induct on the execution of length m, allowing n to vary. If $m = 0$, then $t = s$; take $r = n$ and use the given final execution.

Otherwise the first execution starts with $s \to u$. The final execution cannot have length zero: that would make $s = z$ final while an outgoing transition exists. It therefore starts with $s \to u'$. Determinism gives $u = u'$. Apply the induction hypothesis to the two remaining executions. If their lengths are $m - 1$ and $n - 1$, the result gives $n - 1 = (m - 1) + r$ and the required execution $t \longrightarrow^r z$. Hence $n = m + r$. The Lean proof uses constructor indices rather than natural-number subtraction, so no subtraction side conditions are needed. □

Lemma 2.8 (Uniqueness of successful terminal executions)**.** *If* $s \longrightarrow^m t$, $s \longrightarrow^n z$, $\mathsf{Final}(t)$, *and* $\mathsf{Final}(z)$, *then* $m = n$ *and* $t = z$.

Lean: `Machine.Steps.final_unique`

Proof. The prefix lemma gives $n = m + r$ and $t \longrightarrow^r z$. Finality of t makes it normal. The normal-state lemma yields $r = 0$ and $t = z$, and therefore $m = n$. □

Lemma 2.9 (A state on a successful execution is not stuck)**.** *If* $s \longrightarrow^m t$, $s \longrightarrow^n z$, *and* $\mathsf{Final}(z)$, *then* t *is not stuck.*

Lean: `Machine.Steps.not_stuck_of_final`

Proof. Suppose t were stuck. The prefix lemma gives a remaining execution $t \longrightarrow^r z$. Normality of t forces $t = z$. Since z is final, t is final, contradicting the unsuccessful part of stuckness. □

Lemma 2.10 (Removing a prescribed first transition)**.** *If* $s \to s'$ *and* $s \longrightarrow^{n+1} t$, *then* $s' \longrightarrow^n t$.

Lean: `Machine.Steps.drop_first`

Proof. The positive execution has a first transition $s \to u$ and a remainder $u \longrightarrow^n t$. Determinism identifies u with s', giving the stated remainder. The conclusion concerns the *prescribed* successor s'; without determinism it would only be valid for some successor. □

Lemma 2.11 (A stuck execution continues after an available step)**.** *If* $s \to s'$, $s \longrightarrow^n t$, *and* $\mathsf{Stuck}(t)$, *then for some* k, $s' \longrightarrow^k t$.

Lean: `Machine.Steps.stuck_after_step`

Proof. The execution cannot have length zero, since then $s = t$ would be normal despite $s \to s'$. Its first transition is therefore $s \to u$, with a remaining execution to t. Determinism gives $u = s'$. Take the length of that remainder as k. The code does not retain the additional equality $n = k + 1$, because later uses need only the existence of a remainder. □

3 Cost relations and state refinement

Definition 3.1 (Cost preorder)**.** A cost order is a reflexive, transitive relation Q on $\mathbb{N}$.

Lean: `CostOrder`

The relation and its two preorder proofs are stored together. Antisymmetry, decidability, and compatibility with addition are not part of this definition.

Definition 3.2 (Reasonable triple)**.** A triple (Q, P, S) of cost preorders is *reasonable* if all four of the following implications hold:

$$\begin{aligned} a\,Q\,b \;\wedge\; c\,Q\,d &\implies (a+c)\,Q\,(b+d), && \text{(addition)}\\ a\,P\,b &\implies b \le a, && \text{(decrease)}\\ a\,P\,b &\implies a\,S\,b, && \text{(inclusion)}\\ a\,P\,b \;\wedge\; b\,Q\,c \;\wedge\; c\,S\,d &\implies a\,Q\,d. && \text{(sandwich)} \end{aligned}$$

Lean: `Reasonable`

These four fields express the paper's three conditions: the decrease and inclusion fields are the two parts of its condition (B). The decrease field will justify a strict descent in the soundness proof. Inclusion is used when turning safety's auxiliary equivalences into an asymmetric up-to relation.

Definition 3.3 (Converse and the basic cost orders)**.** The converse preorder is $a\,Q^{-1}\,b$ if $b\,Q\,a$. The four named orders are

$$a\,E\,b \iff a = b, \quad a\,I\,b \iff b \le a,$$
$$a\,H\,b \iff a \le b, \quad a\,U\,b \iff \text{true}.$$

Here E, I, H, and U denote `exact`, `improvement`, `overhead`, and `any`, respectively.

Lean: `CostOrder.converse`

Lean: `CostOrder.exact`

Lean: `CostOrder.improvement`

Lean: `CostOrder.overhead`

Lean: `CostOrder.any`

Verification of the preorder fields. Converse preserves reflexivity. Given $b\,Q\,a$ and $c\,Q\,b$, transitivity of Q gives $c\,Q\,a$, establishing transitivity of Q^{-1}. Equality is reflexive and transitive. The relation I is reflexive by $a \le a$; from $b \le a$ and $c \le b$ one obtains $c \le a$. The corresponding verification for H uses the usual order direction. All requirements for the universal relation U are immediate. □

Lemma 3.4 (Exact costs are reasonable)**.** *The triple* (E, E, E) *is reasonable.*

Lean: `CostOrder.reasonable_exact`

Proof. Substitute equals for equals in the addition condition. The decrease condition reduces to $a \le a$, inclusion is equality included in itself, and the sandwich condition is transitivity of equality. □

Lemma 3.5 (Improvement costs are reasonable)**.** *The triple* (I, I, I) *is reasonable.*

Lean: `CostOrder.reasonable_improvement`

Proof. For addition, $b \le a$ and $d \le c$ give $b + d \le a + c$. Decrease is the definition of I, and inclusion is reflexive inclusion. The sandwich premises give $d \le c \le b \le a$, hence $a\, I\, d$. □

Lemma 3.6 (Unconstrained termination costs are reasonable)**.** *The triple* (U, I, U) *is reasonable.*

Lean: `CostOrder.reasonable_termination`

Proof. Addition, inclusion, and the sandwich condition have universally true conclusions. Decrease is again the definition of I. □

Lemma 3.7 (Overhead with exact auxiliary costs is reasonable)**.** *The triple* (H, E, E) *is reasonable.*

Lean: `CostOrder.reasonable_overhead`

Proof. The addition condition is monotonicity of addition for $\le$. Equality implies the required weak decrease and includes into itself. In the sandwich condition, substitute $a = b$ and $c = d$ into $b \le c$ to obtain $a \le d$. □

Definition 3.8 (State refinement)**.** For a cost preorder Q, define

$$s \preceq_Q t \quad \Longleftrightarrow \quad \begin{array}{c} \text{for every } n, z \text{ with } s \longrightarrow^n z \text{ and } \mathsf{Final}(z), \\ \text{there exist } m, w \text{ with } n\, Q\, m,\ t \longrightarrow^m w, \text{ and } \mathsf{Final}(w). \end{array}$$

State equivalence is mutual refinement: $s \simeq_Q t$ if $s \preceq_Q t$ and $t \preceq_Q s$.

Lean: `Machine.Refines`

Lean: `Machine.Equiv`

If s has no successful execution, its refinement obligation is vacuous. The definition therefore does not distinguish stuckness from divergence unless a surrounding observation supplies additional distinctions. For $Q = I$, the target must terminate in no more steps than the source. For $Q = H$, it must terminate in at least as many steps. For $Q = E$, the counts agree exactly; for $Q = U$, counts are ignored.

Lemma 3.9 (Reflexivity)**.** *For every s, $s \preceq_Q s$.*

Lean: `Machine.Refines.refl`

Proof. Given a successful execution of length n from s, choose that same execution and final state as witnesses. Reflexivity supplies $n\,Q\,n$. □

Lemma 3.10 (Transitivity)**.** *If $s \preceq_Q t$ and $t \preceq_Q v$, then $s \preceq_Q v$.*

Lean: `Machine.Refines.trans`

Proof. Take a successful execution of length n from s. The first refinement supplies a successful execution of length m from t, with $n\,Q\,m$. Apply the second refinement to this execution to obtain a successful execution of length k from v, with $m\,Q\,k$. Transitivity gives $n\,Q\,k$, and these are the required witnesses. Determinism is unnecessary. □

Lemma 3.11 (Monotonicity in the cost relation)**.** *If $Q \subseteq P$ and $s \preceq_Q t$, then $s \preceq_P t$.*

Lean: `Machine.Refines.mono`

Proof. Reuse each execution and final-state witness supplied by the original refinement. Replace its cost proof $n\,Q\,m$ by $n\,P\,m$ using the inclusion. □

4 Counting simulation and its soundness

Definition 4.1 (Up-to closure)**.** Given a relation R on states and cost orders P, S, define

$$\mathsf{UpTo}_{P,S}(R)(s,t) \iff \exists u, v\ (s \preceq_P u \ \wedge\ R(u,v) \ \wedge\ v \preceq_S t).$$

Lean: `Machine.UpTo`

The witnesses occur in path order. In particular the right-hand refinement is from v to t, not from t to v. Auxiliary refinement can change the states at which the candidate relation is resumed.

Definition 4.2 (Counting simulation)**.** A relation R is a (Q, P, S)-counting simulation if whenever $R(s,t)$:

(A) If s is final, then t is final.

(B) For every transition $s \to s'$, one of the following holds:

(i) There exist k, z such that $s' \longrightarrow^k z$ and z is stuck.

(ii) There exist k, l, h_1, h_2 such that

$$s' \longrightarrow^k h_1, \qquad t \longrightarrow^l h_2, \qquad (1+k)\,Q\,l, \qquad \mathsf{UpTo}_{P,S}(R)(h_1, h_2).$$

Lean: `Machine.CountingSimulation`

The leading 1 charges for the already prescribed transition $s \to s'$. Neither k nor l is required to be positive. The stuck alternative is a deliberate part of the definition, consistent with observing only successful termination. The finality condition is stronger than simply allowing the target to reach a final state after some extra steps: it requires the target to be final already when the source is final.

Theorem 4.3 (Soundness of counting simulation)**.** *Suppose M is deterministic, (Q, P, S) is reasonable, and R is a (Q, P, S)-counting simulation. Then*

$$R(s,t) \implies s \preceq_Q t.$$

Lean: `Machine.CountingSimulation.sound`

This is the abstract transition-system form of the paper's Proposition 37. Its proof is a strong induction on the length of a successful source execution. Although simulation provides a step-wise, coinductive style of reasoning, the soundness theorem itself is proved by induction on a finite observation; no coinductive datatype is introduced in this file.

Proof. We prove, simultaneously for all states s, t, z, that if $R(s,t)$, $s \longrightarrow^N z$, and $\mathsf{Final}(z)$, then there are m, w such that $N\,Q\,m$, $t \longrightarrow^m w$, and $\mathsf{Final}(w)$. Use strong induction on N; the induction hypothesis applies to every smaller natural number and every pair of related states.

Base case. If $N = 0$, then $s = z$ is final. Condition (A) makes t final. Choose the zero-step execution at t and use $0\,Q\,0$.

Positive case. Write $N = n + 1$ and decompose the successful execution:

$$s \to s' \longrightarrow^n z.$$

Apply condition (B) of simulation. Its stuck alternative would give a finite execution from s' to a stuck state. This is impossible by the lemma that a state on a successful execution cannot be stuck, applied to the successful execution $s' \longrightarrow^n z$.

Consequently there are k, l, h_1, h_2, u, v satisfying

$$\begin{gathered} s' \longrightarrow^k h_1, \qquad t \longrightarrow^l h_2, \qquad (1+k)\,Q\,l, \\ h_1 \preceq_P u, \qquad R(u,v), \qquad v \preceq_S h_2. \end{gathered}$$

The prefix lemma, using determinism and finality of z, gives a remainder length r such that

$$n = k + r, \qquad h_1 \longrightarrow^r z.$$

Apply $h_1 \preceq_P u$ to this remaining successful execution. It supplies a, z_a with

$$r\,P\,a, \qquad u \longrightarrow^a z_a, \qquad \mathsf{Final}(z_a).$$

The decrease condition of reasonableness gives $a \le r$. Since $N = 1 + k + r$, we have $a < N$. This is the strict inequality needed to invoke the strong induction hypothesis. The proof would not be justified for arbitrary auxiliary cost preorders.

Applying that hypothesis to $R(u,v)$ gives b, z_b such that

$$a\,Q\,b, \qquad v \longrightarrow^b z_b, \qquad \mathsf{Final}(z_b).$$

Now apply $v \preceq_S h_2$. We obtain c, z_c with

$$b\,S\,c, \qquad h_2 \longrightarrow^c z_c, \qquad \mathsf{Final}(z_c).$$

The sandwich condition combines $r\,P\,a$, $a\,Q\,b$, and $b\,S\,c$ to yield $r\,Q\,c$. Additivity of Q then gives

$$(1 + k + r)\,Q\,(l + c), \quad \text{that is,} \quad N\,Q\,(l + c).$$

Concatenate $t \longrightarrow^l h_2$ with $h_2 \longrightarrow^c z_c$. The result is a successful execution of length $l + c$ from t, which supplies the required witnesses. □

The code follows these witnesses exactly: its `count` is $n = k + r$, `smaller` is $a < n + 1$, `rc` is $r\,Q\,c$, and `total` is $(n + 1)\,Q\,(l + c)$. The inclusion $P \subseteq S$ is not needed in this particular proof; it is part of the shared reasonable triple and will be used by the specimen argument.

5 Typed observations and contextual lifting

Definition 5.1 (Observation interface)**.** Let J be a type of interfaces, and for each $i \in J$ let F_i be a type of fragments and C_i a type of contexts. An observation interface consists of a predicate $\mathsf{Allowed}(c)$ for contexts and a relation

$$\mathsf{Obs}(c, a, s), \qquad c \in C_i,\ a \in F_i,\ s \in X.$$

The intended reading is that placing a in c and initializing the result can produce state s.

Lean: `Observations`

The common index i prevents a fragment and a context of incompatible interfaces from being compared. Observation is a relation rather than a total function. It can be undefined, and the structure does not require uniqueness of the produced state. The current code does not yet define it by graph plugging.

Definition 5.2 (Contextual refinement and equivalence)**.** For $a, b \in F_i$, write $a \preceq_Q^{\mathcal{O}} b$ if for every $c \in C_i$ and states s, t,

$$\mathsf{Allowed}(c) \ \wedge\ \mathsf{Obs}(c, a, s) \ \wedge\ \mathsf{Obs}(c, b, t) \implies s \preceq_Q t.$$

Contextual equivalence $a \simeq_Q^{\mathcal{O}} b$ is the conjunction of this refinement and the refinement in the opposite direction.

Lean: `Observations.Refines`

Lean: `Observations.Equiv`

The quantifiers compare every pair of observations of the two fragments in the same allowed context. The definition imposes no existence obligation when either fragment has no observation there. Also, state-refinement reflexivity alone does not establish contextual reflexivity for this fully general interface: two observations of the same fragment might give different states. A concrete, functional initialization relation can supply the missing property, but it is not assumed here.

Proposition 5.3 (Contextual refinement from a lifted relation)**.** *Let T_i be a relation on F_i for each interface i, and let L be a relation on states. Suppose:*

1. *Whenever $T_i(a, b)$ and c is allowed, observations $\mathsf{Obs}(c, a, s)$ and $\mathsf{Obs}(c, b, t)$ imply $L(s, t)$.*
2. *$L(s, t)$ implies $s \preceq_Q t$.*

Then $T_i(a, b)$ implies $a \preceq_Q^{\mathcal{O}} b$.

Lean: `Observations.contextual_of_lifting`

Proof. Fix $T_i(a, b)$, an allowed c, and the two observations. The first hypothesis, named `initial_lift` in Lean, puts their initial states in L. The second hypothesis, named `sound`, then gives the state refinement required by the definition of contextual refinement. □

This isolates the logical step of the paper's Proposition 38. Its concrete proof still has to establish the initial-specimen hypothesis for actual plugging and initialization. The proposition above neither asserts nor requires closure under plugging at this abstract level.

Proposition 5.4 (Contextual equivalence from a lifted relation)**.** *Under the same initial-lifting hypothesis, if $L(s,t)$ implies $s \simeq_Q t$, then $T_i(a,b)$ implies $a \simeq_Q^{\mathcal{O}} b$.*

Lean: `Observations.contextual_equiv_of_lifting`

Proof. For the forward refinement, apply the preceding proposition using the forward component of state equivalence. For the reverse refinement, fix observations of b and a in an allowed context. Put them into `initial_lift` in the order a, b, obtaining L from the state of a to the state of b. Use the reverse component of its state equivalence. This does not require that T_i or L be symmetric. □

6 The abstract specimen interface

Definition 6.1 (Focus modes)**.** The type of modes has exactly three constructors: search, backtrack, and rewrite.

Lean: `Focus`

They represent the paper's searching, backtracking, and triggering focuses. The derived decidable equality allows their distinctness to be checked by finite case analysis. This datatype records only a mode, not a graph position or a focus edge.

Definition 6.2 (Specimen system)**.** A specimen system G over M consists of a type Σ of specimens, maps $\ell, r : \Sigma \to X$, a mode map $\mathsf{mode} : \Sigma \to \mathsf{Focus}$, a predicate Enter on specimens, and a predicate Root on states. Its laws are:

1. *Rootedness preservation:* $\mathsf{Root}(s)$ and $s \to t$ imply $\mathsf{Root}(t)$.
2. *Final-state agreement:* for every $c \in \Sigma$, $\mathsf{Final}(\ell(c)) \iff \mathsf{Final}(r(c))$.
3. *Context-only traversal:* suppose both endpoints of c are rooted and c is either backtracking or searching without entering. If either endpoint has an outgoing transition, there is a specimen d whose focus is remote and such that

$$\ell(c) \to \ell(d), \qquad r(c) \to r(d).$$

The terms "rooted," "entering," and "remote" receive their precise abstract meanings from these data and the definitions immediately below.

Lean: `SpecimenSystem`

The corresponding fields are `Specimen`, `left`, `right`, `mode`, `entering`, `rooted`, `rooted_step`, `same_final`, and `context_step`. The last three are hypotheses needed to construct an instance, not results proved here from a graph model. The intended specimen consists of a multi-hole focussed context and two pointwise related vectors of fragments. That intended representation is not a field of this abstract structure.

In particular, the interface does not identify modes with a focus field on states, define rootedness as reachability from an initial state, or define entering by incidence with a hole. The context-traversal law captures the consequence of graph locality and output-closure that the simulation proof uses. Its availability premise is symmetric between the two endpoints, which will be essential in the converse proof.

Definition 6.3 (Remote, settled, and lifted specimens)**.** A specimen c is remote if

$$\mathsf{Remote}(c) \iff \mathsf{mode}(c) \neq \mathsf{rewrite} \ \vee \ \neg\mathsf{Enter}(c).$$

It is settled if

$$\mathsf{Settled}(c) \iff \mathsf{mode}(c) = \mathsf{backtrack} \ \vee \ (\mathsf{mode}(c) = \mathsf{search} \wedge \neg\mathsf{Enter}(c)).$$

Its lifted state relation is

$$\mathcal{L}_G(s,t) \iff \exists c \in \Sigma \ \mathsf{Remote}(c) \wedge \mathsf{Root}(\ell(c)) \wedge \mathsf{Root}(r(c)) \wedge s = \ell(c) \wedge t = r(c).$$

Lean: `SpecimenSystem.Remote`

Lean: `SpecimenSystem.Settled`

Lean: `SpecimenSystem.Lifting`

Remote excludes only an entering rewrite focus. It allows an entering search focus. Settled is stronger: it excludes every rewrite focus and also excludes an entering search focus. The two equalities in the lifting definition expose a particular specimen as a witness for the endpoints.

Lemma 6.4 (Settled implies remote)**.** *If* $\mathsf{Settled}(c)$*, then* $\mathsf{Remote}(c)$*.*

Lean: `SpecimenSystem.settled_remote`

Proof. A backtracking or searching mode is distinct from the rewrite mode. Therefore the first disjunct in the remote predicate holds in either case. □

Lemma 6.5 (Rootedness along a finite execution)**.** *If* $\mathsf{Root}(s)$ *and* $s \longrightarrow^n t$*, then* $\mathsf{Root}(t)$*.*

Lean: `SpecimenSystem.rooted_steps`

Proof. Induct on the execution. At length zero nothing changes. For a positive execution, the one-step rootedness law makes its first successor rooted; the induction hypothesis propagates this property through the remainder. □

Definition 6.6 (Abstract non-rewrite quasi-specimen)**.** For relations A, B on states, define $\mathsf{Quasi}_G(A,B)(s,t)$ to mean that *if* s and t are rooted, then there is a specimen c with

$$\mathsf{mode}(c) \neq \mathsf{rewrite}, \qquad \mathsf{Root}(\ell(c)), \quad \mathsf{Root}(r(c)),$$
$$A(s,\ell(c)), \qquad B(r(c),t).$$

Lean: `SpecimenSystem.Quasi`

The outer implications matter: no witness is required if either endpoint is unrooted. Subsequent proofs establish both rootedness hypotheses before using a quasi-specimen. Unlike the paper's full definition, this abstract consequence does not compare focus labels of s, t with that of c; there is no state-focus map in the interface. It specifically packages the non-rewrite situation used by safety and robustness.

Lemma 6.7 (A rooted quasi-specimen supplies an up-to witness)**.** *If* $\mathsf{Quasi}_G(\preceq_P, \preceq_S)(s,t)$ *and both* s,t *are rooted, then* $\mathsf{UpTo}_{P,S}(\mathcal{L}_G)(s,t)$*.*

Lean: `SpecimenSystem.quasi_upTo`

Proof. Obtain its witness c. Its non-rewrite mode makes it remote. Together with rootedness of its endpoints, this gives $\mathcal{L}_G(\ell(c), r(c))$. Use $\ell(c)$ and $r(c)$ as the intermediate states in the up-to definition, with the two refinements furnished by the quasi-specimen. □

Definition 6.8 (Input-safety)**.** The predicate $\mathsf{InputSafe}_G(Q,P)$ requires the following for every specimen c whose endpoints are rooted, whose mode is search, and which is entering. At least one of these three cases must hold:

(I) Both endpoints have finite executions to stuck states. Their counts may be unrelated, and either count may be zero.

(II) For some settled specimen d and $k,l \in \mathbb{N}$,

$$\ell(c) \longrightarrow^{1+k} \ell(d), \quad r(c) \longrightarrow^{l} r(d), \quad (1+k)\,Q\,l.$$

(III) For some states a,b and $k,l \in \mathbb{N}$,

$$\ell(c) \longrightarrow^{1+k} a, \quad r(c) \longrightarrow^{1+l} b, \quad (1+k)\,Q\,(1+l),$$

and $\mathsf{Quasi}_G(\simeq_P, \simeq_P)(a,b)$ holds.

Lean: `SpecimenSystem.InputSafe`

Case II requires a positive left execution but permits zero right steps. Case III requires positive executions on both sides and auxiliary *equivalences*. Retaining both directions of those equivalences is necessary for reusing safety in the converse theorem.

Definition 6.9 (Robustness)**.** The predicate $\mathsf{Robust}_G(Q,P,S)$ requires that for every specimen c with rooted endpoints, rewrite mode, and $\neg\mathsf{Enter}(c)$, each transition $\ell(c) \to s'$ satisfies one of:

(i) s' has a finite execution to a stuck state;

(ii) for some k,l,a,b,

$$s' \longrightarrow^{k} a, \quad r(c) \longrightarrow^{l} b, \quad (1+k)\,Q\,l,$$

and $\mathsf{Quasi}_G(\preceq_P, \preceq_S)(a,b)$ holds.

Lean: `SpecimenSystem.Robust`

This is a rooted, remote restriction of the consequences used from the paper's robustness condition. The paper's alternative that an endpoint is unrooted is contradictory under the present premises. The abstract machine does not classify its transition relation into search and rewrite rules; it is the specimen's mode that determines where this predicate is invoked. Establishing the correspondence with concrete rewrite transitions is part of the missing hypernet instance.

7 The forward robustness argument

Throughout this section M is deterministic, (Q, P, S) is reasonable, and G is a specimen system satisfying $\mathsf{InputSafe}_G(Q, P)$ and $\mathsf{Robust}_G(Q, P, S)$. In particular, all the interface laws of Definition 6.2 are available.

Theorem 7.1 (Forward construction of a counting simulation)**.** *The relation $\mathcal{L}_G$ is a (Q, P, S)-counting simulation.*

Lean: `SpecimenSystem.countingSimulation`

This is an abstract form of the forward clause of the paper's Proposition 40. The proof checks the two simulation fields directly.

Proof. Take $\mathcal{L}_G(s, t)$ and choose a witnessing specimen c. Thus $s = \ell(c)$, $t = r(c)$, the specimen is remote, and both endpoints are rooted.

Finality. If s is final, final-state agreement for c makes t final. This verifies simulation condition (A).

A common context-traversal case. Suppose c is settled and there is a prescribed transition $s \to s'$. The context-traversal law gives a remote specimen d and transitions

$$\ell(c) \to \ell(d), \qquad r(c) \to r(d).$$

Determinism identifies s' with $\ell(d)$. Both new endpoints are rooted, so $\mathcal{L}_G(\ell(d), r(d))$. The simulation matching case can use zero further left steps and one right step. Its cost obligation is $(1 + 0)\, Q\, 1$, which is reflexivity of Q. To obtain the up-to relation, choose $\ell(d), r(d)$ as intermediate states and insert reflexive refinements for P and S on either side of their lifted relation. These are the local proof witnesses named `context_case` and `settled_case` in Lean.

It remains to cover all possible modes of c.

Backtracking. Such a specimen is settled by definition, so the common context-traversal argument applies.

Searching without entering. This is the other settled case and uses the same argument.

Entering search. Apply input-safety, obtaining one of its three cases.

(I) There is a stuck execution from $\ell(c)$. Since $\ell(c) \to s'$ is available, the stuck-after-step lemma removes that first step and gives a stuck execution from s'. This is the stuck alternative of simulation. Safety also supplies a stuck execution on the right, but the forward simulation condition does not need it.

(II) There are d, k, l with d settled,

$$\ell(c) \longrightarrow^{1+k} \ell(d), \qquad r(c) \longrightarrow^{l} r(d), \qquad (1 + k)\, Q\, l.$$

Remove the prescribed first transition from the left execution using determinism. The resulting execution from s' to $\ell(d)$ has length k. Rootedness propagates to both endpoints of d, and settledness implies remoteness. Hence $\mathcal{L}_G(\ell(d), r(d))$. Add reflexive auxiliary refinements as in the context-traversal case. The matching counts are k, l, with the cost condition already supplied by safety.

(III) There are k, l, a, b with the positive executions and quasi-specimen of safety III. Remove the prescribed first step from the left execution, giving $s' \longrightarrow^{k} a$; retain the right

execution of length $1 + l$. Both a and b are rooted. Unpack the quasi-specimen to get a non-rewrite specimen d with rooted endpoints and

$$a \simeq_P \ell(d), \qquad r(d) \simeq_P b.$$

Use the forward half of the first equivalence to obtain $a \preceq_P \ell(d)$. The forward half of the second gives $r(d) \preceq_P b$, and monotonicity with $P \subseteq S$ changes this to $r(d) \preceq_S b$. Thus the quasi-up-to lemma supplies $\mathsf{UpTo}_{P,S}(\mathcal{L}_G)(a, b)$. The matching counts are $k, 1 + l$, and their required cost relation is exactly $(1 + k)\, Q\, (1 + l)$.

Rewrite mode. Remoteness now implies $\neg\mathsf{Enter}(c)$, since the other remote disjunct contradicts the current mode. Apply robustness to the prescribed transition $\ell(c) \to s'$. If it provides a stuck execution from s', use the stuck alternative of simulation. Otherwise it provides matching executions of lengths k, l, the cost proof $(1+k)\, Q\, l$, and a quasi-specimen up to $\preceq_P, \preceq_S$. Its left endpoint is rooted by the prescribed step followed by the k-step execution; its right endpoint is rooted by its l-step execution. The quasi-up-to lemma therefore supplies the required up-to relation. This completes every mode and every simulation condition. □

Theorem 7.2 (Forward state sufficiency)**.** *Under the hypotheses of this section,*

$$\mathcal{L}_G(s, t) \implies s \preceq_Q t.$$

Lean: `SpecimenSystem.sufficiency`

Proof. The preceding theorem constructs a counting simulation on $\mathcal{L}_G$. Apply counting-simulation soundness, Theorem 4.3, with the same deterministic machine and reasonable triple. □

All graph-specific information in this theorem is supplied through the specimen system and the safety and robustness premises. The theorem's short proof is therefore not evidence that the missing graph-locality verification is automatic.

8 Opposite specimens and the converse argument

Definition 8.1 (Opposite specimen system)**.** The opposite system G^{op} has the same specimens, modes, entering predicate, and rooted-state predicate as G. It swaps the two endpoint maps:

$$\ell_{G^{\mathrm{op}}}(c) = r_G(c), \qquad r_{G^{\mathrm{op}}}(c) = \ell_G(c).$$

Lean: `SpecimenSystem.opposite`

Verification of its interface laws. Rootedness preservation is unchanged. Final-state agreement is the reverse implication pair of the original agreement. For context-only traversal, exchange the two rootedness hypotheses and the two alternatives of the availability premise. Apply the original law to get a remote specimen d and two transitions. Return those transitions in the opposite order. This explains why the interface law permits availability on either side, rather than only on the left. □

Lemma 8.2 (Lifting commutes with taking the opposite)**.** *For all states s, t,*

$$\mathcal{L}_{G^{\mathrm{op}}}(s,t) \iff \mathcal{L}_G(t,s).$$

Lean: `SpecimenSystem.lifting_opposite`

Proof. In either direction retain the witnessing specimen and its remoteness. Exchange the two rootedness proofs and the two endpoint equalities. The resulting tuple is precisely the witness required by the other lifting definition. □

Theorem 8.3 (Converse construction of a counting simulation)**.** *Suppose M is deterministic, (Q, P, S) is reasonable, $\mathsf{InputSafe}_G(Q^{-1}, P)$ holds, and G^{op} is robust for (Q, P, S). Then $\mathcal{L}_{G^{\mathrm{op}}}$ is a (Q, P, S)-counting simulation.*

Lean: `SpecimenSystem.countingSimulation_converse`

The safety premise is a property of the *original* system at the converse cost relation. It is not assumed that the opposite system already satisfies input-safety. This distinction is important when the right execution in safety II has length zero.

Proof. Choose a witness c of $\mathcal{L}_{G^{\mathrm{op}}}(s,t)$, so $s = r(c)$, $t = \ell(c)$, c is remote, and both endpoints are rooted. Finality of $r(c)$ implies finality of $\ell(c)$ by the reverse direction of final-state agreement. This establishes condition (A). For condition (B), fix a transition $r(c) \to s'$.

Settled case. If c is settled, the symmetric availability premise of context-only traversal applies to this right transition. Obtain a remote specimen d with transitions

$$\ell(c) \to \ell(d), \qquad r(c) \to r(d).$$

Determinism gives $s' = r(d)$. Propagate rootedness, and obtain $\mathcal{L}_{G^{\mathrm{op}}}(r(d), \ell(d))$. Choose zero further steps from s' and one step from $\ell(c)$. The cost condition is $1\, Q\, 1$, and reflexive refinements give the up-to witness. This covers backtracking and non-entering search.

Entering search. Input-safety for Q^{-1}, P supplies three cases, which must be read in the original endpoint order.

(I) Use the supplied stuck execution from $r(c)$. The prescribed step $r(c) \to s'$ and the stuck-after-step lemma give a stuck execution from s'. The supplied stuck execution from $\ell(c)$ is not needed in this direction.

(II) There are a settled specimen d and counts k, l with

$$\ell(c) \longrightarrow^{1+k} \ell(d), \qquad r(c) \longrightarrow^{l} r(d), \qquad l\, Q\, (1+k).$$

Both endpoints of d are rooted. We distinguish the two possible forms of l.

If $l = j+1$, remove the prescribed transition from the right execution to get $s' \longrightarrow^{j} r(d)$. Keep the left execution of length $1+k$. The opposite lifted pair is $(r(d), \ell(d))$, and its cost condition is $(1+j)\, Q\, (1+k)$, exactly the supplied relation with $l = j+1$. Reflexive auxiliary refinements finish this subcase.

If $l = 0$, the zero-step equality gives $r(c) = r(d)$. Thus the prescribed transition is an outgoing transition from $r(d)$. Apply context-only traversal to the *settled* specimen d. Obtain a remote specimen e and transitions

$$\ell(d) \to \ell(e), \qquad r(d) \to r(e).$$

Determinism identifies s' with $r(e)$. Propagating rootedness gives the opposite lifted pair $(r(e), \ell(e))$. There are zero further source steps, whereas the target execution is

$$\ell(c) \longrightarrow^{1+k} \ell(d) \to \ell(e),$$

of length $(1+k)+1$. Its cost obligation is $1\,Q\,((1+k)+1)$. The safety relation in this subcase is $0\,Q\,(1+k)$. Add $1\,Q\,1$ using additivity of Q to obtain the required relation. This additional traversal is the essential zero-step correction; the proof never attempts to remove a step from a zero-step execution.

(III) There are k, l, a, b with

$$\ell(c) \longrightarrow^{1+k} a, \qquad r(c) \longrightarrow^{1+l} b, \qquad (1+l)\,Q\,(1+k),$$

and a quasi-specimen for auxiliary equivalences. Remove the prescribed right transition to get $s' \longrightarrow^{l} b$. Both a, b are rooted; unpacking the quasi-specimen yields a non-rewrite specimen d with rooted endpoints and

$$a \simeq_P \ell(d), \qquad r(d) \simeq_P b.$$

This time take the *reverse* halves: $b \preceq_P r(d)$ and $\ell(d) \preceq_P a$. Enlarge the latter auxiliary order using $P \subseteq S$ to obtain $\ell(d) \preceq_S a$. Since $\mathcal{L}_{G^{\mathrm{op}}}(r(d), \ell(d))$, these form

$$\mathsf{UpTo}_{P,S}(\mathcal{L}_{G^{\mathrm{op}}})(b, a).$$

The matching counts are $l, 1+k$, with the already supplied cost relation $(1+l)\,Q\,(1+k)$.

Rewrite mode. Remoteness gives $\neg\mathsf{Enter}(c)$. Apply robustness in G^{op} to the prescribed transition from $r(c)$, which is its left endpoint. The stuck alternative is already the required one. Otherwise the two matching executions and their cost relation are supplied directly. Rootedness propagates along the prescribed transition and both executions. Apply the quasi-up-to lemma *in the opposite system* to obtain the up-to relation on $\mathcal{L}_{G^{\mathrm{op}}}$. This completes the proof. □

9 State and contextual consequences

The following results compose the two simulation constructions with the soundness and observation lemmas. They are short in Lean because their mathematical work has been isolated in those earlier results.

Theorem 9.1 (Converse state sufficiency)**.** *If M is deterministic, (Q, P, S) is reasonable, $\mathsf{InputSafe}_G(Q^{-1}, P)$ holds, and G^{op} is robust for (Q, P, S), then*

$$\mathcal{L}_G(s, t) \implies t \preceq_Q s.$$

Lean: `SpecimenSystem.sufficiency_converse`

Proof. The lifting-opposite lemma changes the premise into $\mathcal{L}_{G^{\mathrm{op}}}(t, s)$. The converse construction supplies a counting simulation on this relation. Apply counting-simulation soundness to obtain the stated refinement. □

Theorem 9.2 (Mutual state refinement)**.** *Suppose M is deterministic, (Q, P, S) is reasonable, G is input-safe for both (Q, P) and (Q^{-1}, P), and both G and G^{op} are robust for (Q, P, S). Then*

$$\mathcal{L}_G(s, t) \implies s \simeq_Q t.$$

Lean: `SpecimenSystem.equivalence`

Proof. Forward state sufficiency supplies $s \preceq_Q t$. Converse state sufficiency supplies $t \preceq_Q s$. These are exactly the two components of state equivalence. □

The main cost relation Q itself need not be symmetric. The two directional refinement statements use the same Q, with source and target counts exchanged when the states are exchanged.

For the contextual results, fix the observation interface of the earlier section and a family T_i of relations on fragments. Assume the following *initial-lifting property*: for $T_i(a, b)$, every allowed context c and observations $\mathsf{Obs}(c, a, s)$, $\mathsf{Obs}(c, b, t)$ give $\mathcal{L}_G(s, t)$. This property is the explicit premise `initial_lift` in the source. It connects the abstract specimen system with the separately supplied observation relation.

Theorem 9.3 (Forward contextual sufficiency)**.** *If M is deterministic, (Q, P, S) is reasonable, $\mathsf{InputSafe}_G(Q, P)$ and $\mathsf{Robust}_G(Q, P, S)$ hold, and the initial-lifting property holds, then*

$$T_i(a, b) \implies a \preceq_Q^{\mathcal{O}} b.$$

Lean: `SpecimenSystem.contextual_sufficiency`

Proof. Use contextual refinement from a lifted relation, taking $L = \mathcal{L}_G$. Its initial-lifting premise is precisely the one assumed here. Its state-soundness premise is forward state sufficiency. Applying that proposition to $T_i(a, b)$ gives the desired contextual refinement. □

Theorem 9.4 (Converse contextual sufficiency)**.** *If M is deterministic, (Q, P, S) is reasonable, $\mathsf{InputSafe}_G(Q^{-1}, P)$ holds, G^{op} is robust for (Q, P, S), and the same initial-lifting property holds, then*

$$T_i(a, b) \implies b \preceq_Q^{\mathcal{O}} a.$$

Lean: `SpecimenSystem.contextual_sufficiency_converse`

Proof. Fix $T_i(a, b)$, an allowed context, an observation of b producing s, and an observation of a producing t. Supply the observations to initial lifting in the original a, b order to get $\mathcal{L}_G(t, s)$. Converse state sufficiency then gives $s \preceq_Q t$, the required contextual comparison. □

These are conditional abstract counterparts of the two clauses of the paper's Theorem 6. Refocusing, output-closure, type-correct plugging, and the paper's concrete specimen definitions have not disappeared from the mathematics: their required consequences are currently represented by the specimen-system laws and initial-lifting premise. Instantiating those hypotheses is the remaining concrete formalization problem.

10 Semantic examples and regression proofs

The examples are small abstract machines used to check the meaning and edge cases of the definitions. They do not formalize the paper's lambda-calculus translation, copying UAM, or program-equivalence examples.

10.1 A countdown machine

Definition 10.1 (Countdown)**.** The countdown machine has state set $\mathbb{N}$, transition $s \to t$ if $s = t + 1$, and final predicate $s = 0$.

Lean: `Examples.countdown`

Verification of the machine law. If $s = 0$, an outgoing transition would imply $0 = t + 1$, impossible for a natural number t. Thus final states are normal as required by the machine structure. □

Lemma 10.2 (Countdown is deterministic)**.** *If $s \to a$ and $s \to b$, then $a = b$.*

Lean: `Examples.countdown_deterministic`

Proof. The premises give $s = a + 1$ and $s = b + 1$. Cancellation of successor gives $a = b$. □

Lemma 10.3 (Canonical countdown execution)**.** *For every $n \in \mathbb{N}$, the countdown machine has $n \longrightarrow^n 0$.*

Lean: `Examples.countdown_run`

Proof. Induct on n. For $n = 0$, use the zero-step constructor. For $n + 1$, take the transition $n + 1 \to n$ and prepend it to the induction hypothesis $n \longrightarrow^n 0$. □

Proposition 10.4 (Countdown characterizes the cost relation)**.** *For every cost preorder Q and states $a, b \in \mathbb{N}$,*

$$a \preceq_Q b \quad \Longleftrightarrow \quad a \, Q \, b.$$

Lean: `Examples.countdown_refines_iff`

Proof. Suppose $a \preceq_Q b$. Apply it to the canonical successful execution $a \longrightarrow^a 0$. It gives a successful execution of some length m from b and $a \, Q \, m$. Compare this execution with $b \longrightarrow^b 0$. Determinism and final-execution uniqueness give $m = b$, so $a \, Q \, b$.

Conversely assume $a \, Q \, b$. For any successful execution of length n from a, compare it with $a \longrightarrow^a 0$. Uniqueness gives $n = a$. Choose $b \longrightarrow^b 0$ as the target execution; its cost condition is $n \, Q \, b$, obtained by replacing n with a. □

The file then proves three anonymous examples, each by the proposition and the indicated numerical fact:

1. $5 \preceq_I 3$, because $3 \leq 5$.
2. $3 \not\preceq_I 5$, because $5 \not\leq 3$.

3. $5 \not\preceq_E 3$, because $5 \neq 3$.

Lean: `Examples: improvement from 5 to 3`

Lean: `Examples: no improvement from 3 to 5`

Lean: `Examples: unequal exact counts`

These verify that improvement uses the target count as the smaller quantity. Reversing the intended cost orientation would invalidate the first two examples.

10.2 A crash and an infinite loop

Definition 10.5 (Fault states and the fault machine)**.** There are four distinct states:

$$\mathsf{done}, \quad \mathsf{crash}, \quad \mathsf{stuck}, \quad \mathsf{loop}.$$

The only transitions are

$$\mathsf{crash} \to \mathsf{stuck}, \qquad \mathsf{loop} \to \mathsf{loop}.$$

Only done is final.

Lean: `Examples.FaultState`

Lean: `Examples.fault`

The four names are constructors of a finite datatype with derived decidable equality. Here the constructor stuck names a particular state; $\mathsf{Stuck}(s)$ remains the general predicate defined earlier.

Verification of the machine law. No transition has source done, because both possible source constructors are distinct from it. Consequently a final state has no outgoing transition. □

Lemma 10.6 (The fault machine is deterministic)**.** *Any two transitions with the same source have the same target.*

Lean: `Examples.fault_deterministic`

Proof. Case-analyze the two transition proofs. If both use the crash rule, both targets are stuck. If both use the loop rule, both targets are loop. Either mixed case would equate crash with loop and is impossible. □

Definition 10.7 (The crash–loop candidate)**.** Let $R(s,t)$ mean $s = \mathsf{crash}$ and $t = \mathsf{loop}$.

Lean: `Examples.crashLoop`

Lemma 10.8 (A simulation using the stuck alternative)**.** *The crash–loop candidate is an* (E,E,E)*-counting simulation.*

Lean: `Examples.crash_loop_simulation`

Proof. For its only related pair, the source is crash, which is not final; hence the finality condition is vacuous. Its only successor is stuck. Use the zero-step execution from that successor to itself in the stuck alternative of simulation. It is normal because neither transition rule starts there, and it is not final because it differs from done. □

The next anonymous example concludes $\mathsf{crash} \preceq_E \mathsf{loop}$.

Lean: `Examples: crash refines loop`

Proof. Apply counting-simulation soundness to the preceding simulation, fault determinism, the reasonable triple (E,E,E), and the only pair of R. □

There is no requirement to match a successful source execution, because none exists. Thus the result is consistent even with exact costs. The example exercises the stuck branch of the implemented proof; it does not prove that every conceivable up-to simulation proof of this same refinement must use that branch.

10.3 The zero-step converse case

Definition 10.9 (Visit specimens and endpoint maps)**.** The type of specimens has three distinct constructors $\mathsf{entering}, \mathsf{settled}, \mathsf{finished}$, with derived decidable equality. The endpoint maps into the countdown machine are

$$\begin{aligned} \ell(\mathsf{entering}) &= 2, \quad r(\mathsf{entering}) = 1, \\ \ell(\mathsf{settled}) &= 1, \quad r(\mathsf{settled}) = 1, \\ \ell(\mathsf{finished}) &= 0, \quad r(\mathsf{finished}) = 0. \end{aligned}$$

Lean: `Examples.Visit`

Lean: `Examples.visitLeft`

Lean: `Examples.visitRight`

Definition 10.10 (Visit specimen system)**.** Every visit specimen has search mode. The entering predicate holds exactly of the constructor entering. Every state is declared rooted. Together with the endpoint maps above, these data define a specimen system over countdown.

Lean: `Examples.visitSystem`

Verification of the interface laws. Rootedness preservation is immediate since rootedness is the true predicate. Final-state agreement holds because the endpoint pairs are $(2, 1)$, $(1, 1)$, and $(0, 0)$: only in the third pair is either endpoint zero, and then both are.

For context-only traversal, consider the specimen. The constructor entering cannot satisfy the settled premise: its mode is search and its entering predicate holds. At the constructor settled, both endpoints are 1. Choose finished and the countdown transitions $1 \to 0$ on both sides; its search mode is remote. At finished, both endpoints are zero and neither has an outgoing transition, contradicting the availability premise. These cases exhaust the datatype. □

This is a deliberately abstract instance. For example, all its modes are search, including that of the pair of final states. The specimen interface does not enforce a concrete UAM convention for final focus modes.

Lemma 10.11 (Visit input-safety)**.** *The visit system is input-safe for* (H^{-1}, E).

Lean: `Examples.visit_safe`

Proof. The entering premise forces the input specimen to be entering. Use safety II with output specimen settled and $k = l = 0$. The left execution is the single countdown transition $2 \to 1$; the right execution has zero steps at 1. The output specimen is searching without entering and is therefore settled. The cost requirement is $1\, H^{-1}\, 0$, or $0 \leq 1$. □

Lemma 10.12 (Visit converse robustness)**.** *The opposite visit system is robust for* (H, E, E).

Lean: `Examples.visit_converse_robust`

Proof. Its mode is always search. The rewrite-mode premise of robustness is therefore impossible, so every required implication holds vacuously. □

Proposition 10.13 (The converse zero-step regression)**.** *In countdown,* $1 \preceq_H 2$.

Lean: `Examples.converse_zero_step_case`

Proof. Use converse state sufficiency for the visit system. Countdown is deterministic, (H, E, E) is reasonable, and the preceding two lemmas supply its safety and converse robustness premises. The entering specimen is remote, both its endpoints are rooted, and it witnesses $\mathcal{L}_G(2, 1)$. The theorem consequently gives $1 \preceq_H 2$.

In the underlying converse simulation construction, safety II provides the pair $(1, 1)$ after one original left step and zero original right steps. The prescribed right step is $1 \to 0$. Context traversal at $(1, 1)$ supplies $(0, 0)$ and adds one transition to the original left execution, now $2 \to 1 \to 0$. Thus the source terminates in one step and the target in two, satisfying $1\, H\, 2$. This is precisely the zero-step subcase proved in the converse argument. □

11 Finite carriers and concrete interfaced graphs

We now turn to actual graph objects. The constructions in the following sections prove their own incidence and typing obligations. They do not assume the specimen laws used in the preceding abstract argument. We work with small finite vertex and edge carriers; operation labels may live in a larger universe.

11.1 Enumerations and bijections

Definition 11.1 (Finite enumeration)**.** An enumeration of a type A consists of a finite list l and a proof that every $a \in A$ belongs to l. Repetitions are allowed. This witnesses finiteness without requiring decidable equality or a canonical choice of names.

Lean: `Enumeration`

The empty type is enumerated by the empty list: completeness follows because there is no element to check. The finite ordinal $\{0, \ldots, n-1\}$ is enumerated in increasing order by Lean's finite range; its library membership theorem proves completeness, including when $n = 0$.

Lean: `Enumeration.empty`

Lean: `Enumeration.fin`

For a disjoint union $A + B$, concatenate the tagged enumerations of A and B. An element has exactly one tag, and completeness of the corresponding list places it in the appropriate half of the concatenation.

Lean: `Enumeration.sum`

For a subset $\{a \in A \mid p(a)\}$, attach membership certificates to the enumeration of A and retain just entries satisfying p. Classical decidability of p allows this filtering. Every subset element is retained because its certificate proves the test true. The resulting definition is noncomputable for an arbitrary proposition p; no algorithm deciding p is claimed.

Lean: `Enumeration.subtype`

For a dependent sum $\sum_{a \in A} B_a$, enumerate each fibre, tag it with a, and concatenate over the enumeration of A. A pair (a, b) appears because a occurs in the outer list and b in the enumeration of its fibre.

Lean: `Enumeration.sigma`

Definition 11.2 (Bijection)**.** A bijection $f : A \cong B$ consists of functions $f : A \to B$ and $f^{-1} : B \to A$ with both inverse equations, pointwise.

Lean: `Bijection`

Identity uses the identity function twice. Inversion interchanges the two functions and their inverse laws. Composition uses $g \circ f$ forwards and $f^{-1} \circ g^{-1}$ backwards; cancelling the inner inverse pair and then the outer pair proves each law.

Lean: `Bijection.refl`

Lean: `Bijection.symm`

Lean: `Bijection.trans`

Bijections extend to disjoint unions by applying the appropriate map inside each tag. Case analysis on the tag reduces both inverse laws to the component laws.

Lean: `Bijection.sum`

Every bijection is injective: apply its inverse to an equality of images and cancel. An enumeration transports along a bijection by mapping its forward function over the list. For $b \in B$, the listed preimage $f^{-1}(b)$ maps to b, proving completeness.

Lean: `Bijection.injective`

Lean: `Bijection.enumeration`

Lemma 11.3 (Distinctness under restricted injectivity)**.** *If a list l has no repeated elements and f is injective on elements of l, then* $\mathrm{map}(f, l)$ *has no repeated elements.*

Proof. Distinct positions in l contain distinct elements. If their images were equal, restricted injectivity would identify those elements, a contradiction. Lean expresses this as preservation of the pairwise inequality predicate under mapping. □

Lean: `nodup_map_on`

This restricted version will be essential: replacement can identify an input with an output, while remaining injective on each incidence list separately.

11.2 Ports, signatures, and graph invariants

Definition 11.4 (Signature and ports)**.** A signature σ assigns an ordered input-label list and output-label list to every operation. With internal carrier V and boundary lengths n, m, vertices are the disjoint union

$$\mathrm{Port}(V, n, m) = V + \mathrm{Fin}(n) + \mathrm{Fin}(m).$$

The three constructors are internal, input, and output. In particular the two boundary sets are disjoint even if their wire labels agree.

Lean: `Signature`

Lean: `Port`

The predicates IsInput and IsOutput test the corresponding constructor. A carrier map $f : V \to W$ acts only on internal vertices and fixes each boundary position. It is injective whenever f is: unlike constructors cannot have equal images, and equality within the internal constructor reduces to injectivity of f. The input and output predicates are preserved and reflected by this map, by the same three-case analysis.

Lean: `Port.IsInput`

Lean: `Port.IsOutput`

Lean: `Port.map`

Lean: `Port.map_injective`

Lean: `Port.map_isInput`

Lean: `Port.map_isOutput`

Given boundary-label lists I, O and internal labelling $\lambda : V \to L$, the label of an internal vertex is $\lambda(v)$, of input i is I_i, and of output j is O_j. The finite-index types ensure that these lookups are defined.

Lean: `Port.label`

Definition 11.5 (Concrete typed net)**.** A net $G : I \to O$ has finite internal and edge carriers, labels for both, and ordered source and target vertex lists $s(e), t(e)$ for each edge. It satisfies:

1. Each individual source list and target list has no repetitions.
2. A vertex belongs to the sources of at most one edge, and to the targets of at most one edge.
3. A vertex belongs to some source list exactly when it is not an output; it belongs to some target list exactly when it is not an input.
4. Mapping vertex labels over $s(e)$ and $t(e)$ gives precisely the input and output lists of the signature of the edge label.

Lean: `Net`

These fields are part of a net's certificate of well-formedness. Coverage excludes isolated vertices. It does not exclude cycles or a vertex belonging to both the source and target of the same edge. The full vertex type and its label function are abbreviations for the port type and the labelling just defined.

Lean: `Net.Vertex`

Lean: `Net.vertexLabel`

From forward source coverage, membership in $s(e)$ implies that a vertex is not an output. From target coverage, membership in $t(e)$ implies that it is not an input. Apply the appropriate equivalence to the witness edge e.

Lean: `Net.source_not_output`

Lean: `Net.target_not_input`

Taking lengths in the two typing equations shows that incidence-list lengths are exactly the corresponding signature arities: mapping labels does not change length.

Lean: `Net.source_length`

Lean: `Net.target_length`

The ith source port of e is its source list indexed by a position in the signature's input list; the preceding length equality justifies the lookup. Target ports are defined in the same way with the output list.

Lean: `Net.sourcePort`

Lean: `Net.targetPort`

Each indexed port belongs to its incidence list, by the list lookup membership lemma. Within either list, equality of indexed ports implies equality of indices, because that list has no duplicates; equality of finite ordinals follows from equality of their underlying natural numbers.

Lean: `Net.sourcePort_mem`

Lean: `Net.targetPort_mem`

Lean: `Net.sourcePort_injective`

Lean: `Net.targetPort_injective`

The label of source port i is the ith signature input label, and the label of target port i is the ith signature output label. Apply indexed lookup to the appropriate list typing equation, then commute lookup with mapping.

Lean: `Net.sourcePort_label`

Lean: `Net.targetPort_label`

Finally, membership in a source list is equivalent to being its indexed port for some signature input position. Given membership, choose its list index and transport the bound along the arity equality. Conversely, indexed ports belong to their lists. The target statement uses the output arity and the identical membership argument.

Lean: `Net.mem_source_iff_port`

Lean: `Net.mem_target_iff_port`

11.3 Elementary graph constructors

The ordered finite range has no duplicates: equality of its entries at two positions gives equality of their underlying natural numbers and hence of the positions.

Lean: `finRange_nodup`

Definition 11.6 (Corolla and empty graph)**.** The corolla of operation o has one edge labelled o, no internal vertices, and all of its prescribed inputs and outputs in signature order. Its sources are precisely the input constructors and its targets the output constructors.

Well-formedness. Finite ranges and their constructor maps give distinct incidence lists. There is only one edge, so edge uniqueness is immediate. Every input occurs in its source range and every output in its target range, and the opposite boundary constructors cannot occur there. There are no internal vertices to check. Mapping labels over either ordered range recovers the corresponding signature list, proving typing. □

Lean: `Net.corolla`

The empty graph has empty internal and edge carriers and empty boundaries. Each incidence or label function has empty domain; all obligations follow by eliminating the purported element of an empty type. A bare identity wire is not provided: a single vertex serving as both boundary input and boundary output would violate the disjoint-boundary representation used here.

Lean: `Net.empty`

12 Shallow plugging, including self-loop holes

Fix a host $C : I \to O$, an edge h to remove, and a replacement H whose boundary lists are exactly the input and output signature lists of C's label at h. All constructions in this section are in the namespace Net.Replacement unless explicitly stated otherwise.

12.1 Carriers and boundary identification

Definition 12.1 (Replacement carriers)**.** The new internal vertices are $C_V + H_V$, with disjoint tags providing freshness. The new edges are $(C_E \setminus \{h\}) + H_E$. The outer boundary remains I, O. The surviving-edge subtype carries a proof of inequality to h, so the removed edge cannot accidentally survive as an old edge.

Lean: `Net.Replacement.Vertex`

Lean: `Net.Replacement.Edge`

The host embedding j tags each old internal vertex on the left and fixes the outer boundary. The insertion map k tags a replacement internal vertex on the right, sends replacement input i to $j(s_h(i))$, and sends replacement output i to $j(t_h(i))$. Thus gluing is expressed by a function rather than a quotient of all vertex names.

Lean: `Net.Replacement.host`

Lean: `Net.Replacement.insert`

The host embedding is injective by injectivity of the left sum tag and the port map lemma. Its image is disjoint from freshly tagged replacement internals: case analysis on a host vertex gives either a boundary constructor or an internal left tag, neither equal to an internal right tag.

Lean: `Net.Replacement.host_injective`

Lean: `Net.Replacement.host_ne_internal`

Lemma 12.2 (The two restricted insertion injections)**.** *The insertion map is injective on vertices that are not outputs. It is also injective on vertices that are not inputs.*

Proof. For the first claim, exclude output cases using the hypotheses. Two internal images agree only when their right-tagged names agree. An internal image cannot agree with an input image, by freshness. Two input images agree only when their host source ports agree, and source-port injectivity identifies their indices. For the second claim, exclude input cases instead, use target-port injectivity on two outputs, and use the same freshness argument on mixed cases. □

Lean: `Net.Replacement.insert_source_injective`

Lean: `Net.Replacement.insert_target_injective`

If a non-output replacement vertex has insertion image $j(v)$, it must be some input i, with $s_h(i) = v$. Outputs are excluded by hypothesis, internals by freshness, and in the input case host injectivity gives the equation. Dually, a non-input replacement vertex mapping to $j(v)$

must be output i with $t_h(i) = v$. These inversion lemmas control mixed old/new incidence cases.

Lean: `Net.Replacement.insert_source_host`

Lean: `Net.Replacement.insert_target_host`

12.2 Preservation of incidence invariants

Sources and targets of surviving host edges are their old lists mapped by j. Those of inserted edges are the replacement lists mapped by k.

Lean: `Net.Replacement.source`

Lean: `Net.Replacement.target`

Lemma 12.3 (Distinct incidence entries)**.** *Every new source list and every new target list has no duplicates.*

Proof. A surviving host list uses the injective map j. A replacement source list contains no outputs, so the first restricted injection of k applies to its members. A replacement target list contains no inputs, so the second injection applies. In all cases use preservation of distinctness under restricted injectivity. □

Lean: `Net.Replacement.source_nodup`

Lean: `Net.Replacement.target_nodup`

Lemma 12.4 (Unique source and target incidence)**.** *A vertex belongs to the sources of at most one new edge, and to the targets of at most one new edge.*

Proof. For source incidence, split on the two edge tags. Two host edges reduce, by injectivity of j, to old source uniqueness. Two inserted edges reduce, by restricted source injectivity of k, to source uniqueness in H. In the mixed case the shared vertex is an inserted input, by the source inversion lemma; its corresponding host vertex is therefore a source port of h. Old source uniqueness identifies the surviving host edge with h, contradicting its subtype certificate. The other mixed ordering is the same contradiction with the equality reversed. For target incidence, repeat the four cases with target uniqueness, restricted target injectivity, and the target inversion lemma: the mixed vertex is now an inserted output and a target port of h. □

Lean: `Net.Replacement.source_unique`

Lean: `Net.Replacement.target_unique`

A vertex in a new source list is not an outer output. In the host case use the old source property and preservation of boundary constructors. In the inserted case an internal vertex remains internal, an input maps to a source port of h, and an output is impossible in a replacement source list. For a new target list, exchange source/input with target/output: inserted inputs are impossible and inserted outputs map to host target ports, which cannot be outer inputs.

Lean: `Net.Replacement.source_not_output`

Lean: `Net.Replacement.target_not_input`

Lemma 12.5 (Coverage of surviving host vertices)**.** *For every host vertex v that is not an output, $j(v)$ belongs to a new source list. If v is not an input, $j(v)$ belongs to a new target list.*

Proof. Choose an old source edge by coverage in C. If it differs from h, retain it. If it equals h, express v as source port i; replacement input i belongs to a source edge by coverage in H, and its image is $j(v)$. For target coverage, choose an old target edge. Retain it unless it is h; in that case express v as target port i and use coverage at replacement output i. □

Lean: `Net.Replacement.source_host_covered`

Lean: `Net.Replacement.target_host_covered`

Full source coverage now follows by splitting a new vertex into a host internal, a fresh internal, an outer input, or an outer output. Host internals and inputs use the preceding lemma. Fresh internals use replacement coverage and map the witness edge into the right edge tag. Outer outputs contradict the required non-output hypothesis. The reverse implication is the source exclusion already proved. Full target coverage uses host internals and outer outputs, replacement coverage for fresh internals, and excludes outer inputs; its reverse implication is target exclusion.

Lean: `Net.Replacement.source_coverage`

Lean: `Net.Replacement.target_coverage`

12.3 Labels, typing, and the certified result

New internal labels use the host label on left tags and replacement label on right tags. Surviving edges keep their labels; inserted edges receive their replacement labels.

Lean: `Net.Replacement.internalLabel`

Lean: `Net.Replacement.edgeLabel`

Labelling commutes with j, by the three port cases. It also commutes with k: internal labels agree by definition, while input and output labels agree by the host indexed-port typing lemmas and the matching replacement interface.

Lean: `Net.Replacement.host_label`

Lean: `Net.Replacement.insert_label`

For source typing, split the edge tag and compose the incidence map with the vertex-label map. The commuting equation just proved reduces the result to source typing in C or H, respectively. For target typing the same composition argument reduces to their target typing equations.

Lean: `Net.Replacement.source_typed`

Lean: `Net.Replacement.target_typed`

Theorem 12.6 (Well-formed shallow plugging)**.** *There is a net $C[h := H] : I \to O$ with the carriers, labels, and incidence lists above. It preserves the entire ordered external interface and all graph invariants, even when h has a vertex in both its source and target lists.*

Proof. Enumerate the new internal carrier by the sum enumeration. Enumerate surviving host edges by subset filtering, and concatenate the tagged replacement-edge enumeration. Supply each of the distinctness, uniqueness, coverage, and typing proofs established above as the corresponding net field. This assembles the certified graph. Classical filtering of the surviving-edge subtype accounts for the noncomputable qualifier. □

Lean: `Net.plug`

There is deliberately no claim that k is globally injective in this theorem. If a host source port equals a host target port, its replacement input and output are identified. The separate source and target injection arguments are sufficient for every net invariant.

13 Paths, renaming, and isomorphism

13.1 Paths and their behaviour under plugging

Definition 13.1 (Adjacency and path)**.** Edges e, f are adjacent when some vertex belongs to both $t(e)$ and $s(f)$. A path is a nonempty edge list: a singleton is a path; prepending e to a path starting at f gives a path when e, f are adjacent. Edges may repeat, and this notion does not specify a chosen boundary vertex at either end.

Lean: `Net.Adjacent`

Lean: `Net.Path`

For a list beginning e, f, the path predicate is equivalent to adjacency of e, f together with the path predicate for the tail starting at f. Invert the path constructor for the forward implication and apply it for the reverse one.

Lean: `Net.path_pair_iff`

Any map on edges preserving adjacency maps paths to paths. Induct on the path: singletons remain singletons, and the inductive step transports its first adjacency and then its tail path. If the edge map preserves and reflects adjacency, it preserves and reflects paths. Induct on the list: the empty list is a path on neither side, singletons are paths on both sides, and longer lists reduce to the first adjacency equivalence and the tail induction hypothesis.

Lean: `Net.Path.map`

Lean: `Net.path_map_iff`

Lemma 13.2 (Host and inserted adjacencies)**.** *Two surviving host edges are adjacent after plugging exactly when they were adjacent before plugging. Two inserted edges that were adjacent remain adjacent.*

Proof. For host reflection, pull both incidence witnesses back through their mapped lists. Equality of their host images implies equality of the original vertices by injectivity of j, giving an old shared witness. For host preservation map an old shared witness through j. For inserted preservation map the shared replacement witness through k; injectivity is unnecessary in this direction. □

Lean: `Net.Replacement.adjacent_host_iff`

Lean: `Net.Replacement.adjacent_insert`

Definition 13.3 (Non-self-loop hole)**.** The hole h is non-self-looping if no vertex belongs to both $s(h)$ and $t(h)$. This requires the two lists to be disjoint; it is stronger than distinctness within each individual list, already required of every net.

Lean: `Net.Replacement.NonSelfLoop`

For such a hole, k is globally injective. Equal internal images reduce to equal names; mixed internal/boundary cases contradict freshness. Two input images or two output images reduce to the appropriate port injection. Equality of an input image and an output image would exhibit a host vertex in both $s(h)$ and $t(h)$, contrary to non-self-looping.

Lean: `Net.Replacement.insert_injective`

Consequently inserted adjacency is reflected as well as preserved: pull the two mapped-list witnesses back and use global injectivity of k to identify them. The other implication is the unconditional preservation result above.

Lean: `Net.Replacement.adjacent_insert_iff`

Theorem 13.4 (Path preservation and reflection)**.** *A list of surviving host edges is a path after plugging if and only if its old edge list is a path. Any replacement path gives an inserted path. For a non-self-loop hole the converse holds for inserted paths too.*

Proof. For host paths, induct on the surviving-edge list. Empty and singleton lists are immediate; a longer list reduces to host adjacency equivalence and the tail hypothesis. For inserted preservation, apply path mapping to inserted adjacency preservation. For inserted reflection under the stated restriction, apply the general path equivalence lemma to inserted adjacency equivalence. □

Lean: `Net.Replacement.path_host_iff`

Lean: `Net.Replacement.path_insert`

Lean: `Net.Replacement.path_insert_iff`

13.2 Transport along carrier bijections

A bijection on internal names extends to ports by mapping only internal vertices. Its inverse laws follow by cases on a port: internal cases use the original inverse laws and boundary cases are reflexive. Mapping the identity is pointwise the identity, also by port cases; function extensionality gives the corresponding identity of functions.

Lean: `Port.bijection`

Lean: `Port.map_id`

Lean: `Port.map_id_fun`

Fix bijections $a : G_V \cong V'$ and $b : G_E \cong E'$. Reindexed sources and targets at $e' \in E'$ are the old incidence lists at $b^{-1}(e')$, mapped by the port extension of a.

Lean: `Net.Reindex.source`

Lean: `Net.Reindex.target`

Source uniqueness survives this transport: two shared mapped witnesses are equal by port injectivity, so old source uniqueness identifies their old edge indices; apply b and cancel $b \circ b^{-1}$. The target uniqueness proof uses old target uniqueness and the same cancellation.

Lean: `Net.Reindex.source_unique`

Lean: `Net.Reindex.target_unique`

For source coverage, a new source witness pulls back to an old source vertex, which is not an output; port maps preserve this property. Conversely, pull a non-output vertex backwards through a^{-1}, choose an old source edge by coverage, and map both the edge and vertex forwards. The two inverse laws prove that this is a witness for the original new vertex. Target coverage repeats this argument with non-input vertices and old target coverage.

Lean: `Net.Reindex.source_coverage`

Lean: `Net.Reindex.target_coverage`

Give an internal name w the label of $a^{-1}(w)$. Labelling an old vertex after mapping by a then agrees with its old label: the internal case cancels $a^{-1} \circ a$, and boundary labels are fixed.

Lean: `Net.Reindex.label_map`

Definition 13.5 (Reindexed graph)**.** The reindexed net uses the new carriers, inverse-transported labels, and the incidence lists just defined.

Well-formedness. Transport finite enumerations along the bijections. Distinctness follows from port injectivity, uniqueness and coverage from the preceding lemmas. For typing, compose incidence mapping with the vertex-label map, use label commutation, and apply the original typing equation. The boundary lists are unchanged. □

Lean: `Net.reindex`

13.3 Graph isomorphism and shallow context congruence

Definition 13.6 (Isomorphism)**.** An isomorphism $G \cong H$ between nets with the same signature and boundary lists is a bijection of internal vertices and a bijection of edges preserving internal labels, edge labels, and both ordered incidence lists. Its port action fixes boundary positions.

Lean: `Net.Iso`

Identity bijections and identity list maps yield a reflexive isomorphism. For the inverse isomorphism, invert both bijections. Inverse label preservation follows by applying the original equation at an inverse image and cancelling. To invert an incidence equation, map its two sides by the inverse port map and cancel the forward/inverse maps pointwise. For composition, compose carrier bijections; compose label equalities, and use successive list mapping followed by the two incidence-preservation equations.

Lean: `Net.Iso.refl`

Lean: `Net.Iso.symm`

Lean: `Net.Iso.trans`

Isomorphisms preserve and reflect adjacency: transport a shared witness forwards for preservation; for reflection pull both witnesses back from the incidence maps and identify their preimages by port injectivity. Applying the general path equivalence lemma then preserves and reflects paths, with their edge lists mapped by the edge bijection.

Lean: `Net.Iso.adjacent_iff`

Lean: `Net.Iso.path_iff`

The original graph is isomorphic to its reindexing. Use the chosen carrier bijections; inverse cancellation proves label preservation and each incidence identity directly from the definition of reindexing.

Lean: `Net.reindexIso`

Suppose $f : H \cong K$ and both graphs have the interface of host hole h. The intended isomorphism between $C[h := H]$ and $C[h := K]$ is the identity on host internal names and f on replacement internal names, and likewise the identity on surviving host edges and f on inserted edges. Sum bijections supply their inverse laws.

Lean: `Net.PlugIso.vertices`

Lean: `Net.PlugIso.edges`

The resulting port map commutes with host embedding: check the three port constructors, all of which are reflexive. It commutes with insertion too: internal vertices use the replacement part of the sum map, and boundary vertices reduce to the host-embedding equation at the corresponding hole port.

Lean: `Net.PlugIso.map_host`

Lean: `Net.PlugIso.map_insert`

Theorem 13.7 (Congruence of shallow plugging)**.** *For every typed host and hole, $H \cong K$ implies $C[h := H] \cong C[h := K]$. No non-self-loop hypothesis is required.*

Proof. Use the sum bijections above. For each label equation split the carrier tag: host labels are unchanged and inserted labels use f's corresponding equation. For a host source list, commute the new port map past host embedding pointwise. For an inserted source list, commute it past insertion and then use source preservation by f. Repeat with target lists and target preservation. These arguments prove all isomorphism fields without reflecting through insertion. □

Lean: `Net.plugIso`

Here edge labels are preserved by literal equality. Section 23 defines a recursive relation that also identifies independently renamed box contents and proves the corresponding congruence at arbitrary addressed depth.

14 Concrete uniqueness of operation paths

Choose a predicate P on operation labels in a net G. This section first proves a generic graph theorem; the next section shows that the single-source condition is automatic for the typed operation labels of the linear hierarchy.

Definition 14.1 (Unary operations and traversal)**.** Selected operations are unary when each edge whose label satisfies P has a source list of length one. An operation step $s \rightsquigarrow t$ chooses such an edge e with $s \in s(e)$ and $t \in t(e)$.

Lean: `Net.UnaryOperations`

Lean: `Net.OperationStep`

Reverse traversal is the machine whose transition from t to s is $s \rightsquigarrow t$ and whose successful final states are boundary inputs. Inputs are normal: any reverse step from an input would put that input in a target list, contradicting target exclusion.

Lean: `Net.reverseOperations`

Lemma 14.2 (Determinism derived from incidence)**.** *If selected operations are unary, reverse operation traversal is deterministic.*

Proof. Two reverse steps from the same vertex select edges both targeting that vertex. Target uniqueness identifies the edges. Their source list has length one, so both predecessor vertices equal its only entry and hence equal one another. There is no additional determinism premise. □

Lean: `Net.reverseOperations_deterministic`

Definition 14.3 (Edge-recording trace)**.** A reverse operation trace from v to w records its actual edge list. The empty trace starts and ends at the same vertex. A nonempty trace first chooses an operation edge targeting the current vertex and a vertex in its source, then continues recursively from that source vertex.

Lean: `Net.OperationTrace`

A trace with edge list l yields exactly $|l|$ steps in the reverse machine. Induct on its construction: the empty trace is a zero-step execution and each trace constructor supplies the first step of the counted execution.

Lean: `Net.OperationTrace.steps`

Theorem 14.4 (Exact uniqueness to inputs)**.** *Under the unary condition, two reverse traces from the same vertex to inputs have equal edge lists and equal ending input vertices.*

Proof. Induct on the first trace. If it is empty, the starting vertex is an input; the second trace cannot be nonempty, since no input belongs to a target list. Thus it too is empty and both conclusions follow. If the first trace is nonempty, the second cannot be empty, for the same target-exclusion reason. Their first edges are equal by target uniqueness. Their next vertices agree by the determinism lemma. Apply the induction hypothesis to their tails and prefix the common edge to the resulting list equality. □

Lean: `Net.OperationTrace.unique_to_inputs`

A forward operation path from s to t with edge list l means a reverse trace from t to s with list reverse(l). Empty paths are allowed in this endpoint-sensitive notion, unlike the nonempty edge-only path predicate of the preceding section.

Lean: `Net.OperationPath`

Two forward operation paths from inputs to the same vertex therefore have equal forward edge lists and equal starting input indices. Apply exact trace uniqueness, use injectivity of list reversal, and then injectivity of the input constructor.

Lean: `Net.operationPath_unique_from_inputs`

Theorem 14.5 (No positive cycle on an input-rooted operation path)**.** *If a vertex v is reached by a forward operation path from an input, every reverse operation trace from v back to v has an empty edge list.*

Proof. The input-rooted path gives a reverse execution from v to a successful input. Prefix that execution by the proposed cycle to obtain another successful execution from v to the same input. Deterministic terminal uniqueness identifies their lengths. Thus $|cycle| + |path| = |path|$, so the cycle has length zero and is empty. □

Lean: `Net.no_operation_cycle_on_input_path`

This is a statement about cycles at input-reachable vertices, not a global acyclicity restriction on nets. The self-loop example below is a valid net with a cycle and no boundary inputs.

15 Typed boxes, deep occurrences, and contexts

15.1 Relabelling without changing incidence

A signature-preserving map f of operation alphabets relabels a net by replacing each edge label o with $f(o)$. All carriers, enumerations, vertex labels, and incidence data are retained. Hence their invariant proofs are retained too. For typing, compose the old typing equality with the inverse of the stipulated signature equality for $f(o)$. This yields the new signature list at that edge.

Lean: `Net.relabel`

15.2 The linear wire and label discipline

Definition 15.1 (Wire types, operation arities, and holes)**.** A wire type is either a term wire $*$ or a thunk wire $T^n(*)$, carrying a natural number n of bound variables. An operation specification assigns each operation an argument count $a(o)$ and an ordered list of binder counts $b(o)$. A hole label contains a natural-number name together with arbitrary input and output wire lists. Equality of hole labels includes these lists as well as the name.

Lean: `Linear.Wire`

Lean: `Linear.Operations`

Lean: `Linear.Hole`

Primitive edge labels are operations, variables, focus labels from the abstract three-mode type, or typed holes. This introduces concrete focus *labels*; it does not yet define the UAM transition relation.

Lean: `Linear.Primitive`

Their signature is as follows. Every non-hole primitive has one term input. Operation o has $a(o)$ term outputs followed by the thunk outputs specified in $b(o)$. A variable or focus has one term output. A hole uses precisely the input and output lists stored in its label. Ordering is part of each of these definitions.

Lean: `Linear.primitiveSignature`

Definition 15.2 (Alphabets and typed boxes)**.** An alphabet is a label type and a signature on it. A box over alphabet A stores numbers n, m and a body net over A of type

$$* \longrightarrow *^{n+m}.$$

The corresponding outer box edge will have type $T^n(*) \longrightarrow *^m$. Thus body boundary and outer boundary differ: n body outputs account for binding, and m for exposed free variables.

Lean: `Linear.Alphabet`

Lean: `Linear.Box`

All alphabet label types live in one fixed higher universe, while graph carriers remain small types. The base alphabet contains just primitives, raised into that universe by a universe lift; its signature reads the underlying primitive. The successor alphabet is the disjoint union of

primitives and boxes over the previous alphabet. Its signature uses the primitive signature on the left and the stated external box interface on the right.

Lean: `Linear.baseAlphabet`

Lean: `Linear.nextAlphabet`

Recursively, depth zero uses the base alphabet and depth $d + 1$ uses the successor of the depth-d alphabet. A hypernet at depth bound d is a net over that alphabet. The bound limits possible nesting; it need not be the actual minimum depth of the particular graph.

Lean: `Linear.alphabet`

Lean: `Linear.Hypernet`

The operation predicate selects primitive operation labels only. At depth zero it inspects the universe-lifted primitive; at a successor depth it accepts only the primitive-operation summand, excluding boxes and all other primitive kinds.

Lean: `Linear.IsOperation`

Selected operation edges have exactly one source. Rewrite the incidence length using signature typing, split on the depth and label constructors, discard labels excluded by the predicate, and compute the operation input list $[*]$.

Lean: `Linear.unary_operations`

It follows that two operation paths from inputs to the same vertex have identical edge lists and input indices in every typed hypernet. Instantiate the generic operation-path uniqueness theorem with the preceding derived unary proof. This is the graph-theoretic uniqueness conclusion associated with the paper's Lemma 22 for this representation; neither determinism nor a locality field is assumed.

Lean: `Linear.operation_path_unique`

15.3 Box construction and promotion of the depth bound

The box constructor takes a body with the specified $n + m$ output interface and forms the corolla of its box label in the successor alphabet. Corolla well-formedness proves all outer graph conditions. The result has exactly one outer box edge and the external type $T^n(*) \to *^m$.

Lean: `Linear.box`

Promotion raises the available depth bound by one without changing the outer graph. At depth zero, send a lifted primitive to the primitive summand at depth one. At a successor depth, keep primitives and recursively promote each box body, retaining its bound/free metadata. These maps preserve signatures because neither primitive signatures nor external box arities change. Apply the generic relabel construction at each recursive step.

Lean: `Linear.raise`

Outer sources and targets are unchanged by promotion. The code records these as heterogeneous equalities: the dependent graph types are indexed by different depths, although after unfolding either defining case the incidence functions are definitionally the same. Splitting on depth proves both equalities by reflexivity.

Lean: `Linear.raise_source`

Lean: `Linear.raise_target`

15.4 Finite occurrence addresses at arbitrary depth

Definition 15.3 (All edge occurrences)**.** At depth zero, an occurrence is a shallow edge. At depth $d + 1$, an occurrence is either a shallow edge or a pair (e, a) where e is a box edge and a an occurrence in its body. A primitive edge has the empty type of child occurrences.

Lean: `Linear.AllEdge`

The parent edge is part of every deep address. In particular identical bodies placed at different parent edges do not identify their child occurrences.

Theorem 15.4 (Finiteness of all occurrences)**.** *Every finite-depth hypernet has an explicit finite enumeration of its shallow and deep edge occurrences.*

Proof. Recurse on the depth bound. At zero use the graph's edge enumeration. At a successor bound, enumerate shallow edges in the left summand. For the right summand enumerate the dependent sum over outer edges: a primitive contributes an empty enumeration, and a box contributes the recursively obtained enumeration of its body. The sum and dependent-sum enumeration constructions already prove completeness of this list. Recursion decreases the depth at every box descent. □

Lean: `Linear.allEdgesFinite`

Every given occurrence belongs to this enumeration, by its completeness field. This is the finiteness content of Appendix A, Lemma 19, for the typed hierarchy. The list need not be duplicate-free, since completeness suffices here.

Lean: `Linear.every_edge_enumerated`

15.5 Hole lookup and the context condition

A primitive hole lookup returns its stored label for a hole constructor and no label otherwise. Full occurrence lookup uses that function on a shallow primitive, returns no hole at a shallow box edge, and recursively looks inside a deep box address. A purported child occurrence under a primitive has empty type and is eliminated. Dependent transports in Lean express that the selected label really is the box whose body is being searched.

Lean: `Linear.primitiveHole`

Lean: `Linear.holeAt`

Definition 15.5 (Unique holes and contexts)**.** A hypernet has unique holes if, for each complete typed hole label h, any two occurrences whose lookup returns h are equal. A context is a hypernet together with this property. This permits multiple different hole labels and also permits no holes; it is an at-most-once condition for each label over all depths.

Lean: `Linear.UniqueHoles`

Lean: `Linear.Context`

Lemma 15.6 (Boxing preserves unique holes)**.** *If a body has unique holes, its one-edge box has unique holes.*

Proof. A shallow occurrence in the outer box is not a hole, so either proposed hole occurrence must be deep. The outer graph has only one edge; therefore their parent indices agree. Their lookups are the body lookups, whose occurrences agree by the body's uniqueness assumption. Tagging these equal child addresses with their common parent gives equality of the original occurrences. □

Lean: `Linear.box_unique_holes`

Boxing a context now means boxing its underlying graph and supplying this lemma as the context certificate. Shallow plugging at any fixed hierarchy depth is just the generic typed net plugging construction with that depth's alphabet; its interface constraints are checked by the type of its replacement argument.

Lean: `Linear.boxContext`

Lean: `Linear.plug`

This last operation returns a hypernet. The later context-calculus development now proves that its result is a context under explicit freshness conditions (Section 19), including a sharp necessary-and-sufficient condition for shallow plugging. The typed hierarchy has not yet been proved adequate for the paper's original set-based presentation of graph-labelled edges.

16 Addressed deep replacement and concrete locality

16.1 Changing one box body

A per-edge signature-preserving relabelling generalizes alphabet relabelling: it assigns a new label directly to each edge, rather than requiring the assignment to factor through its old label. All carriers, incidence, and vertex labels stay unchanged. The two new typing equations follow by composing each old typing proof with the inverse of the specified per-edge signature equality.

Lean: `Net.relabelEdges`

Definition 16.1 (Replacement of a box label)**.** Suppose outer edge e is labelled by a box (n, m, B). Given a new body B' of the same body type $* \to *^{n+m}$ and depth bound, change just the label at e to (n, m, B'), retaining every other edge label.

Well-formedness. Use classical equality testing on outer edges to select e. At e the old and new labels have the same external input $[T^n(*)]$ and output $*^m$; this follows from the given equality identifying the old label as the box. At every other edge the label is unchanged. These two cases prove the per-edge signature conditions for relabelling, which supplies the certified net. □

Lean: `Linear.replaceBox`

Selection uses an edge occurrence, not equality of box contents. Equal bodies at other edges therefore do not cause simultaneous replacement.

16.2 Typed sites and replacement at a site

Definition 16.2 (Site)**.** A site in an outer hypernet G carries a local depth bound and two wire-label lists J, K. It is either a shallow edge of G, in which case J, K are exactly that edge's signature lists, or a box edge with a proof identifying its box label together with a recursively chosen site in its body. In the latter case the site's local depth and interface are inherited from the child site.

Lean: `Linear.Site`

The indices prevent supplying a replacement of the wrong interface or local depth. A site may select an arbitrary edge, not only an edge labelled as a hole; the separate unique-hole context condition is not assumed here.

Definition 16.3 (Deep plugging)**.** To replace at a shallow site use shallow net plugging. To replace at a site inside a box, recursively replace in its body and use box-label replacement to put that modified body back at the chosen outer edge. The recursion is on the site's finite constructor structure, and the result retains the outer depth bound and boundary interface.

Lean: `Linear.plugAt`

At a shallow site this definition agrees with shallow plugging by reflexivity: both sides unfold to the same term.

Lean: `Linear.plugAt_shallow`

Theorem 16.4 (Locality at every enclosing box)**.** *Replacement strictly inside a selected box leaves all outer source lists and all outer target lists unchanged. It also leaves the label of every other outer edge unchanged.*

Proof. The inside branch changes only an edge label using per-edge relabelling. The source and target fields of that construction are definitionally those of the old graph, so their equalities are reflexive, independent of what the recursive replacement did inside. For another outer edge $f \neq e$, the conditional defining box-label replacement chooses the old-label branch, proving the label equation. Repeated descent applies this fact at each enclosing layer. □

Lean: `Linear.plugAt_inside_source`

Lean: `Linear.plugAt_inside_target`

Lean: `Linear.plugAt_inside_other_label`

These are unconditional, concrete graph locality results about the defined replacement operation. They do not yet prove UAM traversal locality, refocusing, or the specimen interface laws: no concrete machine transition relation has been constructed. The subsequent provenance development proves unique-hole context closure for this concrete replacement at any depth. Section 23 proves recursive isomorphism congruence for this replacement. Associativity of context plugging remains a separate task.

17 Concrete graph examples and regression proofs

17.1 Self-loop gluing and the necessary reflection restriction

Use a singleton wire-label set and a singleton operation alphabet, whose only operation has one input and one output of that label. The graph named loop has one internal vertex, one edge, no outer boundary, and that vertex as both the edge's source and target.

Lean: `Examples.unary`

Lean: `Examples.loop`

Its finite carriers are both Fin(1). Singleton incidence lists are distinct; all edges are equal since their indices are bounded by one. The only internal vertex is incident on both sides, providing coverage, while boundary cases have empty finite-index types. Label typing is reflexive. Thus this is a well-formed net despite its cycle. Let wireEdge be the corolla of the same unary operation, with its two distinct boundary vertices.

Lean: `Examples.wireEdge`

The corolla is not adjacent to itself: a shared target/source witness would have to be both its output constructor and its input constructor, an impossible constructor equality.

Lean: `Examples.corolla_not_self_adjacent`

When the corolla replaces the loop's sole edge, its input and output images are equal. Both are, by definition, the same old internal vertex, because the host hole's source and target port coincide. Hence the inserted edge is adjacent to itself: use that old vertex as the witness and compute the mapped singleton source and target lists.

Lean: `Examples.inserted_boundary_identified`

Lean: Examples.inserted_self_adjacent

Consequently the inserted two-entry list $[e, e]$ is a path, while $[e, e]$ is not a path in the original corolla. The positive part uses its new self-adjacency and a singleton tail path. The negative part extracts its first adjacency and contradicts the corolla result. This proves that unconditional reflection of inserted paths would be false; the non-self-loop hypothesis cannot simply be omitted from that theorem.

Lean: Examples.unconditional_path_reflection_fails

The original loop itself has the two-entry path $[e, e]$, using its sole internal vertex as the shared witness. This verifies that the graph representation has not silently imposed global acyclicity.

Lean: Examples.loop_is_cyclic

The finite ordinal with one element is bijective to the unit type: send its index to the sole unit element and send unit back to zero. The inverse laws follow respectively from the ordinal bound and unit case analysis. An anonymous example reindexes the corolla's edge carrier along this bijection and applies shallow plugging congruence. It obtains isomorphic filled loops despite the different replacement-edge carrier types, testing actual renaming rather than reflexivity alone.

Lean: Examples.oneToUnit

Lean: Examples.example (line 89)

17.2 Values, boxes, and distinguished occurrence addresses

For the linear examples, take two operation labels, value and other-value. Give both zero term arguments and no thunk arguments; each therefore has signature $* \to []$. The graph value is the corolla of the first operation at depth zero.

Lean: Linear.Examples.Op

Lean: Linear.Examples.operations

Lean: Linear.Examples.value

Boxing it with zero bound and free variables yields a closed box of type $T^0(*) \to []$ at depth one. A further example constructs the analogous graph at depth bound two by putting a primitive value at depth one inside a box. This is a larger *available bound*, not a claim that the example has two actually nested boxes.

Lean: Linear.Examples.boxedValue

Lean: Linear.Examples.higherBoxedValue

An anonymous proof verifies that any occurrence in the depth-one boxed value belongs to the recursive occurrence enumeration, by the general completeness theorem. The outer box edge is represented by a shallow left-tagged occurrence; the value edge in its body is a right-tagged pair of parent edge and child edge.

Lean: Linear.Examples.example (line 117)

Lean: Linear.Examples.outerOccurrence

Lean: Linear.Examples.innerOccurrence

These two addresses are unequal. If equal, applying the sum-tag test would give true = false,

which is impossible. This checks separation of outer and inner occurrences even though each individual edge carrier is singleton.

Lean: `Linear.Examples.outer_ne_inner`

The other-value corolla has exactly the same interface as the value corolla. A site inside the boxed value descends through its only box edge and selects the value edge in its body. Replacing at this site with other-value defines replacedInside.

Lean: `Linear.Examples.otherValue`

Lean: `Linear.Examples.insideValue`

Lean: `Linear.Examples.replacedInside`

Its outer source and target functions equal those of the original boxed value; instantiate the two deep-locality theorems at the selected site. Its sole outer edge label is the box with body value$[e :=$ otherValue$]$. For this label equation, unfold the site and replacement, simplify the equality test selecting the sole outer edge, and use reflexivity for the resulting body. The outer box is retained rather than replaced by the value operation itself.

Lean: `Linear.Examples.deep_replacement_source_unchanged`

Lean: `Linear.Examples.deep_replacement_target_unchanged`

Lean: `Linear.Examples.deep_replacement_body`

17.3 An input-rooted operation path

Give the single unit operation one term argument and no binders. Its corolla argument has type $* \to *$. The input, its only edge, and the output form a forward operation path: read this as the one-step reverse trace, check membership in the two singleton incidence lists, and end with an empty trace.

Lean: `Linear.Examples.argumentOperations`

Lean: `Linear.Examples.argument`

Lean: `Linear.Examples.argument_path`

Any reverse operation trace from its output back to that output has empty edge list. Apply the no-cycle theorem to the constructed input-rooted path, deriving the required unary condition from the typed signature. This exercises the connection between concrete graph typing and generic machine terminal uniqueness.

Lean: `Linear.Examples.no_cycle_after_argument`

17.4 Filling a hole inside a certified context

Choose the typed hole label with name zero, one term input, and no outputs. Its corolla singleHole is a depth-zero hypernet. It has unique holes because any two edge occurrences are equal: both are elements of Fin(1), so their underlying indices are zero. Pairing this graph with the proof constructs a context.

Lean: `Linear.Examples.holeLabel`

Lean: `Linear.Examples.singleHole`

Lean: `Linear.Examples.singleHole_unique`

Lean: `Linear.Examples.singleContext`

Box the graph with zero bound and free variables to obtain boxedHole, and box the certified context to obtain boxedContext. The unique-hole certificate in the latter is supplied by the general boxing theorem. Looking up the deep address consisting of the only outer edge and the only body edge returns the chosen hole label by definition.

Lean: `Linear.Examples.boxedHole`

Lean: `Linear.Examples.boxedContext`

Lean: `Linear.Examples.hole_is_deep`

The corresponding site descends into that box and chooses its shallow hole edge. Filling this site with the value corolla produces filledBox. Its outer label is the same zero-bound, zero-free box, now containing singleHole$[e := \mathsf{value}]$; unfold deep replacement and simplify the selected-edge equality test to prove this equation.

Lean: `Linear.Examples.boxedHoleSite`

Lean: `Linear.Examples.filledBox`

Lean: `Linear.Examples.filled_box_body`

Finally every edge of this filled body has no primitive hole label. Split on its edge tag. An old surviving edge would be an element of the singleton old carrier different from its only element, contradicting the surviving-edge certificate. An inserted value edge is an operation, whose primitive hole lookup is definitionally empty. Thus this example proves removal of the old hole, not merely the well-typedness of the filled graph.

Lean: `Linear.Examples.filled_body_no_holes`

18 Occurrence fibres and transport of hole observations

The next milestone completes context closure under explicit hole freshness, including replacement at arbitrary box depth. Its proofs need to distinguish individual occurrences, rather than merely sets of labels. This section provides coordinates for those occurrences without changing the existing hypernet type.

18.1 Coordinates local to an outer edge

Definition 18.1 (Children and occurrence fibre)**.** For a label l at depth zero, its child occurrence type is empty. At depth $d+1$, a primitive label again has no children; a box label with body B has child type AllEdge(B). Define the fibre of l to be

$$F_d(l) = \{\bullet\} + \mathrm{Children}_d(l).$$

The left element is the edge itself; a right element is an occurrence below it.

Lean: `Linear.Children`

Lean: `Linear.Fiber`

The shallow hole observation reads the primitive hole label at depth zero or in the primitive summand at positive depth, and returns no hole on a box label. The child observation eliminates empty child types and otherwise uses the body's existing hole lookup. The fibre observation combines these functions: left elements use shallow lookup and right elements use child lookup.

Lean: `Linear.shallowHole`

Lean: `Linear.childHole`

Lean: `Linear.fiberHole`

An occurrence of a graph G in fibre coordinates is a dependent pair

$$(e, x), \qquad e \in G_E, \quad x \in F_d(\lambda_G(e)).$$

The first component is always an outer edge, even when x goes deep into a box. This representation depends on edge labels and occurrence addresses, not on vertex incidence or a particular enumeration of the carriers.

Lean: `Linear.Occurrences`

Lemma 18.2 (Occurrence coordinates)**.** *The existing occurrence type* AllEdge(G) *is bijective to the fibre-coordinate occurrence type.*

Proof. At depth zero, send an edge e to $(e, \mathsf{left}(\bullet))$; the inverse forgets that singleton component. A right component would belong to the empty type and is impossible. At positive depth, send a shallow edge to the same left-tagged pair, and a deep address (e, a) to $(e, \mathsf{right}(a))$. The inverse removes the fibre tag to reconstruct the corresponding shallow or deep address. Both inverse laws follow by cases on the tags and uniqueness of the unit value. □

Lean: `Linear.occurrenceBijection`

Define a graph's occurrence observation at (e, x) by applying the fibre observation to $\lambda_G(e)$ and x.

Lean: `Linear.occurrenceHole`

The bijection preserves hole observations. At depth zero this is definitional. For a positive-depth shallow occurrence, split the label into primitive or box and compute both lookups. For a deep occurrence, both lookups read the same body address: exposing the label cases makes the equality reflexive. The original hierarchy lookup now uses a directly dependent reader, avoiding unnecessary proof-dependent casts while retaining the same definition by label cases.

Lean: `Linear.occurrenceBijection_hole`

Lemma 18.3 (The context condition in fibre coordinates)**.** *A graph has unique holes exactly when any two fibre-coordinate occurrences observing the same hole label are equal.*

Proof. For the forward direction, apply unique holes to the inverse images of the two occurrences. Observation preservation supplies their lookup hypotheses. Map the resulting equality forwards and cancel the inverse pairs. Conversely, map the two original addresses forwards, use fibre-coordinate uniqueness and observation preservation, and conclude by injectivity of the bijection. □

Lean: `Linear.uniqueHoles_iff_occurrences`

18.2 Transport along equality and edge maps

If labels l, l' are equal, transport of $x \in F_d(l)$ along that equality gives an element of $F_d(l')$ with the same hole observation. Such transport is injective. For each assertion, eliminate the label equality; transport becomes the identity, so the observation equation is reflexive and injectivity is the given equality of transported elements.

Lean: `Linear.fiberHole_transport`

Lean: `Linear.fiber_transport_injective`

A shallow edge map $f : G_E \to H_E$ with equations $\lambda_G(e) = \lambda_H(f(e))$ maps all occurrences by

$$(e, x) \longmapsto (f(e), \mathrm{transport}(x)).$$

The descendant address is preserved through the label equality; it is not searched for again by comparing box contents.

Lean: `Linear.mapOccurrences`

This map preserves hole observation by the equality-transport observation lemma. If f is injective, the occurrence map is injective: equality of mapped pairs first identifies their edge indices by injectivity of f. After substituting that equality, equality of their transported second components implies equality of the original components by transport injectivity. The dependent pairs are therefore equal.

Lean: `Linear.mapOccurrences_hole`

Lean: `Linear.mapOccurrences_injective`

19 Hole inventories and sharp shallow context closure

All hole-label comparisons in this section compare complete typed labels: the name and both interface lists. The occurrence quantifiers include every box depth.

19.1 Hole inventories, absence, and isomorphism

Definition 19.1 (Presence, absence, and disjointness)**.** Write $\mathsf{HasHole}(G, h)$ when some occurrence of G looks up to h. Write $\mathsf{HoleFree}(G)$ when no label has such an occurrence. Two graphs have disjoint holes when no complete hole label occurs in both. The graphs may have different external interfaces and depth bounds.

Lean: `Linear.HasHole`

Lean: `Linear.HoleFree`

Lean: `Linear.HolesDisjoint`

Hole presence is equivalently witnessed by a fibre-coordinate occurrence with the same observation. Transport a witness by the occurrence bijection for one direction and by its inverse for the other, using observation preservation and the inverse equation.

Lean: `Linear.hasHole_iff_occurrences`

A hole-free graph has unique holes vacuously: the first purported hole occurrence already contradicts absence. Disjointness is symmetric by exchanging its two presence premises. Any graph is disjoint from a hole-free graph because a hole in the second graph would contradict its absence property.

Lean: `Linear.holeFree_unique`

Lean: `Linear.HolesDisjoint.symm`

Lean: `Linear.holeFree_disjoint`

Boxing neither creates nor removes a hole label from the inventory of the body. A shallow box occurrence cannot be a hole. A deep occurrence is exactly an address in its body with the same lookup, so it supplies a body witness. Conversely, tag a body witness with the box's only outer edge. Consequently a hole-free body gives a hole-free box: transfer any proposed hole witness to the body and contradict its absence property.

Lean: `Linear.box_hasHole_iff`

Lean: `Linear.box_holeFree`

A label-preserving edge map preserves hole presence: map the fibre-coordinate witness using the occurrence map and its observation equation. An injective such edge map reflects unique holes: map two proposed equal-label occurrences into the target, apply target uniqueness, and use occurrence-map injectivity.

Lean: `Linear.hasHole_map`

Lean: `Linear.uniqueHoles_of_embedding`

It follows that graph isomorphisms preserve and reflect hole inventories, and preserve and reflect the context condition. Apply presence preservation to the forward and inverse edge

bijections for the first equivalence. Apply reflection of uniqueness to the inverse bijection for forward preservation and to the forward bijection for reflection. Label preservation is supplied by the isomorphism equations with the appropriate equality direction.

Lean: `Linear.iso_hasHole_iff`

Lean: `Linear.iso_uniqueHoles_iff`

These results use the literal-label isomorphism notion. Section 23 extends inventory and context invariance to independently renamed box contents.

19.2 Provenance and the abstract collision argument

Definition 19.2 (Provenance certificate)**.** For result R, host C, and replacement H, a provenance certificate is an injective function

$$p : \mathrm{Occurrences}(R) \longrightarrow \mathrm{Occurrences}(C) + \mathrm{Occurrences}(H)$$

that preserves hole observation in the chosen summand. Its left values identify retained host occurrences and its right values identify inserted occurrences. Different result occurrences must have different tagged origins.

Lean: `Linear.Provenance`

This is a reusable intermediate interface. Unlike the earlier specimen laws, the provenance certificates required for actual plugging are constructed below from the concrete operations, rather than left as unproved assumptions.

Every result hole label belongs to the host or replacement inventory. Choose its fibre-coordinate witness, inspect the origin tag, and use preservation to obtain a witness in the indicated graph.

Lean: `Linear.Provenance.holes_subset`

A provenance certificate is *compatible* when no inserted hole label is observed at any retained host origin. Formally, if $p(a) = \mathsf{left}(c)$ and c observes h, then H does not contain h. Host occurrences outside the image of the left branch are irrelevant to this condition.

Lean: `Linear.Provenance.Compatible`

Theorem 19.3 (Unique holes from compatible provenance)**.** *If C, H have unique holes and a provenance certificate for R, C, H is compatible, then R has unique holes.*

Proof. Take two result occurrences observing the same h and inspect their origins. If both origins are retained, host uniqueness identifies their host addresses. If both are inserted, replacement uniqueness identifies their replacement addresses. In either case the tagged origins agree and provenance injectivity identifies the result occurrences. If one origin is retained and one inserted, observation preservation gives a retained host observation of h and a hole witness for h in H, contradicting compatibility. This covers all four tag combinations and proves fibre-coordinate uniqueness, hence the original condition. □

Lean: `Linear.Provenance.unique_of_compatible`

Full disjointness of the host and replacement inventories implies compatibility: any retained host occurrence is in particular a host occurrence. Substituting this fact into the preceding theorem gives unique holes under full disjointness.

Lean: `Linear.Provenance.unique`

19.3 Concrete shallow provenance and a first context operation

The provenance map for $C[e := H]$ reads its outer edge tag. A surviving host edge with fibre address x maps to the corresponding old host edge with x; an inserted edge maps to its replacement edge with x. In either case the edge label, and hence the fibre, is definitionally unchanged.

Injectivity and observation preservation. Mixed edge tags give different origin tags and cannot agree. Two retained origins first identify the old edge values. Their surviving-edge subtype values then agree because their inequality certificates are propositions; equality of the dependent second components identifies their fibre addresses. Two inserted origins are already the same replacement pair. In every case fibre observation is unchanged by definition. Thus these data construct a provenance certificate for any shallow replacement, including when the removed edge is a box. □

Lean: `Linear.shallowProvenance`

The general provenance theorem now proves context closure for shallow plugging under full hole disjointness. A context-valued operation packages the filled graph with this proof, taking certified host and replacement contexts and a proof of disjointness. All graph incidence invariants still come from the earlier certified net plugging construction.

Lean: `Linear.plug_uniqueHoles`

Lean: `Linear.Context.plug`

19.4 Exact inventory and the sharp freshness condition

Definition 19.4 (Retained hole label)**.** A retained hole label for removal of edge e in C is a hole observation in the fibre of an outer edge $f \neq e$. This includes deep occurrences below f. It excludes both the removed edge and every occurrence inside its label.

Lean: `Linear.RetainedHole`

Theorem 19.5 (Exact shallow hole inventory)**.** *For each label h,*

$$\mathsf{HasHole}(C[e := H], h) \iff \mathsf{RetainedHole}(C, e, h) \lor \mathsf{HasHole}(H, h).$$

Proof. Express presence using fibre coordinates. A result occurrence has an outer edge tag that is either a surviving host edge or an inserted edge. The first case supplies exactly the retained witness; the second supplies a replacement witness. Conversely, a retained witness gives a result occurrence at the left edge tag, using its inequality proof to construct the surviving-edge subtype. A replacement witness gives a result occurrence at the right tag. Their hole observations are unchanged by definition. □

Lean: `Linear.plug_hasHole_iff`

Define sharp plugging freshness to mean that no retained host hole label occurs in the replacement. This is weaker than requiring the replacement to be disjoint from the entire original host: labels confined to the removed edge's region may be reused.

Lean: `Linear.PlugFresh`

Sharp freshness implies uniqueness of the filled graph when both inputs have unique holes. Apply the compatible-provenance theorem to shallow provenance. Any result occurrence

with a retained origin must have a surviving host edge tag; its fibre is the retained witness required by sharp freshness. An inserted edge cannot have a retained origin, so the other case is impossible.

Lean: `Linear.plug_uniqueHoles_of_fresh`

Theorem 19.6 (Sharp freshness is necessary)**.** *If $C[e := H]$ has unique holes, sharp plugging freshness holds.*

Proof. Suppose a retained host fibre observes h and H contains h. Form the two result occurrences from their witnesses as in the inventory theorem. Result uniqueness identifies them. Equality of their first components would identify a surviving host edge tag with an inserted edge tag, which is impossible. □

Lean: `Linear.plug_fresh_necessary`

Consequently, for two contexts the filled graph has unique holes if and only if sharp plugging freshness holds. Necessity is the preceding theorem, which did not require uniqueness of either input; sufficiency uses the two input certificates.

Lean: `Linear.plug_uniqueHoles_iff_fresh`

19.5 Consumption of a selected hole and certified composition

In a context, if the removed edge's shallow lookup is h, that label has no retained occurrence elsewhere. Otherwise the retained occurrence and the selected edge's shallow occurrence would be two occurrences of h. Context uniqueness identifies their pairs, so their outer edges agree, contradicting the retained edge's inequality certificate.

Lean: `Linear.removed_hole_not_retained`

It follows that the consumed label h occurs in $C[e := H]$ exactly when it occurs in H. Use the exact inventory theorem and exclude its retained alternative by the preceding result. Thus consumption removes the old occurrence; it does not forbid the replacement from deliberately reintroducing that label.

Lean: `Linear.plug_consumed_hole_iff`

The operation Context.plugFresh packages shallow composition with this sharp freshness premise. Its result graph is ordinary net plugging, and its context certificate is the sharp sufficiency proof. It supports arbitrary finite depth and arbitrary selected outer edges, rather than requiring that the selected edge's label itself be a hole.

Lean: `Linear.Context.plugFresh`

20 Provenance and context closure at arbitrary box depth

The shallow origin map alone is insufficient for deep replacement: an enclosing box edge stays in place while occurrences in its body may change. We construct provenance through each selected enclosing box and then induct on the typed site.

20.1 Local provenance for a changed label

Definition 20.1 (Fibre provenance)**.** For old and new labels l, l' at the same outer depth and a replacement H, a fibre provenance certificate is an injective map

$$F_d(l') \longrightarrow F_d(l) + \mathrm{Occurrences}(H)$$

preserving the hole observation in the indicated summand. It accounts for changing one label while retaining or inserting the occurrences within that label.

Lean: `Linear.FiberProvenance`

An unchanged label has the left inclusion as its origin map, which is injective and preserves observation by reflexivity. If the new label merely has a proof of equality to the old label, first transport its fibre address along that equality and then use the left inclusion. Injectivity follows from transport injectivity; the hole equation is the inverse direction of the transport observation equation.

Lean: `Linear.FiberProvenance.unchanged`

Lean: `Linear.FiberProvenance.retained`

Lemma 20.2 (Assembly of local provenance)**.** *Suppose a signature-preserving per-edge relabelling of G has a fibre provenance certificate at every edge. If only a single designated edge can produce inserted origins, these certificates assemble into provenance for the relabelled graph relative to G, H.*

Proof. At a result pair (e, x) inspect its fibre origin. A retained origin y gives the old graph occurrence (e, y); an inserted origin is already an occurrence of H. Observation preservation is precisely the local certificate's equation. For injectivity, consider the two output tags. Two retained origins identify their outer edges and then their local retained addresses; local injectivity identifies the original fibre addresses. Mixed output tags cannot agree. If both origins are inserted, each outer edge must be the designated one: otherwise the hypothesis that all its origins are retained contradicts its inserted tag. With the outer edges identified, equality of inserted addresses and local injectivity again identify the fibre addresses. Thus the global origin map is injective in every case. □

Lean: `Linear.relabelProvenance`

The single-designated-edge restriction prevents two different outer edges from claiming the same inserted occurrence. The lemma proves this separation; it does not assume global injectivity as an extra graph law.

20.2 Lifting a body's provenance through one box

Suppose the old and new bodies B, B' have the same bound/free interface and a provenance certificate p for B', B, H. Their outer box labels differ only in body content. Construct fibre provenance as follows. The shallow occurrence of the box is retained as the old box's shallow occurrence. A new child address is converted to the fibre-coordinate address of B' and sent through p. Retained body origins are converted back to body addresses and tagged as old box children; inserted origins remain occurrences of H.

Injectivity and observation preservation. A retained shallow box occurrence is distinct from every retained child and every inserted origin by its tags. For two children, inspect the two body-origin tags. Mixed tags cannot give equal lifted origins. If both are retained, equality of the lifted children, followed by the old body's occurrence bijection, identifies the retained body origins. If both are inserted, their equality is already equality of body origins. In both cases body-provenance injectivity and the new body's occurrence-bijection injectivity identify the original children. For observations, the shallow box lookup is empty on both sides. A child's lookup agrees with its body-coordinate observation by the bijection lemma; body provenance gives the intermediate equality. In the retained case the inverse bijection of the old body supplies the remaining observation equation. In the inserted case the body equation already gives the replacement observation. □

Lean: `Linear.boxFiberProvenance`

To lift through a specified edge of a larger graph, use this box certificate at that edge and an equality-transported retained certificate at every other edge. Classical testing of edge equality selects the two cases. At the selected edge, the given label equality identifies its old box, and the new label contains the modified body with unchanged metadata. Elsewhere the label is unchanged, so all local origins are retained. The assembly lemma therefore constructs provenance for the existing concrete box-replacement operation.

Lean: `Linear.replaceBoxProvenance`

20.3 The deep context closure theorem

Theorem 20.3 (Provenance at every typed site)**.** *For every typed site s in G and matching replacement H, the defined result* plugAt(s, H) *has injective, hole-observation-preserving provenance relative to G, H.*

Proof. Induct on the site. A shallow site uses the concrete shallow provenance map. For an inside site, the induction hypothesis gives provenance for the replacement in the chosen body's inner site. Lift this certificate through the selected outer box using the preceding construction. That is exactly the inside branch of the definition of deep plugging. Induction follows the finite site address, so the proof works at every finite nesting depth. □

Lean: `Linear.siteProvenance`

Theorem 20.4 (Deep replacement preserves contexts under compatibility)**.** *If G, H have unique holes and the concrete site provenance is compatible, then* plugAt(s, H) *has unique holes.*

Proof. Use the constructed site provenance in the compatible-provenance uniqueness theorem. No locality or occurrence-injectivity hypothesis remains to be supplied: those properties have been proved for the actual replacement operation. The remaining compatibility premise is explicitly a restriction on colliding hole labels at retained origins. □

Lean: `Linear.plugAt_uniqueHoles_of_compatible`

Full disjointness of G, H gives a simpler sufficient condition: instantiate the full-disjointness provenance theorem with the same site certificate. This condition is stronger than compatibility because it also inspects discarded host occurrences; it is not asserted to be necessary.

Lean: `Linear.plugAt_uniqueHoles`

Every hole label of the deep result belongs to the inventory of the host or replacement, by the provenance inventory-subset theorem. This is an upper bound on the deep inventory, not an exact description of which host labels survive.

Lean: `Linear.plugAt_hasHole_subset`

If H is hole-free, it has unique holes and is disjoint from G. Substituting these two facts gives context closure for replacing any addressed edge by hole-free material, with no separate freshness witness required.

Lean: `Linear.plugAt_holeFree_unique`

Finally, Context.plugAt accepts a certified host context, a typed site, a certified replacement context, and full disjointness. It returns the concrete deeply filled graph paired with its proved context certificate. The original ordered outer interface and enclosing incidence laws are those already supplied by deep replacement.

Lean: `Linear.Context.plugAt`

This provenance construction supplies globally unique-hole context closure and concrete origin tracking through boxes. Section 22 strengthens the inventory bound to an exact syntactic characterization and gives a necessary-and-sufficient freshness condition at every finite box depth. Section 23 supplies recursive isomorphism of box contents and deep replacement congruence. Associativity of plugging remains a separate obligation. None of these results yet defines the concrete UAM.

21 Context composition: absence, reuse, and a collision

21.1 Hole-free filling inside a box

The value corolla is hole-free: any hole witness would claim that the lookup of a primitive operation is a hole label, whereas that lookup is empty. Applying deep context closure for a hole-free replacement to the old boxed-hole site proves that its filled result has unique holes. The context-valued operation also builds a certified filled context from the boxed context and the value corolla, using hole-free uniqueness and disjointness for the replacement.

Lean: `Linear.Examples.value_holeFree`

Lean: `Linear.Examples.filledBox_unique`

Lean: `Linear.Examples.filledContext`

The single-hole corolla has no retained occurrence when its only outer edge is removed. A proposed retained witness includes an edge different from zero, but every element of its edge carrier Fin(1) equals zero. This contradicts the witness's inequality certificate.

Lean: `Linear.Examples.singleHole_retained_absent`

Consequently replacing its only hole by the very same single-hole corolla still gives a context. Use sharp context closure: both input uniqueness proofs are available, and the retained part of the freshness premise is impossible by the preceding result. However, these two identical input graphs do not have fully disjoint inventories: their sole edge is a witness for the same label in each. Applying a proposed full-disjointness proof to those two witnesses gives a contradiction. Together these examples show that sharp freshness admits valid label reuse which the stronger sufficient condition would exclude.

Lean: `Linear.Examples.reuse_removed_label`

Lean: `Linear.Examples.reuse_not_fully_disjoint`

Filling the single-hole corolla with the value is hole-free, not just a context. The exact inventory theorem leaves a retained alternative or a value alternative; the first is impossible by absence of surviving old edges and the second by hole-freeness of the value.

Lean: `Linear.Examples.filled_single_holeFree`

The boxed filled result is hole-free at every depth as well. Express any proposed hole in fibre coordinates. Its outer edge has singleton carrier, so it is zero. The already proved label equation identifies that edge as a box whose body is the filled single-hole graph. Transport the fibre address along that equation, which preserves its hole observation. Its shallow alternative cannot observe a hole because it is a box; its child alternative contradicts the filled body's hole-freeness. Thus no shallow or deep witness exists in the result.

Lean: `Linear.Examples.filledBox_holeFree`

21.2 Two distinct shallow holes in series

For a natural-number name n, define a hole label χ_n with one term input and one term output. The two-hole graph has external type $* \to *$, one internal term vertex v, and edges e_0, e_1 with labels χ_0, χ_1 and incidence

$$\mathsf{input}_0 \xrightarrow{e_0} v \xrightarrow{e_1} \mathsf{output}_0.$$

This notation records the two source and target singleton lists, not an extra path assumption.

Lean: `Linear.Examples.chainHole`

Lean: `Linear.Examples.twoHoles`

Graph invariants. The carriers are Fin(1) and Fin(2), with finite range enumerations. Every incidence list is singleton and hence distinct. For source uniqueness, two edges of different indices have different source constructors, namely input and internal, so a shared source is impossible; equal-index cases give equal edges. The target argument uses internal and output constructors instead. Coverage chooses e_0 for the input source and internal target, and e_1 for the internal source and output target. Outer outputs have no source occurrence and outer inputs no target occurrence by constructor disjointness. There is only one internal index and one index at each boundary, so these cases exhaust all vertices. Every displayed vertex label is term, matching the singleton input and output lists of both typed hole labels. □

This graph has unique holes. Two equal observed labels give equal numeric hole names. If one edge index is zero and the other is not, their names are respectively zero and one, a contradiction. If both are zero they agree directly. If neither is zero, both are one because their ordinal carrier has size two, so they agree again. At depth zero these are all occurrence addresses.

Lean: `Linear.Examples.twoHoles_unique`

21.3 A checked failure without freshness

Let the replacement be the corolla of χ_1, with the same external interface as e_0. It is itself a context: any two occurrences belong to its singleton edge carrier and hence are equal, independently of their lookup hypotheses.

Lean: `Linear.Examples.repeatedHole`

Lean: `Linear.Examples.repeatedHole_unique`

Nevertheless, replacing e_0 in the two-hole graph by this corolla violates unique holes. In the result, the retained host edge e_1 and the inserted replacement edge both observe χ_1. Context uniqueness would identify their occurrences, but their edge tags are different constructors of the disjoint sum. This is a contradiction. Thus well-formed graph plugging and uniqueness of each input graph alone do not guarantee the context condition in the result.

Lean: `Linear.Examples.collision_breaks_unique`

The same witnesses explicitly refute sharp freshness: e_1 and its shallow fibre address witness retained presence of χ_1, and the replacement's only edge witnesses inserted presence. Feeding both to a proposed freshness proof gives false. This regression exercises the actual collision excluded by the necessary-and-sufficient theorem, alongside the valid label-reuse example above.

Lean: `Linear.Examples.collision_not_fresh`

22 Exact inventories and sharp context closure at every depth

The provenance argument establishes a useful upper bound on hole inventories. We now strengthen it to an equality described entirely by the original host and the selected address. This also gives a necessary-and-sufficient freshness condition. Throughout this section s is a site in G, H has the interface and local depth required by s, and $G[s := H]$ denotes $\mathsf{plugAt}(s, H)$. Write $\mathcal{H}(G)$ for the set of full typed hole labels present in G. These statements concern the concrete typed hierarchy; they do not presuppose any specimen laws or a concrete UAM semantics.

22.1 Separating one outer edge from the rest of a graph

Definition 22.1 (Inventory of an edge)**.** For an outer edge e of G, let $E_G(e, h)$ mean that some position in the fibre of e observes h. This includes the edge itself and all descendants within its label. Thus $E_G(e, h)$ is the predicate $\mathsf{EdgeHasHole}(G, e, h)$.

Lean: `Linear.EdgeHasHole`

Definition 22.2 (Uniqueness away from an edge)**.** The predicate $\mathsf{AwayUnique}(G, e)$ says that any two occurrences with outer edges different from e and with the same hole observation are equal. It places no condition on occurrences within the fibre of e.

Lean: `Linear.AwayUnique`

Lemma 22.3 (Partition of uniqueness)**.** *The graph G has unique holes if and only if all three conditions hold: occurrences away from e are unique; hole observations within the fibre of e are unique; and no label is observed both away from e and within that fibre.*

Proof. Global uniqueness restricts to each part. A collision between the two parts would force equality of their outer edges, contradicting the requirement that the first outer edge differ from e. Conversely, take two occurrences of the same hole label. If both lie away from e, use away-uniqueness. If both have outer edge e, use fibre uniqueness. The two mixed cases contradict the cross-part disjointness condition. These cases exhaust all occurrences. □

Lean: `Linear.uniqueHoles_partition`

Lemma 22.4 (Inventories of a box and of a whole graph)**.** *If the label of e is a box with body B, then $E_G(e, h)$ holds exactly when $h \in \mathcal{H}(B)$. For an arbitrary edge e, a label h occurs in G exactly when it occurs away from e or satisfies $E_G(e, h)$.*

Proof. The shallow position of a box is not a hole, while each of its child positions is precisely an occurrence of its body, with the same observation. For the whole-graph statement, write an occurrence as an outer edge and a fibre position, and split according to whether that edge equals e. In the reverse direction either witness is already an occurrence of G. □

Lean: `Linear.edgeHasHole_box`

Lean: `Linear.hasHole_partition`

Lemma 22.5 (Exact inventory after replacing a box body)**.** *Changing the body of box edge e to B' preserves the inventory away from e exactly. The entire new inventory is the union of the old inventory away from e and $\mathcal{H}(B')$.*

Proof. Every other edge has exactly the same label before and after the change. Transport its fibre positions along this label equality; fibre transport preserves observations in both directions. This proves the first claim. Partition the new graph at e and use the preceding box-inventory lemma to identify its selected fibre with the new body's inventory. □

Lean: `Linear.replaceBox_retained_iff`

Lean: `Linear.replaceBox_hasHole_iff`

Lemma 22.6 (Box uniqueness)**.** *The hole observations in a box fibre are unique exactly when its body has unique holes. Consequently, the body of every box in a context has unique holes.*

Proof. A box's shallow position cannot witness a hole. Two child positions observing the same hole are equal precisely when their body occurrences are equal. Injectivity of the child constructor proves the reverse implication. For the consequence, restrict the graph's uniqueness to the selected fibre using the partition lemma, then apply this equivalence. □

Lean: `Linear.boxFiber_unique_iff`

Lean: `Linear.uniqueHoles_box_body`

Lemma 22.7 (Uniqueness after changing one box)**.** *Suppose G is a context. Replacing the body of box edge e by B' yields a context if and only if B' has unique holes and its hole inventory is disjoint from the inventory of G away from e.*

Proof. First the unchanged part remains unique: transport two of its occurrences to G along their unchanged label equalities, apply uniqueness in G, and use injectivity of fibre transport to recover equality in the new graph. The uniqueness partition for the changed graph now has a proved first condition. Its second condition is equivalent to uniqueness of B', by box uniqueness. Its third condition is exactly the stated disjointness, by the preservation of the outside inventory and the exact inventory of a box. □

Lean: `Linear.replaceBox_awayUnique`

Lean: `Linear.replaceBox_uniqueHoles_iff`

22.2 A syntactic inventory of retained holes

Definition 22.8 (Retention along a site)**.** Let $R_s(h)$ be defined recursively on the site address. At a shallow site selecting e in G, it is $\mathsf{RetainedHole}(G, e, h)$: there is an occurrence of h under another outer edge. If the site descends through box edge e to an inner site t, set

$$R_s(h) \iff \mathsf{RetainedHole}(G, e, h) \lor R_t(h).$$

This is SiteRetainedHole. It depends only on the host and address, not on the replacement graph or a provenance certificate. It excludes all occurrences below the edge finally removed, while retaining the siblings of every enclosing box. Enclosing box edges are not themselves holes.

Lean: `Linear.SiteRetainedHole`

Theorem 22.9 (Exact deep hole inventory)**.** *For every site and every type-correct replacement, without uniqueness or freshness assumptions,*

$$h \in \mathcal{H}(G[s := H]) \iff R_s(h) \lor h \in \mathcal{H}(H).$$

Lean: `Linear.plugAt_hasHole_iff`

Proof. Induct on the site address. The shallow case is the exact shallow inventory theorem. For a site inside a box, the exact box-replacement inventory is the union of the unaffected outer inventory and the changed body's inventory. The induction hypothesis identifies the latter as the inner retained inventory union the replacement inventory. Reassociate these three alternatives; the first two are exactly R_s. Both directions are preserved at each step, so no retained or inserted label is lost from the characterization. □

Definition 22.10 (Exact deep freshness)**.** The predicate $\mathsf{SiteFresh}(s, H)$ says

$$\forall h, \quad R_s(h) \implies h \notin \mathcal{H}(H).$$

Only retained host labels constrain the replacement; a label that is wholly consumed is available for reuse.

Lean: `Linear.SiteFresh`

Lemma 22.11 (Retention implies original presence)**.** *If $R_s(h)$, then $h \in \mathcal{H}(G)$.*

Proof. Induct on the site. An outside witness is already a host occurrence. For an inner retained witness, the induction hypothesis supplies an occurrence in the original box body; the box-inventory lemma and whole-graph partition lift it back into the host. □

Lean: `Linear.siteRetainedHole_hasHole`

Lemma 22.12 (Replacement uniqueness is necessary for shallow plugging)**.** *If a shallow plugged graph has unique holes, its replacement has unique holes.*

Proof. The right-tagged injection embeds every replacement edge into the result, preserving labels and all fibre observations. Uniqueness reflects along this injective edge map by the established embedding theorem. □

Lean: `Linear.plug_uniqueHoles_replacement`

22.3 Necessary and sufficient context closure

Theorem 22.13 (Exact context criterion at arbitrary depth)**.** *Suppose G has unique holes. Then*

$$\mathsf{UniqueHoles}(G[s := H]) \quad \iff \quad \mathsf{UniqueHoles}(H) \ \wedge \ \mathsf{SiteFresh}(s, H).$$

Thus uniqueness of the replacement is a necessary conclusion in the forward direction, not an additional assumption of the theorem.

Proof. Induct on the address. At a shallow site, uniqueness of the result forces both replacement uniqueness and sharp shallow freshness. The reverse implication is the proved shallow closure theorem.

For an inside site, let B be the old body and t the inner address. Since G is a context, B is a context. The box-replacement criterion says that the result is a context exactly when $B[t := H]$ is a context and its inventory is disjoint from the outer inventory away from the selected box. The induction hypothesis replaces its first condition by uniqueness of H and inner freshness.

For necessity, inner freshness excludes collisions with inner retained labels. A label inserted from H is present in the changed body by the exact inventory theorem; outer disjointness therefore excludes collisions with every retained outer label. These are precisely the two alternatives defining R_s.

For sufficiency, site freshness supplies inner freshness, so induction proves uniqueness of the changed body. Consider a label shared by that body and an unaffected outer edge. The exact inventory theorem gives two cases. If the label comes from H, site freshness is contradicted. If

it is retained inside the body, retention implies its presence in the old body. It would then have occurred both inside and outside that box already in G, contradicting the uniqueness partition of G. This proves the required outer disjointness and completes the induction. □

Lean: `Linear.plugAt_uniqueHoles_iff`

If H is already a context, its uniqueness conjunct is known, leaving the necessary-and-sufficient condition $\mathsf{SiteFresh}(s, H)$ alone.

Lean: `Linear.plugAt_uniqueHoles_iff_fresh`

The constructor Context.plugAtFresh pairs the deep replacement graph with the proof supplied by this equivalence. Unlike full-label disjointness, its required certificate permits reuse of consumed labels at any box depth.

Lean: `Linear.Context.plugAtFresh`

Theorem 22.14 (Exact hole-freeness criterion)**.** *The result $G[s := H]$ is hole-free if and only if no label satisfies R_s and H is hole-free. No uniqueness assumption is needed.*

Proof. The exact inventory is empty precisely when both of its constituent inventories are empty. In either direction, an alleged witness in one part gives, or comes from, a witness in the union. □

Lean: `Linear.plugAt_holeFree_iff`

22.4 Consumption of the selected label

Definition 22.15 (Selected hole label)**.** The partial observation $\mathsf{Site.selectedHole}(s)$ follows the address to its selected edge and reads that edge's own shallow hole label. It does not inspect descendants of the selected edge. Thus selecting a box yields no hole label, even if its body contains holes.

Lean: `Linear.Site.selectedHole`

Lemma 22.16 (Presence and non-retention of a selected hole)**.** *If the selected label is h, then h occurs in the original host. If the host is a context, $R_s(h)$ is false.*

Proof. For presence, the shallow case uses the selected edge's shallow fibre position. The inside case lifts the induction hypothesis from the body through the box. For non-retention, the shallow statement is the existing consumed-label lemma. Inside a box, a retained witness outside the box contradicts the old graph's uniqueness partition, because the selected label occurs inside its body. An inner retained witness contradicts induction applied to that body's uniqueness. □

Lean: `Linear.Site.selectedHole_present`

Lean: `Linear.site_selected_not_retained`

Theorem 22.17 (Exact consumption at arbitrary depth)**.** *If G is a context and s selects a hole labelled h, then*

$$h \in \mathcal{H}(G[s := H]) \quad \Longleftrightarrow \quad h \in \mathcal{H}(H).$$

Proof. Use the exact deep inventory theorem and eliminate its retained-host alternative by non-retention of the selected label. This permits both genuine removal and intentional reintroduction, depending on the replacement. □

Lean: `Linear.plugAt_consumed_hole_iff`

22.5 Concrete regressions inside boxes

In the existing one-hole boxed graph, no label is retained when the inner hole is selected. At the outer level there is no other edge (the carrier has one element), and inside the box there is no other edge either. This proves $\neg R_s(h)$ for every h.

Lean: `Linear.Examples.boxedHole_retained_absent`

Replacing that hole by the same one-hole context therefore preserves uniqueness, by sharp deep freshness. Full disjointness fails because both original graphs contain the selected label. The consumed-label theorem also proves that this label is genuinely present after replacement, witnessed by its occurrence in the inserted graph.

Lean: `Linear.Examples.deep_reuse_unique`

Lean: `Linear.Examples.deep_reuse_not_disjoint`

Lean: `Linear.Examples.deep_reuse_present`

Replacing instead with the hole-free value graph yields a hole-free result: combine absence of retained labels and hole-freeness of the value using the exact hole-freeness criterion. This is a shorter independent derivation of the previous global filling result.

Lean: `Linear.Examples.deep_fill_holeFree`

For a negative example, box the earlier two-hole chain with zero bound variables and one free output. Its external interface is one zero-binding thunk input and one term output. Define a site descending into this box and selecting the first hole. The boxed graph is a context because the two-hole body is a context and boxing preserves uniqueness.

Lean: `Linear.Examples.boxedTwoHoles`

Lean: `Linear.Examples.boxedFirstSite`

Lean: `Linear.Examples.boxedTwoHoles_unique`

The one-hole replacement carrying label 1 is also a context, but it collides with the retained second hole. These two witnesses refute deep freshness. The necessary direction of the deep context theorem then proves that the result fails uniqueness.

Lean: `Linear.Examples.deep_collision_not_fresh`

Lean: `Linear.Examples.deep_collision_breaks_unique`

Finally, the exact inventory distinguishes the labels: label 0 is absent and label 1 is present. The first claim uses consumption and the fact that the replacement's label has name 1, not 0;

the second uses its inserted witness. An inventory records presence rather than multiplicity, so this is consistent with the independently proved duplicate occurrence of label 1.

Lean: `Linear.Examples.deep_collision_inventory`

23 Recursive isomorphism and substitution modulo renaming

Literal equality of edge labels is too restrictive when a label contains a whole graph. Renaming the body of a box changes its graph record, although its mathematical incidence structure is unchanged. This section defines recursive isomorphism, proves its invariants, and proves congruence of replacement at any addressed depth. All comparisons are within a fixed depth bound and fixed ordered outer interface. The quotient construction below does not identify different depth bounds or prove the representation theorem for the paper's set-based presentation.

23.1 Isomorphisms with related labels

Definition 23.1 (Isomorphism relative to a label relation)**.** Fix a signature and a relation R on its labels. An R-isomorphism from G to H consists of bijections $v : G_V \to H_V$ of internal vertices and $e : G_E \to H_E$ of edges. Internal vertex labels agree, corresponding edge labels are related by R, and mapping every source or target list through the induced port map gives the corresponding list in H. The port map fixes each ordered input and output position. We write $G \cong_R H$ for the existence of such data.

Lean: `Net.RelIso`

Lemma 23.2 (Identity, inverse, and composition)**.** *If R is reflexive, every graph has an identity R-isomorphism. If R is symmetric, every R-isomorphism has an inverse. If R is transitive, R-isomorphisms compose.*

Proof. For identity use identity carrier maps and reflexivity of R. For inverse use the inverse carrier bijections. Evaluate the original label equation at the inverse image, cancel the edge bijection, and use symmetry of R. For incidence, map the original incidence equation through the inverse port map; cancellation recovers the reversed equation. Composition uses composed carrier bijections, transitivity of the label relation, and the fact that a list mapped twice is mapped by the composite. Port maps compose because they act on internal names and fix both kinds of boundary position. □

Lean: `Net.RelIso.refl`

Lean: `Net.RelIso.symm`

Lean: `Net.RelIso.trans`

Lemma 23.3 (Incidence paths and ordinary isomorphisms)**.** *Every R-isomorphism preserves and reflects adjacency, hence preserves and reflects paths under its edge bijection. Every ordinary label-preserving isomorphism gives an R-isomorphism when R is reflexive.*

Proof. Map a shared vertex to obtain forward adjacency. For reflection, source and target preservation express the shared target vertex as images of two original ports. Injectivity of the port bijection makes those ports equal, giving the original shared vertex. Apply the existing list-path mapping lemma to this adjacency equivalence. For an ordinary isomorphism, retain all carrier and incidence data; equality of its labels and reflexivity give the required R-relation at each edge. □

Lean: `Net.RelIso.adjacent_iff`

Lean: `Net.RelIso.path_iff`

Lean: `Net.RelIso.ofIso`

23.2 Recursive comparison of box contents

Definition 23.4 (Related boxes)**.** For a relation on bodies of each fixed interface, two boxes are related when they have the same number of bound variables, the same number of free outputs, and related bodies at the interface specified by those two numbers. The constructor BoxRelated.bodies enforces both metadata equalities; it does not merely require equality of their sum.

Lean: `Linear.BoxRelated`

Definition 23.5 (Recursive label equivalence)**.** At depth zero, label equivalence is literal equality. At positive depth, primitive labels are equivalent exactly when they are equal; a primitive and a box are never equivalent. Two box labels are equivalent when their metadata agree and there exists an isomorphism of their bodies whose corresponding labels are recursively equivalent at the smaller depth. This is a well-founded definition because the depth decreases whenever a box body is compared.

Lean: `Linear.LabelEquiv`

Definition 23.6 (Recursive hypernet isomorphism)**.** A recursive isomorphism $G \cong_d H$, represented by DeepIso, is an isomorphism relative to recursive label equivalence at depth d. Its outer carrier bijections are explicit data. The label proof at a box supplies the existence of a further body isomorphism, independently of the names chosen at any other box. Every level preserves its own ordered boundary positions.

Lean: `Linear.DeepIso`

Lemma 23.7 (Equivalence laws for recursive labels)**.** *Recursive label equivalence is reflexive, symmetric, and transitive.*

Proof. Induct on depth. At depth zero the claims are equality laws. At positive depth, primitive cases again reduce to equality and mixed primitive/box cases are impossible. For box reflexivity, the induction hypothesis supplies reflexivity of the body-label relation, so the identity relative isomorphism witnesses body equivalence. For box symmetry, take a witnessing body isomorphism and invert it using symmetry at the smaller depth. For transitivity, matching the two box-relation constructors identifies all metadata, so the two body isomorphisms have a common middle graph and interface. Compose them using transitivity at the smaller depth. Package each resulting body isomorphism in the box relation. □

Lean: `Linear.labelEquiv_refl`

Lean: `Linear.labelEquiv_symm`

Lean: `Linear.labelEquiv_trans`

Identity, inverse, and composition of recursive graph isomorphisms follow by instantiating the generic constructions with these three laws. Ordinary isomorphisms embed into recursive ones by reflexivity of the label relation.

Lean: `Linear.DeepIso.refl`

Lean: `Linear.DeepIso.symm`

Lean: `Linear.DeepIso.trans`

Lean: `Linear.DeepIso.ofIso`

Lemma 23.8 (Signatures and operation labels are invariant)**.** *Equivalent labels have equal ordered input signatures, equal ordered output signatures, and the same operation-label predicate.*

Proof. At depth zero or for two primitives, substitute the label equality. Mixed labels are impossible. For two boxes, the relation constructor identifies the bound/free metadata, and both signatures depend only on those numbers. Both box labels fail the operation predicate. No information about body carrier names is used in these conclusions. □

Lean: `Linear.labelEquiv_input`

Lean: `Linear.labelEquiv_output`

Lean: `Linear.labelEquiv_isOperation`

23.3 Operation traces and paths

Theorem 23.9 (Recursive renaming preserves and reflects operation paths)**.** *Let $f : G \cong_d H$, with port map v and edge bijection e. An operation trace from x to y along the edge list ℓ in G exists exactly when the trace from $v(x)$ to $v(y)$ along $\mathsf{map}(e, \ell)$ exists in H. The analogous statement holds for forward operation paths.*

Proof. First prove forward trace transport by induction on the trace. An empty trace maps to an empty trace. For a nonempty trace, operation-label invariance supplies the required label predicate. Source and target preservation map its two incidence witnesses, and induction supplies the remaining trace.

For reflection transport the target trace through the inverse recursive isomorphism. The inverse port and edge maps cancel the original maps, recovering both original endpoints and exactly the original edge list. A forward operation path is the reverse-oriented trace with its edge list reversed. List mapping commutes with reversal, so the trace equivalence gives the path equivalence. In particular, input and output endpoints retain their boundary indices. □

Lean: `Linear.DeepIso.operationTrace`

Lean: `Linear.DeepIso.operationTrace_iff`

Lean: `Linear.DeepIso.operationPath_iff`

23.4 Transporting occurrences and holes through all boxes

Definition 23.10 (Fibre and hole embeddings)**.** A fibre embedding maps positions in one label to positions in another, injectively, and preserves their complete hole observations, including the absence of a hole. A hole embedding similarly maps graph occurrences injectively and preserves their observations. These maps concern occurrence addresses; their definition does not itself assert any incidence property.

Lean: `Linear.FiberEmbedding`

Lean: `Linear.HoleEmbedding`

Lemma 23.11 (Assembling fibre embeddings)**.** *An injective map of outer edges and a fibre embedding over each mapped edge together give a hole embedding of graphs.*

Proof. Map (a, x) to the mapped edge paired with the image of x under its fibre embedding. If two results coincide, projecting their edges and using edge-map injectivity identifies the original edges. Their remaining fibre positions now have the same type, and fibre-map injectivity identifies them. Observation preservation is exactly the preservation property of the corresponding fibre embedding. □

Lean: `Linear.HoleEmbedding.ofFibers`

A hole embedding also transports the original deep addresses: first use the occurrence-coordinate bijection of the source, apply the hole embedding, then use the inverse coordinate bijection of the target. This defines HoleEmbedding.allEdge.

Lean: `Linear.HoleEmbedding.allEdge`

The resulting address map is injective. Apply the target coordinate bijection to an equality of images, cancel its inverse, use embedding injectivity, and finally use source-coordinate

injectivity. It preserves hole lookup because both coordinate bijections preserve observation and the middle embedding does so as well.

Lean: `Linear.HoleEmbedding.allEdge_injective`

Lean: `Linear.HoleEmbedding.allEdge_hole`

Lemma 23.12 (Recursive labels admit fibre embeddings)**.** *If two labels are recursively equivalent, an injective, observation-preserving map of their fibres exists.*

Proof. Induct on depth. Equal primitives use the identity map. Mixed label cases are impossible. For equivalent boxes, choose a witnessing isomorphism of their bodies. For each body edge, the induction hypothesis supplies a fibre embedding from its recursive label-equivalence proof; choose one such map per edge. Assemble these maps over the body isomorphism's injective edge map to obtain a body hole embedding, and express it in original deep-address coordinates.

Map the box's shallow position to the shallow position, and each child address through this body map. The two alternatives remain disjoint. Within the shallow alternative equality is immediate; within the child alternative injectivity is the proved injectivity of body-address transport. Shallow observations are empty on both sides, while child observations agree by the body's transport theorem. This constructs the desired embedding. Classical choice selects witnesses of the existential body-isomorphism and embedding statements; it does not supply any graph invariant as an additional assumption. □

Lean: `Linear.labelEquiv_hasEmbedding`

The noncomputable function labelEquiv_embedding chooses the embedding whose existence was just proved. Applying it at every outer edge of a recursive isomorphism, and assembling over its edge bijection, defines DeepIso.holeEmbedding.

Lean: `Linear.labelEquiv_embedding`

Lean: `Linear.DeepIso.holeEmbedding`

Lemma 23.13 (Presence transports and uniqueness reflects)**.** *A hole embedding transports every present hole label from source to target. If the target has unique hole observations, so does the source.*

Proof. Map the occurrence witnessing a present label and use preservation of lookup. For uniqueness, map two source occurrences with equal hole observations. Target uniqueness makes their images equal, and embedding injectivity makes the original occurrences equal. □

Lean: `Linear.HoleEmbedding.hasHole`

Lean: `Linear.HoleEmbedding.unique`

Theorem 23.14 (Hole and context invariance under recursive isomorphism)**.** *Recursively isomorphic graphs have identical hole inventories. One has unique holes if and only if the other does; one is hole-free if and only if the other is.*

Proof. Construct hole embeddings from the isomorphism and its inverse. Presence transport gives the two implications for inventory equality. Uniqueness reflection, in the two directions, gives the context equivalence. Negating hole presence for every label gives the hole-freeness equivalence. The two chosen embeddings are not asserted to be mutually inverse; the two implications and injectivity are sufficient for these results. □

Lean: `Linear.DeepIso.hasHole_iff`

Lean: `Linear.DeepIso.uniqueHoles_iff`

Lean: `Linear.DeepIso.holeFree_iff`

23.5 Congruence of shallow and deep replacement

For a fixed host C and shallow selected edge h, an R-isomorphism $f : H \cong_R K$ gives the candidate carrier bijections for the plugged graphs: use identity on the host summand and f on the replacement summand. For edges, the host summand is the subtype of edges different from h.

Lean: `Net.RelIsoPlug.vertices`

Lean: `Net.RelIsoPlug.edges`

The induced port map commutes with host embedding, by a case split on internal, input, and output ports. It commutes with insertion after the replacement's own port map: an internal port uses the right summand, and a boundary port reduces to the host-embedding equation at the selected source or target port.

Lean: `Net.RelIsoPlug.map_host`

Lean: `Net.RelIsoPlug.map_insert`

Theorem 23.15 (Shallow congruence for related labels)**.** *For reflexive R, an R-isomorphism $H \cong_R K$ induces $C[h := H] \cong_R C[h := K]$. No non-self-loop condition is required.*

Proof. Use the preceding sum bijections. Host internal labels are unchanged and replacement internal labels are preserved by f. Host edge labels satisfy reflexivity of R, and inserted edge labels use the relation supplied by f. For a host incidence list, commute the port map with host embedding pointwise. For an inserted incidence list, commute it with insertion and then use the source or target preservation equation of f. This establishes every field of the resulting relative isomorphism without requiring insertion to be globally injective. □

Lean: `Net.RelIso.plug`

Lemma 23.16 (Box congruence)**.** *Boxing recursively isomorphic bodies with the same metadata gives recursively isomorphic boxes. Replacing one fixed box edge in a surrounding graph by either of those bodies also gives recursively isomorphic graphs.*

Proof. For a single boxed graph, use identity maps on its empty internal carrier and one-edge carrier. Its labels are related by the body-isomorphism witness, and its boundary lists depend only on the common metadata. In a surrounding graph, use identity maps on both outer carriers. At the selected edge use the related box-body witness; at every other edge use reflexivity of recursive label equivalence. The two outer incidence structures are identical, so their preservation equations reduce to mapping a list by identity. □

Lean: `Linear.DeepIso.box`

Lean: `Linear.replaceBox_deepIso`

Theorem 23.17 (Congruence at every addressed depth)**.** *For a fixed host G and typed site s, if $H \cong_r K$, then*

$$G[s := H] \cong_d G[s := K].$$

No context-uniqueness, freshness, or non-self-loop assumption is needed.

Proof. Induct on the site. A shallow site uses the generic shallow congruence with the reflexive recursive-label relation. For a site inside a box, induction gives an isomorphism between the two changed bodies. Lift it through the selected box using box congruence. Repeating this step crosses every enclosing box in the address. □

Lean: `Linear.plugAt_deepIso`

Two consequences connect this congruence with the preceding milestone. Exact site freshness is invariant when the replacement is changed by recursive isomorphism: the retained host predicate stays fixed, and the replacement hole inventories coincide. Context validity of the two plugged results is also equivalent, by context invariance applied to their constructed recursive isomorphism. The latter statement does not assume the host is a context.

Lean: `Linear.siteFresh_deepIso_iff`

Lean: `Linear.plugAt_deepIso_unique_iff`

23.6 Graphs and replacement modulo recursive renaming

Definition 23.18 (Recursive isomorphism classes)**.** At fixed depth d and interface (I, O), declare $G \sim H$ when a recursive isomorphism exists. Identity, inverse, and composition prove that this is an equivalence relation, packaged as deepSetoid. Its quotient is HypernetModuloIso, the type of graphs modulo independent carrier renaming at all levels of their hierarchy. Write $[G]$ for the class of G.

Lean: `Linear.deepSetoid`

Lean: `Linear.HypernetModuloIso`

Definition 23.19 (Plugging a replacement class)**.** For a fixed concrete host and site s, define

$$\mathsf{plugAtModuloIso}(s, [H]) = [G[s := H]].$$

Well-definedness. If H and K represent the same source class, their recursive isomorphism induces one between the plugged graphs by deep congruence. The results therefore belong to the same target class. Quotient elimination defines the displayed operation from this representative-independent function. Its value on a concrete representative is the displayed equation by the computation rule of quotient elimination. □

Lean: `Linear.plugAtModuloIso`

Lean: `Linear.plugAtModuloIso_mk`

Context validity descends to classes as well: define $\mathsf{moduloIsoUniqueHoles}([G])$ to mean that G has unique holes. The isomorphism-invariance theorem makes these propositions equivalent for any two representatives; propositional extensionality supplies the equality required by quotient elimination.

Lean: `Linear.moduloIsoUniqueHoles`

These constructions quotient the replacement argument for a fixed host and address. Section 24 extends invariance to corresponding sites in isomorphic hosts. Section 25 proves associativity and units. A quotient of host/site pairs remains a separate obligation.

23.7 A strict extension of literal-label isomorphism

Use the earlier two-hole chain. The permutation $i \mapsto 1 - i$ on its two-edge carrier exchanges edge names. Applying it twice is identity because $0 \leq i < 2$. Reindex the chain's edges by this bijection and keep its internal vertex name unchanged; call the result H'. The existing reindexing isomorphism, embedded into recursive isomorphism, witnesses $H \cong_0 H'$.

Lean: `Linear.Examples.swapTwo`

Lean: `Linear.Examples.renamedTwoHoles`

Lean: `Linear.Examples.twoHoles_deepIso`

The records H and H' are unequal. If they were equal, their edge-label functions would agree at the edge named zero. Reading the hole name there gives 0 in H and 1 in H', a contradiction.

Lean: `Linear.Examples.renamedTwoHoles_ne`

Box H' with zero bound variables and one free output. Box congruence relates this box to the earlier box of H.

Lean: `Linear.Examples.renamedBoxedTwoHoles`

Lean: `Linear.Examples.boxedTwoHoles_deepIso`

No literal label-preserving isomorphism relates these two boxed graphs. Such an isomorphism's label equation at the sole source edge would make their box labels equal. Equality of these labels forces equality of the body records, contrary to the preceding result. This proves that recursive isomorphism identifies strictly more graphs than the earlier literal-label notion.

Lean: `Linear.Examples.boxedTwoHoles_no_literalIso`

The renamed box remains a context by recursive invariance of unique holes. The two different

box records represent the same quotient class by quotient soundness applied to their recursive isomorphism.

Lean: `Linear.Examples.renamedBoxedTwoHoles_unique`

Lean: `Linear.Examples.renamedBoxes_same_class`

For a nontrivial addressed replacement, use the earlier site selecting the first hole inside a box. Substituting H or H' there gives recursively isomorphic results by deep congruence. Finally, the primitive operations named value and otherValue are not equivalent: at depth zero recursive label equivalence is equality, and these are distinct constructors. Thus carrier renaming does not erase semantic operation labels.

Lean: `Linear.Examples.deep_plug_renaming`

Lean: `Linear.Examples.distinct_operations_not_equivalent`

24 Transporting replacement sites through isomorphisms

An address denotes an occurrence, not a permanent numerical edge name. If an isomorphism exchanges two edges, the selected occurrence must move with it. This section proves existence of that transport at arbitrary box depth, congruence when both host and replacement change, and invariance of the exact freshness condition. Depth bounds and ordered outer interfaces remain fixed.

24.1 Transport of boundary indices and shallow hosts

Definition 24.1 (Boundary transport)**.** Given equalities $I = I'$ and $O = O'$ of ordered interfaces, transport a port by fixing its internal name, or by retaining its numerical input or output index and using the equality to prove the new bound. Transport a graph to the new interface by keeping its internal vertices, edges, enumerations, and labels, and mapping every incidence list through this port transport. This graph is written $H^{I',O'}$ below.

All graph invariants follow by eliminating the two interface equalities: the port map is then the identity, and every invariant is exactly the original one. Reflexive port transport is pointwise the identity, hence is the identity function by function extensionality. Reflexive graph transport is equal to the original graph by record equality and the identity law for list mapping. Keeping carriers definitionally unchanged makes the construction useful even before the interface equality proofs have been eliminated.

Lean: `Port.cast`

Lean: `Port.cast_refl`

Lean: `Port.cast_refl_fun`

Lean: `Net.reboundary`

Lean: `Net.reboundary_refl`

Definition 24.2 (Complement restriction)**.** For a bijection $f : A \to B$ and $a \in A$, restrict f to a bijection $A \setminus \{a\} \to B \setminus \{f(a)\}$. Its inverse is the restriction of f^{-1}.

Injectivity shows that a surviving source cannot map to $f(a)$. Conversely, if $f^{-1}(b) = a$, the right inverse equation gives $b = f(a)$, contradicting survival. Both inverse laws follow from those of f, since subtype elements with equal underlying values are equal.

Lean: `Bijection.except`

Lemma 24.3 (Transport of attachment positions)**.** *Let $f : C \cong_R D$ and let e be a selected edge. If its corresponding input interfaces are equal, mapping the ith source port of e by f gives the ith source port of $f(e)$. The analogous assertion holds for targets when the output interfaces are equal.*

Proof. Apply optional list lookup at index i to the source-list equation of f. The typing invariant identifies the source-list length with the input-interface length, so both lookups are in bounds. Lookup commutes with mapping; cancelling the optional-result constructor gives the port equation. The interface equality supplies the bound for the target index. Repeat for the target-list equation. No disjointness between source and target lists is used. □

Lean: `Net.RelIso.sourcePort`

Lean: `Net.RelIso.targetPort`

Theorem 24.4 (Shallow host congruence)**.** *Suppose $f : C \cong_R D$, R is reflexive, and corresponding selected edges have equal input and output interfaces. For every replacement A of the source interface,*

$$C[e := A] \cong_R D[f(e) := A^{I',O'}].$$

Proof. On internal vertices use the disjoint sum of the host vertex bijection and the identity on replacement vertices. On edges use the complement restriction of the host edge bijection and the identity on replacement edges. These are the auxiliary maps `HostPlug.vertices` and `HostPlug.edges`. The whole-graph port map commutes with host inclusion by case analysis on a port. It also commutes with replacement insertion followed by boundary transport: internal vertices are immediate, and input and output ports use the attachment-position lemmas just proved.

For surviving edges, label relatedness is supplied by f; for inserted edges, use reflexivity of R. Internal vertex labels are preserved by the host isomorphism or by identity. For incidence of a surviving edge, rewrite with f's incidence equation, combine successive list maps, and apply commutation with host inclusion. For an inserted edge, combine the insertion and whole-port maps and use commutation with replacement insertion. The same argument proves the target equation. Self-loops are allowed throughout. □

Lean: `Net.HostPlug.vertices`

Lean: `Net.HostPlug.edges`

Lean: `Net.HostPlug.map_host`

Lean: `Net.HostPlug.map_insert`

Lean: `Net.RelIso.plugHost`

24.2 Structural correspondence of addresses

Definition 24.5 (Address interface transport)**.** Given $s : \mathsf{Site}(G, r, J, K)$ and equalities $J = J'$, $K = K'$, transport its interface indices to obtain an address of type $\mathsf{Site}(G, r, J', K')$. The selected occurrence is unchanged. Plugging A at this transported address equals plugging the boundary-transported A at the original address.

The definition is equality elimination. For the plugging equation eliminate the two equalities and use reflexive graph boundary transport; both sides then become the same substitution.

Lean: `Linear.Site.reinterface`

Lean: `Linear.plugAt_reinterface`

Lemma 24.6 (Corresponding boxes and their replacement)**.** *If $f : G \cong_{d+1} H$ and edge e of G carries box (n, m, B), then $f(e)$ carries a box (n, m, B') with $B \cong_d B'$. Moreover, replacing these two box bodies by any $A \cong_d A'$ gives recursively isomorphic whole graphs.*

Proof. For existence, inspect the target label in the label-relatedness proof for e. A primitive target contradicts the definition of recursive label equivalence. For a box target, the constructor of the box relation forces matching n, m and supplies a body isomorphism. This reasoning constructs an existential statement, so elimination of the propositional box relation is legitimate.

For replacement retain the vertex and edge bijections of f. At e the new labels are related by the given isomorphism $A \cong_d A'$. At every other edge injectivity ensures that its target is different from $f(e)$, so both labels remain unchanged and their original relation applies. Neither replacement changes incidence or internal vertex labels; the remaining equations are exactly those of f. □

Lean: `Linear.DeepIso.boxAt`

Lean: `Linear.DeepIso.replaceBox`

Definition 24.7 (Corresponding sites)**.** The proposition $s \simeq t$, implemented by `SiteIso`, is generated by two structural rules. A shallow site selecting e corresponds to the site selecting $f(e)$ for any recursive host isomorphism f; transport the latter's interface indices back along the signature equalities. For sites inside boxes, use a recursive outer-host isomorphism, corresponding selected box edges with equal binding/free metadata, and structurally corresponding inner sites. Both sites have the same local depth and replacement interface.

This is a relation on addresses themselves, defined before proving anything about the results of plugging. An inner correspondence may choose any suitable isomorphism between the box bodies; the outer relation stores existence of a body isomorphism, not a distinguished naming map for every inner vertex.

Lean: `Linear.SiteIso`

Theorem 24.8 (Existence of transported sites)**.** *For every recursive isomorphism* $f : G \cong_d H$ *and every site* $s : \mathsf{Site}(G, r, J, K)$*, there exists* $t : \mathsf{Site}(H, r, J, K)$ *with* $s \simeq t$*.*

Proof. Induct on s. For a shallow site use the shallow correspondence constructor and the proved equality of corresponding signatures. For an inside site, extract the corresponding target box and a body isomorphism using the preceding lemma. Apply the induction hypothesis to the inner site and that body isomorphism, then assemble the outer correspondence with the inside constructor. This follows a finite address down the hierarchy and therefore terminates. □

Lean: `Linear.Site.transport_exists`

Definition 24.9 (Chosen site transport)**.** Write s^f for a site chosen from this existence theorem by classical choice. It has the original local depth and interface in the target host, and satisfies $s \simeq s^f$ by the specification theorem for choice.

The construction is noncomputable. No uniqueness, definitional round-trip law, or canonical choice of inner box isomorphisms is asserted.

Lean: `Linear.Site.transport`

Lean: `Linear.Site.transport_spec`

24.3 Substitution and freshness under transport

Theorem 24.10 (Simultaneous host and replacement congruence)**.** *If* $s \simeq t$ *and* $A \cong_r B$*, then*

$$\mathsf{plugAt}(s, A) \cong_d \mathsf{plugAt}(t, B).$$

Proof. First take the same replacement on both sides and induct on the structural correspondence. In the shallow case rewrite plugging at a transported interface using its equation, then apply shallow host congruence. Recursive label equivalence supplies the signature equalities and reflexivity required there. In the inside case the induction hypothesis supplies an isomorphism of the replaced inner bodies. Lift it through the corresponding outer boxes using box replacement congruence. This proves `SiteIso.plug`. For distinct isomorphic replacements, compose that isomorphism with the previously proved congruence in the replacement at the fixed target site t. □

Lean: `Linear.SiteIso.plug`

Lean: `Linear.SiteIso.plug_congr`

Lemma 24.11 (Quotient substitution agrees after host transport)**.** *For every replacement isomorphism class* $[A]$*,* $\mathsf{plugAtModuloIso}(s, [A]) = \mathsf{plugAtModuloIso}(s^f, [A])$*.*

Proof. Use quotient induction to choose a representative A. The specification of s^f gives a structural correspondence; substitution congruence gives a recursive isomorphism between the two results. Quotient soundness identifies their classes. □

This is invariance under transporting the host/site pair. A separate quotient type of host/site pairs has not been constructed. Section 25 proves the nested composition and unit laws.

Lean: `Linear.plugAtModuloIso_transport`

Lemma 24.12 (Invariance of retained holes)**.** *A recursive host isomorphism preserves and reflects the holes outside a selected outer edge. Structurally corresponding sites retain exactly the same hole labels at every depth.*

Proof. An outer retained occurrence consists of an edge $x \neq e$ and a hole-observing position in its label. Map x by the host edge bijection and the position by the previously constructed label-fibre embedding. Injectivity preserves $x \neq e$, and the embedding preserves its hole observation. For reflection, use the inverse edge bijection and the inverse isomorphism's fibre embedding; the right inverse equation ensures that the preimage survives.

Interface transport does not change retention: eliminate the interface equalities. Now induct on structural site correspondence. The shallow case uses this equality elimination and the outer retained-hole equivalence. The inside case uses the definition of deep retention as a disjunction: either the occurrence lies under a surviving outer edge, or it is retained by the inner site. Apply the outer equivalence and the induction hypothesis to the two disjuncts. □

Lean: `Linear.DeepIso.retainedHole_iff`

Lean: `Linear.SiteRetainedHole_reinterface`

Lean: `Linear.SiteIso.retainedHole_iff`

Lemma 24.13 (Invariance of freshness and result validity)**.** *For $s \simeq t$ and $A \cong_r B$, exact freshness holds for (s, A) if and only if it holds for (t, B). The two substituted results satisfy global unique-hole validity together, whether or not either original host is valid.*

Proof. Freshness forbids a label occurring both in the retained host and in the replacement. Transport the first occurrence with retained-hole equivalence and the second with the replacement isomorphism's hole-inventory equivalence. This proves each direction. For result validity, simultaneous substitution congruence supplies a recursive isomorphism; apply its preservation and reflection of unique holes. No original-host validity hypothesis is needed for this invariance statement. □

Lean: `Linear.SiteIso.fresh_iff`

Lean: `Linear.SiteIso.plug_unique_iff`

24.4 A moved address and a wrong-address counterexample

Use the two-hole chain and its edge swap from the preceding section. The first hole, formerly edge zero, is edge one after renaming. Define the target deep site to enter the renamed box and select edge one. The outer boxed isomorphism and the shallow correspondence for the edge swap prove structural correspondence with the original deep site.

Lean: `Linear.Examples.renamedFirstSite`

Lean: `Linear.Examples.firstSites_correspond`

Simultaneous congruence now allows both the host body and the inserted chain to be renamed independently. The replacement consisting of hole one still collides with the retained hole one:

freshness invariance transports the previously proved failure of freshness to the new site.

Lean: `Linear.Examples.simultaneous_deep_renaming`

Lean: `Linear.Examples.renamed_collision_not_fresh`

For contrast within this example, define the wrong target site by keeping the old numerical name zero. It selects hole one instead of hole zero. After inserting the one-hole replacement, the wrong result retains hole zero, witnessed by the surviving edge one inside the renamed box. The exact deep inventory theorem therefore proves that hole zero is present. In the original substitution, hole zero is consumed and absent, as already proved. A recursive isomorphism would preserve hole presence, contradicting these two facts. Thus the wrong-address substitution is not recursively isomorphic to the original substitution; moving the address is essential.

Lean: `Linear.Examples.wrongRenamedSite`

Lean: `Linear.Examples.wrong_address_changes_result`

25 Associativity, units, and exact context composition

We now compose addressed substitutions, rather than merely compare renamed versions of one substitution. If s selects an occurrence in G, inserting H there creates a copy of every occurrence of H. A site t in H thus induces a site in $\mathsf{plugAt}(s, H)$. The main theorem identifies substitution at that induced site with first substituting within H and then inserting the result at s. The identification is a concrete recursive isomorphism; on isomorphism classes it is equality. We also prove the corolla unit laws and an exact context-validity criterion for the composite, including cases where an intermediate collision is removed by the final substitution.

25.1 Shallow associativity with explicit carrier bijections

Let C be a graph, e a selected edge, H a replacement of its interface, f an edge of H, and A a replacement of the interface of f. Compare

$$L = (C[e := H])[\mathsf{inr}(f) := A], \qquad R = C[e := H[f := A]].$$

These have the same outer interface, but different carrier records.

Definition 25.1 (Reassociation bijections)**.** The internal-vertex bijection from L to R is

$$(C_V + H_V) + A_V \longrightarrow C_V + (H_V + A_V),$$

which keeps each original name and changes only its sum tags. Its three cases send a host vertex to the host summand, a middle vertex to the middle summand, and a final vertex to the final summand. The edge bijection does the same for surviving host edges, surviving middle edges, and final inserted edges.

More explicitly, the left surviving-edge carrier consists of edges of $(C_E \setminus \{e\}) + H_E$ other than $\mathsf{inr}(f)$. A surviving host edge maps to the host summand on the right. An edge $\mathsf{inr}(y)$ maps to the surviving-middle summand, with $y \neq f$ obtained by injectivity of the sum constructor. Final edges map to final edges. The inverse maps reverse these three cases; inverse equations are proved by cases, with proof irrelevance identifying the subtype certificates.

Lean: `Net.PlugAssoc.vertices`

Lean: `Net.PlugAssoc.edges`

Lemma 25.2 (Attachment and port commutation)**.** *The source and target positions of the inserted edge* $\mathsf{inr}(f)$ *are the images, under insertion into* C*, of the corresponding positions of* f *in* H*. The reassociation port map commutes with inclusion of original host ports, inclusion of middle ports, and insertion of final replacement ports.*

Proof. The attachment assertions unfold positional lookup in the inserted incidence lists and use lookup through a list map. For host ports, case analysis on internal, input, and output ports proves the equation directly. For middle ports, internal vertices are immediate; an input or output is attached to a host port, so use the host equation. For final ports, internal vertices are again immediate. A boundary port attaches at f; rewrite that attachment by the positional lookup assertion and apply the middle-port equation. No argument assumes that an input attachment differs from an output attachment. □

Lean: `Net.PlugAssoc.sourcePort_inserted`

Lean: `Net.PlugAssoc.targetPort_inserted`

Lean: `Net.PlugAssoc.map_host`

Lean: `Net.PlugAssoc.map_middle`

Lean: `Net.PlugAssoc.map_insert`

Theorem 25.3 (Shallow associativity)**.** *There is a literal-label graph isomorphism* $L \cong R$*.*

Proof. Use the two reassociation bijections. Vertex and edge labels agree by the three carrier cases. For sources, split an edge into surviving host, surviving middle, or final replacement. In the host case the left source list maps the original host list through two inclusions and reassociation; combine list maps and apply host commutation. The middle case uses middle commutation, and the final case uses final insertion commutation. These give exactly the three right-hand source lists. The target proof is identical with target lists. The isomorphism preserves literal labels and is valid for self-loops and cycles. □

Lean: `Net.plug_assoc`

25.2 Composition at arbitrary box depth

Lemma 25.4 (Overwrite and insertion of box bodies)**.** *Replacing the same box body twice has exactly the effect of the last replacement. Changing the body of an inserted box commutes, up to literal-label isomorphism, with changing that body before shallow insertion.*

Proof. For overwrite, the two graphs have the same carriers, incidence, and internal labels. At the selected edge both final labels contain the last body. At any other edge both conditional updates keep the original label. Extensionality of the label function and proof irrelevance give equality of the graph records.

For commutation with insertion use identity carrier bijections. Surviving host labels are unchanged. On inserted edges the two tests for being the selected edge agree, because $\mathsf{inr}(x) = \mathsf{inr}(f)$ exactly when $x = f$. Incidence is unchanged by box replacement. The insertion functions also agree: internal ports have the same tags, and boundary ports attach to the same outer host positions. List mapping then proves both incidence equations. □

Lean: `Linear.replaceBox_overwrite`

Lean: `Linear.plug_replaceBox_iso`

Definition 25.5 (Composed site)**.** Let $s : \mathsf{Site}(G, r, J, K)$ and $t : \mathsf{Site}(H, q, U, V)$, where H has the replacement interface of s. Define

$$s \star t : \mathsf{Site}(\mathsf{plugAt}(s, H), q, U, V).$$

If s is shallow and t is shallow, select the inserted edge $\mathsf{inr}(f)$. If s is shallow and t enters a box, enter the corresponding inserted box and retain t's inner address. If s enters a box, rebuild that box with the first replacement already performed, then descend to the recursively composed inner site.

This is recursion on the outer address. The new enclosing box carries exactly the body produced by the first substitution, so the box-label equation is the selected branch of the update. Local depth and interface indices are preserved by construction. Although the surrounding graph operations use classical finite-subset constructions, no isomorphism or target address is selected by an existence theorem here.

Lean: `Linear.Site.comp`

Theorem 25.6 (Deep associativity)**.** *For every compatible s, t, A,*

$$\mathsf{plugAt}(s \star t, A) \cong_d \mathsf{plugAt}(s, \mathsf{plugAt}(t, A)).$$

No uniqueness, freshness, or non-self-loop premise is required.

Proof. Induct on s. When s and t are shallow use the explicit shallow associator and embed its literal-label isomorphism into recursive isomorphism. When s is shallow and t enters a box, use the box/insertion commutation lemma. When s enters a box, the left expression first updates that box to contain $\mathsf{plugAt}(s_{\mathrm{inner}}, H)$ and later updates it again after following the composed inner address. Overwrite reduces these two updates to one. The induction hypothesis compares its final body to the body obtained by performing the inner substitution before the outer one. Box replacement congruence lifts that body isomorphism through the enclosing box. □

Lean: `Linear.plugAt_assoc`

Lemma 25.7 (Quotient composition and three-stage coherence)**.** *On recursive isomorphism classes,*

$$P_{s\star t}([A]) = P_s(P_t([A])), \qquad P_s = \mathsf{plugAtModuloIso}(s, -).$$

For a third compatible site u, the two functions associated with $(s \star t) \star u$ and $s \star (t \star u)$ also agree on every replacement class.

Proof. For the binary equation use quotient induction and quotient soundness applied to deep associativity on representatives. For three stages, apply the binary equation twice to each side; both reduce to $P_s(P_t(P_u([A])))$. The ambient concrete hosts of the two composite addresses can differ, so this is a statement about the induced quotient results, not literal equality of address records in different hosts. □

Lean: `Linear.plugAtModuloIso_comp`

Lean: `Linear.plugAtModuloIso_comp_three`

The selected-hole observation of $s \star t$ equals that of t. Induct on s; for shallow s, either form of t retains the same selected label, and for inside s the definition discards the enclosing box and the induction hypothesis applies.

Lean: `Linear.Site.selectedHole_comp`

25.3 Exact inventories and context validity of composites

Write $R_s(h)$ for $\mathsf{SiteRetainedHole}(s, h)$ and $H_A(h)$ for $\mathsf{HasHole}(A, h)$.

Lemma 25.8 (Retained inventories compose by union)**.** *Removing an inserted shallow edge* $\mathsf{inr}(f)$ *retains exactly the holes under surviving original-host edges and surviving middle-graph edges. More generally,*

$$R_{s\star t}(h) \quad \Longleftrightarrow \quad R_s(h) \vee R_t(h).$$

Consequently the final hole inventory is exactly $R_s(h) \vee R_t(h) \vee H_A(h)$.

Proof. For shallow inserted-edge removal, decompose a retained occurrence by its outer edge tag. A surviving host tag already supplies its inequality with the original selected edge. An inserted tag survives precisely when its middle edge differs from f. Its fibre position and hole observation are unchanged. Conversely, construct the appropriate tagged edge from either of these witnesses.

For the general statement induct on s. A shallow s and shallow t give the preceding result directly. If t enters a box, append its retained inner inventory and reassociate disjunction. For inside s, replacement of its selected box preserves all retained outer-edge fibres; apply that fact and the induction hypothesis to the inner composite, then reassociate disjunction. Finally apply the exact inventory theorem for replacement at $s \star t$. □

Lean: `Linear.retainedHole_inserted`

Lean: `Linear.siteRetainedHole_comp`

Lean: `Linear.plugAt_comp_hasHole_iff`

Lemma 25.9 (Composite freshness)**.** *Exact freshness for* $(s \star t, A)$ *is equivalent to both*

$$\forall h,\ \neg(R_s(h) \wedge H_A(h)) \quad \textit{and} \quad \mathsf{SiteFresh}(t, A).$$

Proof. Use the retained-union theorem. In the forward direction include either retained inventory into the union and apply composite freshness. In the reverse direction, split a retained witness according to the union and apply the corresponding freshness assumption. □

Lean: `Linear.siteFresh_comp_iff`

Theorem 25.10 (Sharp context closure for composition)**.** *Suppose G and H have unique holes. Then* $\mathsf{plugAt}(s \star t, A)$ *has unique holes if and only if all three conditions hold:*

1. *A has unique holes;*
2. *A is fresh for the composite site $s \star t$;*
3. $\forall h,\ \neg(R_s(h) \wedge R_t(h))$.

Thus the three inventories R_s, R_t, H_A must be pairwise disjoint. The intermediate graph $\mathsf{plugAt}(s, H)$ *need not have unique holes.*

Proof. By deep associativity it suffices to check uniqueness of $\mathsf{plugAt}(s, \mathsf{plugAt}(t, A))$. Apply sharp context closure first with host G, then with host H. This gives uniqueness of A, freshness for (t, A), and disjointness of R_s from the hole inventory of $\mathsf{plugAt}(t, A)$. The latter inventory is $R_t \cup H_A$ by the exact inventory theorem. Splitting that union gives disjointness of R_s from R_t and from H_A. Combine the last condition with freshness for (t, A) using the composite-freshness lemma. Conversely, reverse each implication: the pairwise exclusions make the inner result fresh for s, and both applications of sharp closure establish the final result's uniqueness. □

Lean: `Linear.plugAt_comp_unique_iff`

Definition 25.11 (Certified composed plugging)**.** Given contexts G, H, A, compatible sites s, t, composite freshness, and disjoint retained inventories R_s, R_t, package $\mathsf{plugAt}(s \star t, A)$ with the uniqueness proof just obtained. This defines `Context.plugAtComp`; it requires no certificate that the intermediate plugged graph is a context.

Lean: `Linear.Context.plugAtComp`

25.4 Corolla units

Definition 25.12 (Substitution unit)**.** For a signature label o, let $\mathbf{c}_o$ be its one-edge corolla, with its sole edge selected. This is a substitution unit of interface $(\sigma_{\text{in}}(o), \sigma_{\text{out}}(o))$. It is an addressed graph, with no uniqueness premise on its label; no free wire identifying an input vertex with an output vertex is introduced.

We prove $\mathbf{c}_o[0 := H] \cong H$ and $C[e := \mathbf{c}_{\ell(e)}] \cong C$ using literal-label isomorphisms.

A corolla has no surviving edge after removing zero: an element of $\{i : \mathsf{Fin}(1) \mid i \neq 0\}$ contradicts the fact that every index below one is zero. Its ith source and target attachment positions are its input and output ports at i, respectively. These equations follow by unfolding lookup in the mapped finite-range lists.

Lean: `Net.PlugUnit.noEdge`

Lean: `Net.PlugUnit.corolla_sourcePort`

Lean: `Net.PlugUnit.corolla_targetPort`

Lemma 25.13 (Left unit)**.** *Replacing the sole edge of* $\mathbf{c}_o$ *by* H *yields a graph isomorphic to* H*.*

Proof. The vertex bijection discards the empty left summand in $\varnothing + H_V$. The edge bijection discards the impossible surviving-corolla summand and retains H_E. Their inverses include into the right summands. Both inverse laws follow by cases, eliminating impossible elements.

Mapping an inserted port through the vertex projection gives the original port: internal ports are immediate; input and output ports use the corolla attachment equations. For each inserted edge, combine the incidence list maps and apply this port equation, leaving the original source or target list. Surviving corolla edges are impossible. All labels agree directly on the remaining summands. □

Lean: `Net.PlugUnit.leftVertices`

Lean: `Net.PlugUnit.leftEdges`

Lean: `Net.PlugUnit.left_insert`

Lean: `Net.corolla_plug_iso`

Lemma 25.14 (Right unit)**.** *Replacing* e *in* C *by its own labelled corolla yields a graph isomorphic to* C*.*

Proof. The vertex bijection discards the empty right summand. The edge bijection maps each surviving host edge to itself and the inserted corolla edge to e. Its inverse tests whether an edge equals e: if so use the sole corolla edge; otherwise use the surviving-host subtype. The inverse laws follow from this test and uniqueness of an element of $\mathsf{Fin}(1)$.

The vertex projection cancels inclusion of host ports. Enumerating all source positions of e reconstructs C's source list: the lengths agree by typing, and each indexed element agrees by lookup in the finite-range map. The target list is reconstructed in the same way. Thus surviving-edge incidence agrees by cancellation of host inclusion; the inserted corolla's full boundary list maps to exactly the source or target list of e. Labels agree on both edge cases. This proof also covers an edge whose source and target share vertices. □

Lean: `Net.PlugUnit.rightVertices`

Lean: `Net.PlugUnit.rightEdges`

Lean: `Net.PlugUnit.right_host`

Lean: `Net.PlugUnit.sourcePort_list`

Lean: `Net.PlugUnit.targetPort_list`

Lean: `Net.plug_corolla_iso`

25.5 Restoration and unit equations at deep sites

Definition 25.15 (Original selected fragment)**.** The original fragment at a site s is the corolla carrying the label of its selected edge. For an inside site descend to its inner site. If the selected edge is itself a box, its original fragment includes that complete box label and its original body. The fragment has exactly the local depth and interface required by s.

Lean: `Linear.Site.original`

Lemma 25.16 (Deep restoration)**.** *Substituting the original fragment at any site restores the entire host up to recursive isomorphism.*

Proof. First note that reinstalling a box's original body gives a literal-label isomorphism with identity carriers: at the selected edge use its original box-label equality, and elsewhere no label changes. Incidence and internal labels are untouched.

Now induct on the site. For a shallow site apply the right unit. For an inside site, the induction hypothesis restores its inner host up to recursive isomorphism. Box congruence transports that isomorphism to the surrounding updated box. Compose it with the just-proved isomorphism for reinstalling the original body. □

Lean: `Linear.replaceBox_original_iso`

Lean: `Linear.plugAt_original`

Consequently, plugging the original fragment class at s returns the original host class, by quotient soundness. A selected one-edge corolla acts as identity on every replacement class of its interface: quotient induction reduces this to the left-unit isomorphism. Together with the composition equations above, these are unit and associativity laws for the induced substitution operations on recursive isomorphism classes. They do not construct a quotient of addresses or prove an interchange law for disjoint sites.

Lean: `Linear.plugAtModuloIso_original`

Lean: `Linear.plugAtModuloIso_corolla`

25.6 Regression proofs: self-loops, mixed depth, and repaired collisions

For the concrete one-edge self-loop, instantiate shallow associativity with two unary corollas. Instantiate the right unit with the loop's own corolla as well. Both retain the loop's shared source/target vertex, demonstrating that neither theorem relies on a non-self-loop hypothesis.

Lean: `Graph.Examples.loop_associativity`

Lean: `Graph.Examples.loop_unit`

Define a unary variable fragment V as a corolla of the variable label. It has interface $* \to *$ and no holes: any alleged hole observation would equate `none` with `some`. Let t select the sole edge of the earlier replacement containing hole one. Its retained inventory is empty, because there is no other edge in its one-element carrier.

Lean: `Linear.Examples.variableFragment`

Lean: `Linear.Examples.variableFragment_holeFree`

Lean: `Linear.Examples.repeatedSite`

Lean: `Linear.Examples.repeatedSite_retained_absent`

Take the earlier boxed two-hole host and let s select hole zero inside it. Inserting the one-hole replacement carrying label one gives the already proved collision with the surviving hole one. This intermediate graph is invalid. Now fill the inserted occurrence at $s \star t$ with V. The composite criterion proves uniqueness: V is hole-free, hence unique and fresh; and the empty retained inventory of t is disjoint from the retained host inventory.

Lean: `Linear.Examples.intermediate_collision_repaired`

The final graph still contains the host's hole one, witnessed by the surviving inner edge. Hole zero is absent: it cannot be retained by s because it was the selected unique hole; it cannot be retained by t because that inventory is empty; and it cannot occur in V because V is hole-free. The exact composite inventory theorem exhausts these three possibilities.

Lean: `Linear.Examples.repaired_inventory`

For a mixed-depth example, form a one-edge shell whose label is the box label of the boxed two-hole host, and select that shell's sole edge. Insert the boxed host, then follow its deep first-hole site. Deep associativity compares this with filling the boxed host first and then inserting it into the shell.

Lean: `Linear.Examples.boxShell`

Lean: `Linear.Examples.shellSite`

Lean: `Linear.Examples.shallow_then_deep_associativity`

A three-stage instance adds the sole site of the repeated-hole replacement and then the variable fragment; the two site-composition bracketings agree on the resulting quotient class by three-stage coherence. Finally, restoring the original fragment at the boxed first-hole site recovers the original boxed host by deep restoration.

Lean: `Linear.Examples.three_stage_composition`

Lean: `Linear.Examples.deep_restoration`

26 Concrete focuses and the boundary argument of Lemma 39(1)

This section returns to the proof order of Appendix F. We formalize the boundary argument in Proof F.4: replacing focus-free fragments preserves the context's focus mode and whether its source is an input. The result holds for arbitrarily deep sites and finite sequences of fillings. It does not use output-closure, rootedness, determinism, or transitions. It supplies the initial/final agreement needed in Lemma 39(1), for the present substitution representation. Connecting it to the paper's full specimen definition is still an obligation; no concrete `SpecimenSystem` is constructed here.

26.1 Global focus freedom and a unique shallow focus

Definition 26.1 (Focus labels and focus freedom)**.** For a mode m, the symbol $\mathsf{focusLabel}(d, m)$ is the primitive focus symbol in the alphabet of depth d. At depth zero it uses the universe-lift wrapper; at a successor depth it uses the primitive summand. A label is *focus-free* if it is a primitive other than every focus symbol, or a box whose body has only focus-free labels. A graph is focus-free when all its edge labels satisfy this recursively defined condition. Thus the condition excludes focuses inside boxes at every depth, not merely at the outer layer.

Lean: `Linear.focusLabel`

Lean: `Linear.LabelFocusFree`

Lean: `Linear.FocusFree`

Definition 26.2 (Focussed hypernet)**.** A focussed hypernet consists of a hypernet G, a distinguished shallow edge f, its mode m, and the following certificates. The label of f is $\mathsf{focusLabel}(d, m)$; every other edge has a focus-free label; the source and target lists of f are respectively $[s]$ and $[t]$; $s \neq t$; and it is not the case that s is an input and t an output. The label of f is primitive, so has no box children. Consequently these certificates enforce exactly one focus globally, with the shallowness, self-acyclicity, and non-exposure required in §6.2. The explicit singleton incidence certificates record the endpoints without choosing positional casts.

Lean: `Linear.Focussed`

Lemma 26.3 (Focus-free addressed substitution)**.** *If G and H are focus-free, so is $\mathsf{plugAt}(s, H)$ for every site s in G with the interface of H.*

Proof. Induct on the site. At a shallow site, every surviving edge retains its old label, and every inserted edge has a label of H, so both kinds are focus-free. Inside a box, focus freedom of the host implies focus freedom of that box's body. Apply the induction hypothesis to the body. Replacing its label preserves focus freedom of that edge; every other outer label remains unchanged. □

Lean: `Linear.focusFree_plugAt`

Definition 26.4 (Boundary initiality and finality)**.** For a focussed hypernet, define Initial to mean that the mode is search and the focus source is an input. Define Final by replacing search with backtrack. These predicates are defined on any interface for use in the preservation lemmas. On states of type $* \to \epsilon$, there is exactly one input, giving the boundary characterizations used in §6.4 and Proof F.4. Finality is a syntactic predicate here; absence of outgoing transitions from final states has not yet been proved, because transitions on this explicit focus-edge state representation are not yet given by all UAM rules. Sections 27 and 29 prove search-only boundary laws in coordinates and on explicit states, respectively.

Lean: `Linear.Focussed.Initial`

Lean: `Linear.Focussed.Final`

26.2 Preservation through one filling

Lemma 26.5 (Shallow filling preserves the focussed structure)**.** *Let G be focussed at f and let $e \neq f$. Replacing e by a focus-free H produces a focussed hypernet with the same mode.*

Proof. The new focus is the retained edge $\mathsf{inl}(f)$ with unchanged label. Every other retained edge inherits focus freedom from G; every inserted edge inherits it from H. Its endpoints are the host images of s, t, and its incidence lists are their singleton lists by functoriality of list mapping. The host map is injective, so these endpoints remain distinct. It maps internal ports to internal ports and fixes input and output constructors. Therefore exposure cannot be introduced. □

Lean: `Linear.Focussed.plug`

Lemma 26.6 (Shallow boundary preservation and agreement)**.** *A shallow filling as above is initial exactly when its host is initial, and final exactly when its host is final. Any two focus-free fillings of the same selected edge therefore agree on initiality and finality.*

Proof. The focus mode is unchanged. The host image of its source is an input if and only if the original source is an input, by case analysis on the three port constructors. Substitute these facts into the two boundary predicates. For two fillings, compose each equivalence with the inverse equivalence for the other filling. This is exactly the argument of Proof F.4. □

Lean: `Linear.Focussed.plug_initial_iff`

Lean: `Linear.Focussed.plug_final_iff`

Lean: `Linear.Focussed.specimen_boundary_agreement`

Lemma 26.7 (Replacing a box body preserves an outer focus)**.** *Replacing a box body by a focus-free body of the same interface preserves a surrounding focussed hypernet.*

Proof. The box edge cannot be the focus edge: their labels belong to distinct summands of the alphabet. Keep the focus, its mode, and its two endpoints. All incidence lists are unchanged by box-body replacement, so the self-acyclicity and non-exposure certificates remain valid. The changed label is focus-free by the hypothesis on the new body, and all other non-focus labels are unchanged. □

Lean: `Linear.Focussed.replaceBox`

Definition 26.8 (A site avoiding the focus)**.** A shallow site avoids the focus when its selected edge differs from the focus edge. A site strictly inside a box automatically avoids the focus, since the focus is shallow and primitive. This condition does not assert that the selected edge is a hole: the preservation results intentionally apply to a larger class of replacements than paper specimens require.

Lean: `Linear.Focussed.Avoids`

Lemma 26.9 (Deep focussed filling and boundary agreement)**.** *A focus-free filling of any site avoiding the focus preserves the focussed structure and both boundary predicates. Consequently two such fillings of one site agree on initiality and finality.*

Proof. For a shallow site use the shallow filling construction. For a site inside a box, the box is distinct from the focus, hence its old body is focus-free. Focus-free addressed substitution proves the modified body focus-free. Apply the box-body preservation construction. In this second case the outer mode and endpoints remain literally the same, so both boundary equivalences are reflexive. Finally compose the equivalences for two fillings through their common host. □

Lean: `Linear.Focussed.plugAt`

Lean: `Linear.Focussed.plugAt_boundary_iff`

Lean: `Linear.Focussed.deep_specimen_boundary_agreement`

26.3 Finite fillings and the state space

Definition 26.10 (Finite filling sequence)**.** A filling sequence starts with a focussed graph and consists of finitely many addressed substitutions of focus-free graphs at sites avoiding the current focus. The constructors are the empty sequence and extension by one filling. Sites may occur at any box depth. Each intermediate focus is the certified focus transported by the preceding construction. In particular, successive fillings of a context's holes are included whenever their sites are supplied. This representation does not yet define a simultaneous tuple-substitution API or identify all such sequences modulo permutation.

Lean: `Linear.Focussed.Fillings`

Theorem 26.11 (Boundary agreement for finite fillings)**.** *Every finite filling sequence preserves initiality and finality. If two sequences start from the same focussed context, their endpoints agree on both predicates.*

Proof. Induct on the sequence. The empty case is reflexivity. In the extension case, compose the one-step boundary equivalence with the induction hypothesis. For two sequences, compose the resulting equivalences through their common initial context. No template relation between replacements is necessary. Thus once concrete specimens supply these filling witnesses, the initial/final conclusion of Lemma 39(1) follows without an extra assumed locality law. □

Lean: `Linear.Focussed.fillings_boundary_iff`

Lean: `Linear.Focussed.fillings_boundary_agreement`

Definition 26.12 (Concrete state)**.** A state packages a hypernet of type $* \to \epsilon$, a focussed structure on that hypernet, and a proof that it contains no holes at any depth. This is the state space of §6.4 in our graph representation. It is not yet equipped with a `Machine`: Section 29 now supplies an erasure-based search relation on this state space, but graph-rewrite adequacy, substitution, and compute transitions remain open.

Lean: `Linear.State`

26.4 Nonvacuous checks

The regression graph has an input, an internal vertex, and an output, with a focus edge from the input to the internal vertex followed by a unary hole. It is obtained by relabelling the existing two-edge chain, preserving all signatures. Its focus is certified directly: the only other edge is the hole, the endpoints differ, and its target is internal, so the focus is not exposed.

Lean: `Linear.Examples.focusChain`

Lean: `Linear.Examples.focusChain_focussed`

The unary variable corolla is focus-free because a variable label cannot equal a focus label. Filling the second edge of the search chain by this corolla remains initial by shallow boundary preservation. The backtracking chain is final, whereas the search chain is not, since the two mode constructors differ.

Lean: `Linear.Examples.variableFragment_focusFree`

Lean: `Linear.Examples.search_filling_initial`

Lean: `Linear.Examples.boundary_modes_distinct`

Finally, boxing the search chain gives a graph whose outer edge is a box but whose body contains a focus. If it were focus-free, applying that predicate to the outer box edge and then the first body edge would say that the search focus label differs from itself, a contradiction. This checks that the definition really inspects nested boxes.

Lean: `Linear.Examples.boxed_focus_not_free`

27 The five search rules on underlying graph coordinates

The next operational dependency is Fig. 9(a–e). We now define these five rules on actual ordered incidences of a fixed hypernet and prove determinism and preservation under shallow hole filling. The token is represented by a mode and a vertex of the underlying graph. This follows the paper's observation that search leaves the underlying graph unchanged. The relationship to the explicit focus-edge representation is still an adequacy obligation: explicit insertion and incidence recovery are now proved in Section 28, and Section 29 supplies general erasure and search on explicit states. Equivalence with the paper's contextual graph rewrites remains open. The relation is defined even on graphs with holes; its intended state interpretation restricts the underlying graph to be focus-free, hole-free, and of type $* \to \epsilon$, with a token on a term vertex. The stronger structural lemmas below do not need these restrictions.

27.1 Operation kinds and certified incidence views

Definition 27.1 (Operation kind)**.** The two operation kinds are passive and active. A function from operations to these kinds specifies the partition from §5.1. Finishing a passive operation produces backtrack mode; finishing an active operation produces rewrite mode. Neither finishing mode is search, by distinction of constructors.

Lean: `Linear.OperationKind`

Lean: `Linear.OperationKind.finish`

Lean: `Linear.OperationKind.finish_ne_search`

The embedding of primitive labels into each depth alphabet is the universe lift at zero and the left summand at successor depth. It is injective: project the lift at depth zero, or use injectivity of the left injection at successor depth.

Lean: `Linear.primitiveLabel`

Lean: `Linear.primitiveLabel_injective`

Definition 27.2 (Search operation view)**.** A search operation view records an actual edge e with operation label o, its source s, and lists A, T of arguments and thunks. Its certificates assert

$$\mathsf{source}(e) = [s], \qquad \mathsf{target}(e) = A +\!\!+ T, \qquad |A| = \mathsf{arity}(o).$$

Thus A is exactly the ordered term-argument prefix. The ambient graph's signature typing already specifies the types of the remaining thunk targets. This view exposes certified data; it adds no machine-law assumption.

Lean: `Linear.SearchOperation`

Lemma 27.3 (Existence and uniqueness of operation data)**.** *Every shallow operation edge has such a view. Two views of the same edge have the same operation, source, and argument list. Every argument is a target of the edge, and the argument list has no repeated vertices.*

Proof. Signature typing gives a singleton source list and a target list at least as long as the arity. Choose its unique source; take the first arity-many targets for A and drop that prefix for T. The take/drop identity gives the target certificate, and the length bound gives the arity certificate. For uniqueness, the edge label determines the operation by injectivity of

primitive labels. Equality of singleton source lists determines the source. The two argument lists have the same length; taking that many entries from the equal target lists identifies them. Argument membership follows by inclusion of the left append summand. No repetitions follows by restriction of the whole target list's no-repetition certificate. □

Lean: `Linear.SearchOperation.exists_view`

Lean: `Linear.SearchOperation.data_unique`

Lean: `Linear.SearchOperation.argument_mem`

Lean: `Linear.SearchOperation.arguments_nodup`

27.2 Local outcomes and the search relation

Definition 27.4 (Entry and return outcomes)**.** Entering an operation with argument list A yields search at its first argument if A is nonempty. If A is empty, it yields the operation's finishing mode at its source. Returning from argument index i yields search at argument $i + 1$ if that entry exists; otherwise it yields the finishing mode at the source. Indices are zero-based. Return transitions additionally require that argument i exists, so the last case cannot be reached by returning from an out-of-range index.

Lean: `Linear.SearchOperation.enter`

Lean: `Linear.SearchOperation.resume`

These outcomes are independent of the chosen view of an edge. Substitute the operation, source, and argument equalities from data uniqueness into their definitions; the resulting expressions are identical.

Lean: `Linear.SearchOperation.enter_unique`

Lean: `Linear.SearchOperation.resume_unique`

Definition 27.5 (Search position and step)**.** A search position is a pair (m, v) of mode and outer graph vertex. The search relation has three constructors covering the five rules:

1. At a variable edge's singleton source v, search becomes rewrite at v (Fig. 9a).
2. At an operation's source, search yields its entry outcome (Fig. 9b for nonempty arguments and Fig. 9e for empty arguments).
3. Backtracking at argument i yields its return outcome (Fig. 9c when another argument exists and Fig. 9d when it does not).

Every edge mentioned is shallow. Box contents and thunk targets are not searched by these rules. No rule changes the underlying graph, and this relation contains neither substitution nor extrinsic computation steps.

Lean: `Linear.SearchPosition`

Lean: `Linear.SearchStep`

The four named operation rules specialize the two grouped constructors by rewriting the

argument list or the lookup of the next argument. Together with the variable constructor, they expose precisely the five cases of Fig. 9(a–e).

Lean: `Linear.SearchStep.first_argument`

Lean: `Linear.SearchStep.nullary`

Lean: `Linear.SearchStep.next_argument`

Lean: `Linear.SearchStep.last_argument`

27.3 Determinism and initial/final boundaries

Theorem 27.6 (Search determinism)**.** *Two search steps with the same input coordinates have the same output coordinates.*

Proof. Case-split on their rule constructors. Search and backtrack inputs cannot agree. For two variable steps the result is rewrite at the same vertex. For a variable step and an operation-entry step, uniqueness of the edge having the given source forces their edges equal, contradicting their distinct primitive labels. For two operation-entry steps, source incidence uniqueness again identifies the edges; entry-outcome uniqueness identifies the results. For two return steps, both starting vertices occur in their operation target lists. Target incidence uniqueness identifies the edges and hence their argument lists. Since these lists have no repetitions, equal arguments force equal indices. Return-outcome uniqueness then identifies the results. The more general helper permits two separately named input coordinates with an explicit equality; the deterministic theorem specializes that equality to reflexivity. No determinism hypothesis is assumed. □

Lean: `Linear.SearchStep.result_unique`

Lean: `Linear.SearchStep.deterministic`

Lemma 27.7 (Search boundary laws)**.** *A rewrite-mode coordinate has no search step. A backtracking coordinate at an input has no search step. No search step yields search mode at an input.*

Proof. No constructor starts in rewrite mode. A step from backtrack must return from an argument, which is a target of an edge. The graph's incidence conditions forbid an input vertex from being a target. For the last assertion, a variable step ends in rewrite, and finishing an operation never produces search. Therefore a search-mode result must be an argument selected on entry or on return. It is again an edge target and cannot be an input. Apply this stronger non-input assertion to rule out any proposed step into an initial coordinate. □

Lean: `Linear.SearchStep.no_rewrite`

Lean: `Linear.SearchStep.final_no_step`

Lean: `Linear.SearchStep.search_target_not_input`

Lean: `Linear.SearchStep.no_initial_target`

The finality assertion here concerns search steps only. It is not yet the paper's assertion that final states have no transition of any kind; that also requires the substitution and compute rules and the focus-edge bridge.

27.4 Preservation under context filling

Lemma 27.8 (Transport of retained operation data)**.** *Replacing an edge other than an operation edge retains a view of that operation. Entry and return outcomes commute with the host port map.*

Proof. Keep the operation label and use the surviving-edge injection. Map the source, argument list, and thunk list through the host map. Mapping a singleton and an append gives the two incidence certificates, and mapping preserves argument length. Entry commutation follows by the empty/nonempty cases of the argument list. Return commutation follows because lookup in a mapped list is the mapped optional lookup; split according to whether the next argument exists. Modes are unchanged by the port map. □

Lean: `Linear.SearchOperation.plug`

Lean: `Linear.SearchOperation.plug_enter`

Lean: `Linear.SearchOperation.plug_resume`

Theorem 27.9 (Existing context search steps survive filling)**.** *Suppose G has a shallow hole edge h and a search step $p \to q$. For every interface-compatible filling H, the graph $G[h := H]$ has a search step from the host image of p to the host image of q.*

Proof. In every rule, the interacting edge is a variable or operation and therefore cannot equal the hole edge. In the variable case, retain that edge, its label, and its mapped singleton source to apply the same rule. In the entry case, use the retained operation view and entry commutation. In the return case, lookup commutes with mapping, so the old argument witness supplies the mapped argument witness; apply return commutation to identify the result. □

Lean: `Linear.SearchStep.plug`

This theorem starts with a step of the context itself. Lemma 39(3) instead starts with a step of a filled specimen and must establish that it comes from the context. Extracting that step requires the entering/exiting analysis, rootedness, output-closure, and the focus-edge bridge. The preservation theorem proves one needed direction of locality, not the whole lemma.

27.5 Concrete checks of rule selection

The fixture has one operation with two term arguments and one thunk. Its graph is a corolla with targets $0, 1, 2$ in that order; the operation view certifies that the argument list is $[0, 1]$ and the thunk list is $[2]$. The incidence certificates are checked by computation.

Lean: `Linear.SearchExamples.ops`

Lean: `Linear.SearchExamples.graph`

Lean: `Linear.SearchExamples.operation`

Entry selects target 0 and return from target 0 selects target 1. Return from target 1 finishes at the source in the declared mode, for either operation kind. These are direct applications of the corresponding rule theorems.

Lean: `Linear.SearchExamples.enter_first`

Lean: `Linear.SearchExamples.return_next`

Lean: `Linear.SearchExamples.finish_skips_thunk`

A hypothetical return step to search at thunk target 2 would, by determinism, have the same result as the finishing step. It would identify the finishing mode with search, which is impossible. This checks that the full target list has not accidentally been used as the argument list.

Lean: `Linear.SearchExamples.cannot_search_thunk`

The existing nullary value corolla admits the empty argument view and finishes by backtracking when declared passive. The unary variable corolla uses the variable constructor and triggers rewrite without moving the coordinate.

Lean: `Linear.SearchExamples.value_finishes`

Lean: `Linear.SearchExamples.variable_triggers`

28 Explicit focus insertion and realization of typed search positions

This section advances the operational obligation in the remaining-work list. A term-vertex coordinate now determines an actual focussed hypernet, and on a hole-free graph of type $* \to \epsilon$ it determines an actual state. The construction proves all incidence and typing laws, rather than assuming a realization interface. Contracting the inserted focus recovers the original ordered incidence lists. Search preserves the term-vertex condition, so every typed coordinate search step has two such state realizations.

This section proves the insertion direction of the bridge. Section 29 extends it to arbitrary closed focussed states by general erasure and an erasure-then-insertion isomorphism. Neither section yet identifies the coordinate relation with the paper's contextual interaction rules; the endpoint theorem below must not be read as that adequacy statement. Substitution, compute transitions, and the concrete specimen interpretation also remain open.

28.1 Splitting a vertex

Definition 28.1 (Focus insertion carriers and maps)**.** Let G be a hypernet and let v be a term vertex that is not an output. Extend the internal-vertex carrier from V to $V+1$ and the edge carrier from E to $E+1$. Write z for the fresh internal vertex, ι for the inclusion of old ports, and f for the fresh edge. Boundaries are unchanged. Define

$$S_v(w) = \begin{cases} z & w = v, \\ \iota(w) & w \neq v, \end{cases} \qquad C_v(\iota(w)) = w, \quad C_v(z) = v.$$

Here S_v redirects old source occurrences of v, while C_v contracts the new focus. Old internal vertices retain their wire labels and z has label term. The use of a singleton carrier gives explicit finite enumerations.

Lean: `Linear.FocusInsertion.Vertex`

Lean: `Linear.FocusInsertion.host`

Lean: `Linear.FocusInsertion.fresh`

Lean: `Linear.FocusInsertion.collapse`

Lean: `Linear.FocusInsertion.split`

Lean: `Linear.FocusInsertion.internalLabel`

Lemma 28.2 (Map properties)**.** *The inclusion ι is injective and never reaches z. The contraction satisfies $C_v\iota = \mathrm{id}$ and $C_vS_v = \mathrm{id}$; consequently S_v is injective. No image of S_v equals $\iota(v)$. Every new port is either an old port's image or z. Both inclusion and redirection preserve wire labels.*

Proof. Case analysis on internal, input, and output constructors proves the inclusion claims; on internals use injectivity and disjointness of sum tags. For C_vS_v, split on $w = v$: the fresh port contracts to v in the equality case, and otherwise contraction undoes inclusion. Applying C_v to equal redirection images proves injectivity. An image cannot equal $\iota(v)$: in the equality case it is fresh; otherwise inclusion injectivity would force $w = v$. The port classification uses the two internal sum cases and the boundary constructors; every inhabitant of the fresh singleton

carrier equals its sole member. Inclusion preserves labels by cases. Redirection preserves labels because only v changes image, and both v and z have term label. □

Lean: `Linear.FocusInsertion.collapse_host`

Lean: `Linear.FocusInsertion.host_ne_fresh`

Lean: `Linear.FocusInsertion.host_injective`

Lean: `Linear.FocusInsertion.collapse_split`

Lean: `Linear.FocusInsertion.split_injective`

Lean: `Linear.FocusInsertion.split_ne_host`

Lean: `Linear.FocusInsertion.vertex_cases`

Lean: `Linear.FocusInsertion.host_label`

Lean: `Linear.FocusInsertion.split_label`

Definition 28.3 (Inserted incidence lists)**.** For every old edge e, use $S_v(\mathsf{source}_G(e))$ for its source list and $\iota(\mathsf{target}_G(e))$ for its target list, mapping entry by entry. The fresh edge has source $[\iota(v)]$, target $[z]$, and the chosen focus mode as label. Old edge labels, including whole boxed bodies, are unchanged.

Lean: `Linear.FocusInsertion.source`

Lean: `Linear.FocusInsertion.target`

Lemma 28.4 (Incidence uniqueness and coverage)**.** *The inserted incidence lists satisfy source and target uniqueness. A port occurs in a source list exactly when it is not an output, and in a target list exactly when it is not an input.*

Proof. For two old source edges, pull equality of mapped vertices back along the injective S_v and apply source uniqueness in G. An old source cannot meet the fresh edge's source because S_v avoids $\iota(v)$. There is only one fresh edge. The target proof is identical with inclusion in place of redirection and disjointness from z in place of avoidance of $\iota(v)$.

For source coverage, all mapped old sources are non-outputs: a redirected one is internal, and any other one retains its old boundary kind. The fresh edge's source is a non-output by the hypothesis on v. Conversely, an old image $\iota(w)$ with $w = v$ is sourced by the fresh edge. If $w \neq v$, use coverage in G and its retained source edge. The fresh port z is sourced by the old edge that sourced v, which exists because v was not an output. For target coverage, retained targets remain non-inputs and the fresh target is internal. Conversely an old non-input image inherits a target edge from G, and z is targeted by the fresh focus edge. □

Lean: `Linear.FocusInsertion.source_unique`

Lean: `Linear.FocusInsertion.target_unique`

Lean: `Linear.FocusInsertion.source_coverage`

Lean: `Linear.FocusInsertion.target_coverage`

Theorem 28.5 (Well-formed explicit focus insertion)**.** *The construction gives a finite typed hypernet with the same external interface as G.*

Proof. Enumerate each extended carrier by the sum of the old enumeration and the singleton enumeration. Old incidence lists retain their no-repetition property because both maps are injective; the new lists are singletons. Use the proved uniqueness and coverage lemmas. For typing, mapping a port list and then reading labels is the same as reading the old labels, by the label-preservation laws. The new source and target both have term label, exactly the focus signature. These checks populate every field of the concrete net record. □

Lean: `Linear.FocusInsertion.insert`

28.2 Focus certification, contraction, and holes

Theorem 28.6 (Unique focus and contraction)**.** *If G is globally focus-free, the inserted graph is focussed in the sense of §6.2. Contracting the new focus restores each original source and target list.*

Proof. Choose the fresh edge as focus. Every old label remains globally focus-free; the singleton fresh-edge carrier has no second member. The focus endpoints are $\iota(v)$ and z, which are distinct. Its target is internal, so the focus is not exposed. Its incidence lists and mode are those specified by construction. To recover source lists, compose their map by S_v with the map by C_v and use $C_v S_v = \mathrm{id}$. For target lists use $C_v \iota = \mathrm{id}$. These statements concern the retained edges; the focus itself is deleted when interpreting contraction. They establish recovery for the constructed family, not a general erasure operation on arbitrary focussed graphs. □

Lean: `Linear.FocusInsertion.focussed`

Lean: `Linear.FocusInsertion.collapse_sources`

Lean: `Linear.FocusInsertion.collapse_targets`

Lemma 28.7 (Hole freedom and boundary observations)**.** *If G is hole-free at every depth, so is its focus insertion. The explicit focus is initial exactly when its mode is search and v is an input; it is final exactly when its mode is backtrack and v is an input.*

Proof. Represent a hypothetical hole occurrence by an outer edge and a label-local position. If the edge is retained, its unchanged label gives a hole occurrence in G, a contradiction. If it is the fresh edge, its focus label is not a hole and has no children. This covers zero and successor depth alphabets. For boundary observations, the mode is unchanged and the source is $\iota(v)$. Inclusion preserves input status, by cases on the port constructor. □

Lean: `Linear.FocusInsertion.holeFree`

Lean: `Linear.FocusInsertion.boundary_iff`

28.3 Search preserves term typing

Lemma 28.8 (Typed operation data)**.** *An operation view's source has term label. Mapping its argument list through vertex labels gives exactly the list of arity-many term labels. Consequently every argument vertex has term label.*

Proof. Rewrite the operation's source-typing certificate by its singleton source and operation label; the singleton signature forces the source label to be term. For arguments, rewrite the target-typing certificate using the argument/thunk append decomposition. Take an argument-length prefix on both sides. The arity certificate ensures that this is exactly the entire argument-label list on one side and the term prefix of the signature on the other. Finally map membership of an argument into this repeated-term list: every member equals term. □

Lean: `Linear.SearchOperation.source_term`

Lean: `Linear.SearchOperation.arguments_typed`

Lean: `Linear.SearchOperation.argument_term`

Theorem 28.9 (Term typing preservation)**.** *If a search step starts at a term vertex, its result is at a term vertex.*

Proof. A variable trigger does not move, so inherits the premise. Operation entry selects either the first argument or the source when there are no arguments. Operation return selects either the next argument or the source on completion. Apply the operation data lemmas in each case. Neither operation case needs the premise on the old vertex; its result's type follows from graph typing alone. □

Lean: `Linear.SearchStep.preserves_term`

28.4 Actual states from typed coordinates

Definition 28.10 (Typed position and realization)**.** For a graph $G : * \to \epsilon$, a typed position is a search coordinate whose vertex is certified to have term label. Such a vertex cannot be an output, since the output index type is empty. If G is globally focus-free and hole-free, realize a typed position by inserting its mode at its vertex and packaging the resulting graph, unique-focus certificate, and hole-freedom certificate into the concrete state record.

Lean: `Linear.TypedPosition`

Lean: `Linear.TypedPosition.not_output`

Lean: `Linear.TypedPosition.realize`

Definition 28.11 (Initial attachment)**.** The initial typed position is search at the sole input, whose wire label is term by the interface. Realizing it is an initial state: the constructed focus has search mode and its source is that input. More generally, the realized state's initial and final predicates coincide with their coordinate boundary predicates, by the insertion boundary lemma.

Lean: `Linear.TypedPosition.initial`

Lean: `Linear.TypedPosition.initial_realize`

Lean: `Linear.TypedPosition.realize_boundary`

This discharges the graph-construction part of initial attachment. Proposition 38 still needs the concrete observation interface and specimen witnesses required by `initial_lift`.

Definition 28.12 (Advancing a typed position)**.** Given a search step from a typed position, equip its result coordinate with the term-typing proof supplied by preservation. This gives the next typed position on the same underlying graph.

Lean: `Linear.TypedPosition.advance`

Theorem 28.13 (Realization of search endpoints)**.** *For every search step from a typed position on a focus-free, hole-free graph $G : * \to \epsilon$, realize the initial position and its typed successor as states s, t. Their explicit focus modes are the corresponding coordinate modes. At each endpoint, collapsing the inserted focus recovers every original edge's source and target list in G.*

Proof. Use the realization constructor for the input and for the typed successor. Their focus modes are correct by the definition of insertion. Apply the two incidence-recovery theorems at each coordinate. The common underlying graph is unchanged; only the position and mode of the inserted focus may differ. The statement supplies actual state endpoints and recovery certificates, not a new definition of a contextual graph-rewrite step between arbitrary states. □

Lean: `Linear.TypedPosition.step_realizes`

28.5 Boundary and interior regression checks

The existing value corolla is focus-free because its operation label cannot be a focus label. Realizing its initial typed position gives the explicit state `initialValue`. Advancing by its passive nullary search step and realizing the result gives `finalValue`. Their initial and final certificates follow from their modes and common input source. The initial state's distinct focus endpoints are guaranteed by its focussed certificate.

Lean: `Linear.Examples.value_focusFree`

Lean: `Linear.Examples.initialValue`

Lean: `Linear.Examples.finalValue`

Lean: `Linear.Examples.value_boundary_states`

Lean: `Linear.Examples.initial_value_has_distinct_endpoints`

The two-hole chain is also globally focus-free, because neither hole label can be a focus. Insert search at its internal vertex: the first edge still targets the old vertex, the second edge now starts at the fresh vertex, and the new focus connects the old vertex to the fresh one. Unfolding the incidence maps proves all four equalities. This focus is not initial because its source is internal. Unlike the value example, this test deliberately retains holes and therefore tests focussed hypernets rather than closed states.

Lean: `Linear.Examples.twoHoles_focusFree`

Lean: `Linear.Examples.interior_insertion`

Lean: `Linear.Examples.interior_not_initial`

29 Focus, search, and rooted-state paths

We now erase the focus of an arbitrary focussed graph with no outputs, prove that reinsertion recovers the original graph and focus up to isomorphism, and use the construction to define search on actual states. Unlike the previous section's recovery result, the input need not have been constructed by our insertion function. Search is deterministic up to focus-preserving isomorphism, final states have no search step, and no search step ends in an initial state. A concrete initial value state makes a search step to an actual final state.

The remaining operational adequacy obligation is precise: identify this erasure-based relation with the paper's contextual local-rewrite presentation. The later subsections prove both focus round trips, source and target renaming invariance, and descent to literal-label isomorphism classes, where search is deterministic by equality of successor classes. The full machine with substitution and compute transitions is still to be constructed.

29.1 Contraction maps and the retained carriers

Fix a focussed graph $G : I \to \epsilon$, focus edge e_f, source s, and an internal vertex t equal to the focus target. We first work with this explicit witness, then prove that every such graph supplies it.

Definition 29.1 (General focus erasure maps)**.** Remove t from the internal carrier and e_f from the edge carrier, using subtypes of names unequal to the deleted names. Let j include the remaining ports into the old ports. If $w \neq t$, let $k(w)$ keep w with its non-deletion certificate. Self-acyclicity ensures $s \neq t$, so $p = k(s)$ is defined. Set

$$c(w) = \begin{cases} p & w = t, \\ k(w) & w \neq t, \end{cases} \qquad a(x) = \begin{cases} t & j(x) = s, \\ j(x) & j(x) \neq s. \end{cases}$$

The map c contracts the focus. The map a recovers the appropriate old *source* representative, choosing t at the joined port; j recovers the old *target* representative. Boundary ports and their order are unchanged.

Lean: `Linear.FocusErasure.Vertex`

Lean: `Linear.FocusErasure.lift`

Lean: `Linear.FocusErasure.keep`

Lean: `Linear.FocusErasure.position`

Lean: `Linear.FocusErasure.contract`

Lean: `Linear.FocusErasure.expand`

Lemma 29.2 (Port-map identities)**.** *The map j is injective, avoids t, and satisfies $j(k(w)) = w$. Moreover $j(p) = s$, $c(t) = c(s) = p$, $j(c(w)) = w$ for $w \neq t$, and $c(j(x)) = x$. The expansion $a(x)$ never equals s; it satisfies $a(c(w)) = w$ for $w \neq s$ and $c(a(x)) = x$.*

Proof. The inclusion and keeping claims follow by port-constructor cases; subtype extensionality gives injectivity on internal ports, and the subtype certificate excludes t. The inequality $s \neq t$ is exactly focus self-acyclicity rewritten by the chosen target witness. The identities for p, c follow by unfolding and splitting on whether the argument is t. Apply inclusion injectivity to

derive $c(j(x)) = x$ from $j(c(j(x))) = j(x)$. For expansion, when $j(x) = s$ its result is $t \neq s$; otherwise it is already different from s. To prove $a(c(w)) = w$, split on $w = t$. At t, contraction lands at p and expansion returns t. Otherwise inclusion after contraction returns $w \neq s$, so expansion leaves it unchanged. For $c(a(x)) = x$, split on $j(x) = s$. In that case $x = p$ by inclusion injectivity; otherwise use $c(j(x)) = x$. □

Lean: `Linear.FocusErasure.lift_keep`

Lean: `Linear.FocusErasure.lift_injective`

Lean: `Linear.FocusErasure.lift_ne_target`

Lean: `Linear.FocusErasure.source_ne_target`

Lean: `Linear.FocusErasure.lift_position`

Lean: `Linear.FocusErasure.contract_target`

Lean: `Linear.FocusErasure.lift_contract`

Lean: `Linear.FocusErasure.contract_lift`

Lean: `Linear.FocusErasure.contract_source`

Lean: `Linear.FocusErasure.expand_ne_source`

Lean: `Linear.FocusErasure.expand_contract`

Lean: `Linear.FocusErasure.contract_expand`

Lemma 29.3 (Injectivity on retained incidence lists)**.** *A source of a retained edge cannot equal s, and a target of a retained edge cannot equal t. Contraction is injective on vertices other than s, and also on vertices other than t.*

Proof. If a retained edge sourced s, source uniqueness with the focus edge would identify the two edges, contrary to retention. Target uniqueness proves the other exclusion. For injectivity off s, apply a to equality of contracted ports and use $a(c(w)) = w$. For injectivity off t, apply j and use $j(c(w)) = w$. These are restricted injectivity assertions: c itself identifies the two distinct focus endpoints. □

Lean: `Linear.FocusErasure.source_avoids`

Lean: `Linear.FocusErasure.target_avoids`

Lean: `Linear.FocusErasure.source_injective`

Lean: `Linear.FocusErasure.target_injective`

Lemma 29.4 (Label preservation)**.** *Both focus endpoints have term label. Keeping an undeleted port preserves its label, and contraction preserves every port's label.*

Proof. Rewrite the focus source and target typing certificates using their singleton incidences and focus label; its signature is term-to-term. Keeping preserves labels by cases on internal and boundary ports. For contraction, all ports other than t use keeping. At t, its image is the kept source and both endpoints have the same term label. □

Lean: `Linear.FocusErasure.source_term`

Lean: `Linear.FocusErasure.target_term`

Lean: `Linear.FocusErasure.keep_label`

Lean: `Linear.FocusErasure.contract_label`

29.2 A valid focus-free underlying graph

Theorem 29.5 (General erasure construction)**.** *Delete e_f and t, retain every other edge label, and map all retained source and target lists through c. The result is a finite typed hypernet of the same interface $I \to \epsilon$.*

Proof. Finite subtype enumerations supply the carriers. Restricted contraction injectivity on retained sources and targets preserves no-repetition. Given a shared contracted source of two retained edges, restricted injectivity recovers a shared old source, so old source uniqueness identifies the edge names. The target proof is the same with the target restriction.

For source coverage, there are no output ports. Given an erased port x, use the old source coverage at $a(x)$. Since $a(x) \neq s$, its source edge cannot be the focus; contraction of that source gives $c(a(x)) = x$. For target coverage, a retained target w differs from t, so $j(c(w)) = w$ and its input status is unchanged. Conversely, if x is not an input, old target coverage at $j(x)$ gives an edge. It is not the focus because $j(x) \neq t$. Its contracted target is $c(j(x)) = x$. Finally, mapping contracted lists through labels equals reading the old labels, by label preservation, so both signature-typing certificates follow from G. □

Lean: `Linear.FocusErasure.erase`

Lemma 29.6 (Erased invariants)**.** *The erased graph is globally focus-free, its distinguished port p has term label, and erasure preserves global hole-freedom.*

Proof. Every retained edge was different from the unique focus, so its entire label, including any box body, is focus-free. The port p keeps the term-labelled source, hence has term label. A hypothetical hole in the erased graph lies at a retained edge with unchanged label and label-local position; the same occurrence would be a hole in the original graph. Thus a hole-free original has a hole-free erasure. □

Lean: `Linear.FocusErasure.erased_focusFree`

Lean: `Linear.FocusErasure.position_term`

Lean: `Linear.FocusErasure.erased_holeFree`

29.3 Reinsertion recovers an arbitrary focussed graph

Definition 29.7 (Restoring a deleted name)**.** For any type A and distinguished $b \in A$, the restoration bijection $(A \setminus \{b\}) + 1 \to A$ sends retained names to themselves and the fresh singleton name to b. Its inverse tests whether a name equals b. Applying this to t and e_f gives the vertex and edge bijections needed for reinsertion. Reinsert the original focus mode at p in the erased graph. The port p is a non-output because the output interface is empty, and its term-typing certificate was proved above.

Lean: `Linear.FocusErasure.restore`

Lean: `Linear.FocusErasure.position_not_output`

Lean: `Linear.FocusErasure.reinsert`

The bijection's inverse laws follow by retained/fresh cases; the fresh carrier has exactly one member. At ports, restoring host inclusion agrees with j, and restoring the fresh insertion vertex gives t. Restoring the insertion's source-redirection map agrees with a: the test $x = p$ is equivalent to $j(x) = s$, by inclusion injectivity and $j(p) = s$.

Lean: `Linear.FocusErasure.restore_host`

Lean: `Linear.FocusErasure.restore_fresh`

Lean: `Linear.FocusErasure.restore_split`

Theorem 29.8 (Focus-preserving round trip)**.** *Reinserting after erasure is isomorphic to the original graph. The isomorphism restores the focus edge, source, and target, preserves boundary order, and preserves literal edge labels, including complete boxed bodies.*

Proof. Use the restoration bijections. Retained internal labels are unchanged; the fresh insertion vertex has term label, equal to the original label of t. Retained edge labels are unchanged, and the new focus label equals the original focus label by its certificate. For a retained source list, restoration after insertion-redirection after contraction acts as $a \circ c$, which is the identity on these sources because they avoid s. For a retained target list, restoration acts as $j \circ c$, which is the identity because these targets avoid t. Map these pointwise identities over the lists. The new focus source restores to $j(p) = s$, and its target restores to t, giving the original singleton incidence lists. The fresh edge restores to e_f by definition, and the preceding two port identities explicitly identify its endpoints. □

Lean: `Linear.FocusErasure.reinsert_iso`

Lean: `Linear.FocusErasure.reinsert_focus_edge`

Lean: `Linear.FocusErasure.reinsert_focus_endpoints`

Lemma 29.9 (Every closed focussed graph admits erasure)**.** *The focus target of any graph $G : I \to \epsilon$ is internal. Therefore every such focussed graph has a globally focus-free underlying graph, preserving hole-freedom, and reinserting its mode recovers it up to the proved isomorphism.*

Proof. The focus target occurs in a target list, so cannot be an input. There are no outputs. The remaining port constructor is internal. Choose its internal name with its equality certificate, and instantiate the preceding erasure, invariant, and round-trip constructions. This use of choice supplies a witness already proved to exist; it does not assume an erasure law. □

Lean: `Linear.Focussed.target_internal`

Lean: `Linear.Focussed.targetVertex`

Lean: `Linear.Focussed.targetVertex_spec`

Lean: `Linear.Focussed.underlying`

Lean: `Linear.Focussed.underlying_focusFree`

Lean: `Linear.Focussed.underlying_holeFree`

Lean: `Linear.Focussed.underlying_reinsert_iso`

The theorem proves the erasure-then-insertion direction for arbitrary closed focussed graphs. The opposite insertion-then-erasure isomorphism is proved in Section 29.7, and naturality under source isomorphisms in Section 29.11.

29.4 Focus-preserving isomorphisms and state recovery

Definition 29.10 (Focussed isomorphism)**.** A focussed isomorphism is a literal-label graph isomorphism together with equality of the two focus modes and an equation mapping the distinguished focus edge to the distinguished focus edge. Identity, inverse, and composition are obtained from the corresponding graph isomorphisms, using the edge bijection's inverse laws and composition for the focus equations.

Lean: `Linear.Focussed.Iso`

Lean: `Linear.Focussed.Iso.refl`

Lean: `Linear.Focussed.Iso.symm`

Lean: `Linear.Focussed.Iso.trans`

Lemma 29.11 (Endpoints and observations are invariant)**.** *A focussed isomorphism maps both focus endpoints correctly and preserves initiality and finality.*

Proof. Specialize the isomorphism's source-list equation to the focus edge and rewrite both lists as singletons. Singleton injectivity identifies the endpoints. Repeat for target lists. Port maps preserve input status; together with the mode equality, this yields both boundary-predicate equivalences. □

Lean: `Linear.Focussed.Iso.source`

Lean: `Linear.Focussed.Iso.target`

Lean: `Linear.Focussed.Iso.initial_iff`

Lean: `Linear.Focussed.Iso.final_iff`

Definition 29.12 (Extracted state coordinate)**.** For an arbitrary state s, erase its focus and take the typed position whose mode is the old mode and whose vertex is the joined port p. Its typing is the erased-position typing lemma. Re-realizing this coordinate in the globally focus-free, hole-free underlying graph yields a state focus-preservingly isomorphic to s, using the general round trip. Thus the input need not be a canonical insertion or carry an assumed representation certificate.

Lean: `Linear.State.coordinate`

Lean: `Linear.State.recover`

The extracted coordinate is initial or final exactly when s is. Compose the previous realization boundary equivalence with the focussed isomorphism's boundary equivalence.

Lean: `Linear.State.coordinate_boundary`

29.5 Search between actual states

Definition 29.13 (Erasure-based state search)**.** For a fixed operation-kind function, write $s \to_{\mathrm{er}} t$ if there is a typed coordinate q on the underlying graph of s such that the extracted coordinate of s makes one of the five search steps to q, and the realization of q is focus-preservingly isomorphic to t. Thus the target may use arbitrary vertex and edge names. This is a relation on actual state records, rather than merely a construction of two endpoints from a separately supplied graph.

Lean: `Linear.State.Search`

Lemma 29.14 (Existence and target renaming)**.** *Every typed coordinate successor of an extracted state coordinate gives an actual state successor. Replacing a successor by a focus-preservingly isomorphic state preserves the step.*

Proof. For existence, realize the successor coordinate and use the identity focussed isomorphism as witness. For target renaming, retain the same coordinate step and compose its target isomorphism with the supplied isomorphism. □

Lean: `Linear.State.search_realize`

Lean: `Linear.State.search_target_iso`

Theorem 29.15 (Search determinism up to focussed isomorphism)**.** *Two erasure-based search successors of the same state are focus-preservingly isomorphic.*

Proof. Both coordinate steps use the same erased graph and the same extracted input. Coordinate search determinism identifies their output pairs; subtype extensionality identifies the typed positions. Their canonical realizations therefore coincide. Compose the inverse of the first target isomorphism with the second target isomorphism to identify the actual successors. □

Lean: `Linear.State.search_deterministic`

Lemma 29.16 (Explicit-state search boundary laws)**.** *A final state has no erasure-based search step, and no such step ends in an initial state.*

Proof. If the source is final, its extracted coordinate has backtrack mode at an input. Coordinate search cannot start there. If the target is initial, pull initiality back through the target's focussed isomorphism and the realization boundary equivalence. The successor coordinate would be search at an input, which the coordinate search boundary theorem excludes. □

Lean: `Linear.State.final_no_search`

Lean: `Linear.State.no_search_initial`

These are search-only laws. Neither all-transition finality nor the full robustness theorem follows until the remaining transition classes and specimen obligations are supplied.

29.6 Concrete execution and interior recovery

Erase the focus of the previously constructed initial value state. Its value edge survives and has an empty argument/thunk view. Its source contracts from the old focus target to the joined port, which is exactly the extracted state coordinate's vertex. This proves the view's source certificate; the other certificates follow from the nullary value signature.

Lean: `Linear.Examples.erasedValueOperation`

Apply the passive nullary rule, advance the typed coordinate, and realize it. The existence lemma gives an actual state search step. The result is final because its mode is backtrack and its vertex retains the input status of the initial coordinate. This proves that the new state relation has a concrete execution, rather than only universal implications about hypothetical steps.

Lean: `Linear.Examples.value_state_search`

For an interior test, relabel the old focus of the initial value graph as a variable, retaining the value edge and all incidences. Both labels exclude focuses and holes, so this is a focus-free, hole-free closed two-edge chain. Insert search at its internal vertex to obtain a state whose focus source is internal. It is not initial, directly from that source constructor. General state recovery erases and reconstructs it with a focussed isomorphism, including its distinguished focus and mode.

Lean: `Linear.Examples.closedChain`

Lean: `Linear.Examples.closedChain_focusFree`

Lean: `Linear.Examples.closedChain_holeFree`

Lean: `Linear.Examples.interiorState`

Lean: `Linear.Examples.interior_state_not_initial`

Lean: `Linear.Examples.interior_state_recovery`

29.7 Insertion followed by erasure

Fix a focus-free graph $G : I \to []$, a term vertex v, and a mode m. Write $S = \mathrm{insert}(G, v, m)$ for the focussed graph constructed in Section 28. The preceding round trip started with an arbitrary focussed graph. We now establish the opposite direction, including the position at which the focus was inserted.

Definition 29.17 (Deleting a fresh singleton)**.** For any carrier A, deleting the fresh member from $A + \mathbf{1}$ leaves a carrier canonically bijective with A. The forward map sends a retained $\mathsf{inl}(a)$ to a; the inverse sends a to that retained element. An element of the right summand cannot survive, since its singleton index must be zero. Apply general erasure to the fresh focus and fresh target of S to define $E(S)$.

The inverse laws hold by cases on the summand: the retained case is reflexive, and the fresh case contradicts its non-deletion certificate.

Lean: `Linear.FocusInsertion.deleteFresh`

Lean: `Linear.FocusInsertion.erasedInsertion`

Lemma 29.18 (Cancellation on ports)**.** *Let D be the port map induced by deletion of the fresh singleton, j the inclusion of retained ports, and κ the collapse map of insertion. Then $D(x) = \kappa(j(x))$. The joined vertex maps to v, and for every port w of S, $D(c(w)) = \kappa(w)$, where c is general focus contraction.*

Proof. The first identity follows by cases on the retained port. An internal right-summand case is impossible; input ports are fixed. For the joined vertex, inclusion gives the focus source, namely the hosted copy of v; collapse sends it to v. Finally, if w is the fresh target, contraction gives the joined vertex and both sides equal v. Otherwise inclusion after contraction is w, and the first identity applies. □

Lean: `Linear.FocusInsertion.deleteFresh_port`

Lean: `Linear.FocusInsertion.erase_position`

Lean: `Linear.FocusInsertion.erase_contract`

Theorem 29.19 (Insertion–erasure cancellation)**.** *There is an isomorphism $E(S) \cong G$ preserving full labels and ordered incidences. The choice-based operation $S.\mathsf{underlying}$ has such an isomorphism too, and one can choose it to send the joined vertex to v.*

Proof. Use deletion of the singleton on both internal vertices and edges. All surviving labels are inherited. Sources were first split by insertion and then contracted by erasure; the port cancellation identity reduces their composite to collapse after split, which is the identity. Targets were hosted by insertion, so the analogous composite is collapse after hosting, also the identity. These pointwise identities give equality of ordered lists.

The target selected by general erasure is necessarily the fresh internal vertex: both it and the fresh vertex equal the focus target, and the internal port constructor is injective. Substituting this equality identifies the general underlying graph with $E(S)$. For the position-sensitive assertion, carry an arbitrary target together with its equality certificate, identify it with the fresh target, and use the explicit isomorphism and joined-port identity together. This keeps graph and coordinate recovery synchronized. □

Lean: `Linear.FocusInsertion.erase_iso`

Lean: `Linear.FocusInsertion.targetVertex_eq`

Lean: `Linear.FocusInsertion.underlying_iso`

Lean: `Linear.FocusInsertion.underlying_position_exists`

29.8 Search coordinates respect renaming

Let $a : G \cong H$ be a literal-label isomorphism and write a_* for its port map. Internal wire labels are preserved by the isomorphism's label field; boundary labels are preserved because boundary positions are fixed.

Lean: `Linear.iso_vertexLabel`

Definition 29.20 (Transport of operation views and coordinates)**.** Transport an operation view by applying the edge bijection to its edge and a_* to its source and both ordered lists. Keep its operation symbol. Source and target certificates follow from the isomorphism's incidence laws; arity follows because mapping a list preserves length. Transport a typed coordinate (m, v) to $(m, a_* v)$. Wire-label preservation supplies its typing certificate.

Lean: `Linear.SearchOperation.rename`

Lean: `Linear.TypedPosition.rename`

Lemma 29.21 (Transport of the local rules)**.** *Operation entry and argument return commute with transport. Every coordinate search step transports forwards, and a step between transported coordinates exists if and only if the original step exists.*

Proof. Entry depends on whether the argument list is empty or has a first element; mapping preserves these alternatives. Return depends on the optional element at index $i + 1$; lookup in a mapped list is the mapped lookup. These facts prove the two local calculation identities.

Induct on the search rule. The variable rule uses the transported label and singleton source certificate. The operation-entry case uses the entry identity. For argument return, map the certified lookup at index i and use the return identity. This proves forward transport for all five rules. Apply the same result to a^{-1} for reflection: the composite port map is the identity, and neither map changes the mode. The same inverse law proves that transporting a typed coordinate by a and back by a^{-1} returns that coordinate; subtype extensionality discards the irrelevant typing certificates. □

Lean: `Linear.SearchOperation.rename_enter`

Lean: `Linear.SearchOperation.rename_resume`

Lean: `Linear.SearchStep.rename`

Lean: `Linear.SearchStep.rename_iff`

Lean: `Linear.TypedPosition.rename_symm`

29.9 Insertion respects renaming

Extend the vertex and edge bijections of a by the identity on each fresh singleton. The extended port map sends hosted vertices to hosted images and fixes the fresh vertex. It also commutes with splitting: injectivity gives $a_* w = a_* v$ exactly when $w = v$, so both splits select the fresh branch in exactly the same case.

Lean: `Linear.FocusInsertion.rename_host`

Lean: `Linear.FocusInsertion.rename_fresh`

Lean: `Linear.FocusInsertion.rename_split`

Theorem 29.22 (Naturality of insertion)**.** *Insertion at (m, v) in G is focus-preservingly isomorphic to insertion at $(m, a_* v)$ in H. In particular, realizing a typed coordinate commutes with its transport through a.*

Proof. Use the two extended bijections. Retained labels are preserved by a; fresh vertex and edge labels agree by construction. On retained edges, commutation with splitting proves the source equation and commutation with hosting proves the target equation, after applying the corresponding incidence equation of a. The fresh edge has exactly the hosted insertion position as source and the fresh vertex as target, so the two port identities prove its equations. The mode is unchanged, and the fresh focus edge maps to itself. Packaging these fields gives a focussed isomorphism. Applying this construction to typed coordinates gives the realization result; focus-free and hole-free certificates ensure both endpoints are states. □

Lean: `Linear.FocusInsertion.rename_iso`

Lean: `Linear.FocusInsertion.rename_focussed`

Lean: `Linear.TypedPosition.realize_rename`

29.10 Exact characterization of search from realized states

Lemma 29.23 (Recovery of a realized coordinate)**.** *If p is a typed coordinate of a focus-free, hole-free graph G, there is an isomorphism a from the underlying graph of its realization to G such that $a_* \,\mathrm{coordinate}(\mathrm{realize}(p)) = p$.*

Proof. Use the position-sensitive insertion–erasure isomorphism. It restores the vertex, while the extracted mode is definitionally the original mode. Equality of these two components gives equality of coordinates and then of the typed subtypes. □

Lean: `Linear.TypedPosition.coordinate_recovery`

Theorem 29.24 (Coordinate characterization)**.** *For every actual target state t,*

$$\text{realize}(p) \longrightarrow t \quad \Longleftrightarrow \quad \exists q, \quad p \longrightarrow_G q \ \wedge \ \text{realize}(q) \cong_f t.$$

Here the left relation is erasure-based state search, the middle relation is coordinate search on the original graph G, and $\cong_f$ preserves the focus and mode. Both realizations use the same original graph G.

Proof. Choose the recovery isomorphism a from the previous lemma. For the forward direction, a state-search witness consists of a successor coordinate on the erased graph and an isomorphism from its realization to t. Transport this coordinate and its step through a. Coordinate recovery identifies the source with p. Naturality of realization, inverted and composed with the supplied endpoint isomorphism, identifies the new realization with t.

Conversely, transport q and its step through a^{-1}. Inverse transport and coordinate recovery identify the source with the extracted coordinate of realize(p). Invert the realization isomorphism and compose it with the given isomorphism to t. The transported coordinate, transported step, and composite isomorphism are exactly the witnesses required by the state-search definition. □

Lean: `Linear.TypedPosition.search_realize_iff`

Taking $t = \text{realize}(q)$ and the identity endpoint isomorphism shows that every coordinate step realizes an actual state-search step between its two prescribed realizations. This strengthens the earlier endpoint-only realization theorem. It does not identify search with contextual graph rewriting.

Lean: `Linear.TypedPosition.search_realize`

29.11 Erasure and state search respect renaming

Fix a focussed isomorphism $a : f \cong_f g$. For any bijection carrying x to y, restrict it to the complements of x and y. Its forward and inverse maps are the original maps with non-deletion certificates: injectivity prevents a retained point from mapping to the deleted point, and the inverse law proves the analogous claim backwards. Subtype extensionality lifts both inverse laws.

Lean: `Linear.exceptOfEq`

The graph vertex bijection carries the selected internal target of f to that of g. Indeed, the focussed isomorphism identifies the targets, their selection certificates make them internal, and the internal constructor is injective. Restrict this vertex bijection at the selected targets and the edge bijection at the focus edges. These give bijections of the erased carriers.

Lean: `Linear.Focussed.Iso.targetVertex`

Lean: `Linear.Focussed.Iso.erasedVertices`

Lean: `Linear.Focussed.Iso.erasedEdges`

Lemma 29.25 (Naturality of contraction)**.** *Inclusion of retained ports commutes with the restricted vertex bijection. The joined positions correspond, and contraction commutes with renaming.*

Proof. Commutation with inclusion is a case analysis on ports. To compare joined positions, apply the injective inclusion on the target side. The resulting equation is precisely preservation of the focus source. For contraction, separate the deleted target from other vertices. In the first case both sides are the corresponding joined positions. In the second, injectivity ensures that the image is not the target either. Apply inclusion on the target side: both contraction–inclusion composites reduce to their original vertices, and the inclusion commutation identity proves equality. □

Lean: `Linear.Focussed.Iso.erased_lift`

Lean: `Linear.Focussed.Iso.erased_position`

Lean: `Linear.Focussed.Iso.erased_contract`

Theorem 29.26 (Naturality of erasure and state search)**.** *The restricted bijections form an isomorphism of the erased graphs, carrying the extracted coordinate of the first state to that of the second. Search is invariant under focus-preserving isomorphisms in both arguments.*

Proof. Retained labels inherit the original isomorphism's label equations. For each incidence list, use its original equation and commute contraction past the port map pointwise. This proves the erased graph isomorphism. Its coordinate equation follows from preservation of the mode and the joined position.

Given a search step from the first state, transport its successor coordinate through the erased isomorphism. The coordinate equation identifies the new source, and search transport supplies the step. Invert the realization isomorphism and compose with the old endpoint isomorphism to retain the same actual target. This proves source invariance. Combine it with the earlier target invariance to obtain invariance in both arguments; use the inverse isomorphisms for the reverse implication. □

Lean: `Linear.Focussed.Iso.erased`

Lean: `Linear.State.coordinate_iso`

Lean: `Linear.State.search_source_iso`

Lean: `Linear.State.search_iso_iff`

29.12 Search on isomorphism classes of states

Definition 29.27 (State isomorphism class)**.** Two states are equivalent when there exists a focus-preserving literal-label isomorphism between them. Reflexivity, symmetry, and transitivity follow from the corresponding isomorphism constructions. A state class is the quotient by this equivalence; write $[s]$ for the class of s.

This quotient preserves full edge labels literally, including box bodies. It does not identify independently renamed contents of nested boxes. Section 29.14 constructs the recursive state quotient and proves projection and exact-length search lifting between the two quotients.

Lean: `Linear.stateSetoid`

Lean: `Linear.StateClass`

Lean: `Linear.StateClass.ofState`

Definition 29.28 (Quotient search and observations)**.** Define $[s] \longrightarrow [t]$ by state search $s \longrightarrow t$. The preceding two-argument invariance proves independence of representatives, so quotient elimination defines a proposition on classes. Initiality and finality descend in the same way, using their focussed-isomorphism invariance.

On supplied representatives, all three descended propositions agree with the original propositions by definition. The representative equations are therefore reflexive equivalences, not extra assumptions.

Lean: `Linear.StateClass.Search`

Lean: `Linear.StateClass.search_iff`

Lean: `Linear.StateClass.Initial`

Lean: `Linear.StateClass.Final`

Lean: `Linear.StateClass.initial_iff`

Lean: `Linear.StateClass.final_iff`

Theorem 29.29 (Deterministic quotient search and its boundaries)**.** *If $S \longrightarrow T$ and $S \longrightarrow U$, then $T = U$. A final class has no search successor, and no search step reaches an initial class.*

Proof. Eliminate the three quotient arguments to representatives s, t, u. The two representative steps have a common source, so state-level determinism gives a focussed isomorphism $t \cong_f u$. Quotient soundness identifies their classes. For each boundary theorem eliminate its two class arguments; the goal becomes the already proved state-level boundary law. □

Lean: `Linear.StateClass.search_deterministic`

Lean: `Linear.StateClass.final_no_search`

Lean: `Linear.StateClass.no_search_initial`

These are results for the search relation alone. Substitution, computation, rootedness, refocusing, and contextual-rewrite adequacy remain necessary for the full concrete UAM argument.

29.13 Concrete quotient-search regression

The passive nullary value step now runs from the previously defined `initialValue` to the specific `finalValue`, using the general coordinate-to-state theorem. Erase and reconstruct `initialValue` to define `recoveredValue`. Its carriers are rebuilt by deletion and insertion, but state recovery supplies a focussed isomorphism to the original. Source invariance therefore gives a step from this reconstructed state to `finalValue`. Quotient soundness identifies its class with the original initial class. Finally, quotient determinism compares any successor of that initial class with the concrete value step: it must equal the class of `finalValue`.

Lean: `Linear.Examples.value_search_final`

Lean: `Linear.Examples.recoveredValue`

Lean: `Linear.Examples.recovered_value_search`

Lean: `Linear.Examples.recovered_value_class`

Lean: `Linear.Examples.value_class_successor`

29.14 Recursive renaming of focussed states

The preceding quotient fixes box-body labels literally. We now extend search to the recursive graph isomorphisms of Section 23, allowing independent changes of carrier names inside every nested box. All constructions stay at one fixed finite depth bound. Compatibility across depth bounds and with the paper's set-based representation remains a separate obligation.

Lemma 29.30 (Primitive labels and focus freedom)**.** *Recursive label equivalence preserves each specified primitive symbol exactly. It also preserves and reflects absence of focuses throughout a label, and recursive graph isomorphisms preserve and reflect global focus freedom.*

Proof. At depth zero label equivalence is equality. At positive depth a primitive can only relate to the same primitive: the primitive–box alternatives are false. This proves the first assertion, including the variable and operation symbols needed by search.

For focus freedom, induct on depth. Primitive cases again reduce to equality; mixed cases are impossible. For related boxes, take the supplied recursive isomorphism of their bodies. In the forward direction, choose the inverse image of each target edge under its edge bijection, apply the induction hypothesis to the related labels, and cancel the bijection with its inverse. In the reverse direction, apply the induction hypothesis to the image of each source edge. Applying this label result edgewise, with the same inverse-image argument, gives the graph-level equivalence. □

Lean: `Linear.labelEquiv_primitive`

Lean: `Linear.labelEquiv_focusFree`

Lean: `Linear.DeepIso.focusFree_iff`

Definition 29.31 (Focussed recursive isomorphism)**.** A focussed recursive isomorphism consists of a recursive graph isomorphism, equality of the modes, and an equation carrying the distinguished focus edge to the distinguished focus edge. Literal-label focussed isomorphisms embed by treating equality of labels as recursive label equivalence.

Identity uses recursive identity; inversion uses the inverse graph bijections and the reversed mode equation; composition composes the graph maps and both focus equations. Thus these constructions retain the distinguished data, not only the unpointed graph.

Lean: `Linear.Focussed.DeepIso`

Lean: `Linear.Focussed.DeepIso.refl`

Lean: `Linear.Focussed.DeepIso.symm`

Lean: `Linear.Focussed.DeepIso.trans`

Lean: `Linear.Focussed.DeepIso.ofIso`

The focus source and target correspond: apply the ordered incidence equations to the distinguished edge and use its singleton source and target lists. The port maps preserve input status, so these endpoint equations and mode equality preserve and reflect both initiality and finality.

Lean: `Linear.Focussed.DeepIso.source`

Lean: `Linear.Focussed.DeepIso.target`

Lean: `Linear.Focussed.DeepIso.initial_iff`

Lean: `Linear.Focussed.DeepIso.final_iff`

29.15 Search and insertion under recursive renaming

Let $a : G \cong_d H$ be a recursive graph isomorphism and a_* its outer port map. Wire labels still agree: internal labels agree by the graph-isomorphism field, and boundary labels agree because boundary positions are fixed.

Lean: `Linear.deepIso_vertexLabel`

Transport an operation view by the outer edge bijection and port map, retaining its operation symbol. The primitive-label lemma proves the operation-label certificate even when other edges carry independently renamed boxes. Ordered incidence certificates follow from the graph equations, and arity follows from preservation of list length. Transport a typed coordinate by keeping its mode and applying a_* to its vertex; wire-label preservation proves the new typing certificate.

Lean: `Linear.SearchOperation.deepRename`

Lean: `Linear.TypedPosition.deepRename`

Lemma 29.32 (Recursive transport of coordinate search)**.** *Operation entry and return commute with recursive transport. A search step between transported coordinates exists exactly when the original step does. Transporting a typed coordinate forwards and then backwards returns it.*

Proof. Entry examines the empty/nonempty argument-list alternatives, which mapping preserves. Return uses optional lookup at index $i+1$, which commutes with mapping. For search, induct on the rule: the variable case uses exact preservation of the variable label and singleton source; entry uses its commutation equation; return also maps the certified lookup at index i. This proves forward transport. Apply it to the inverse recursive isomorphism and cancel its port bijection to reflect steps. The same cancellation proves the typed-coordinate inverse law, by equality of the mode and vertex and then subtype extensionality. □

Lean: `Linear.SearchOperation.deepRename_enter`

Lean: `Linear.SearchOperation.deepRename_resume`

Lean: `Linear.SearchStep.deepRename`

Lean: `Linear.SearchStep.deepRename_iff`

Lean: `Linear.TypedPosition.deepRename_symm`

Theorem 29.33 (Recursive naturality of insertion)**.** *Inserting a focus at (m, v) in G is recursively focus-preservingly isomorphic to inserting it at $(m, a_* v)$ in H. Realization of typed coordinates respects this isomorphism.*

Proof. Extend the outer vertex and edge bijections by fixing the inserted singleton names. Hosted vertices map to hosted images, and the fresh vertex is fixed. Splitting commutes with this map because injectivity says $a_* w = a_* v$ exactly when $w = v$; hence both sides take the fresh branch in the same case.

Retained edge labels inherit their recursive equivalence from a. The fresh focus labels agree literally, so reflexivity supplies their recursive equivalence. Retained source lists use commutation with splitting, retained target lists use commutation with hosting, and the fresh focus equations use the hosted and fresh port identities directly. The mode and fresh focus edge are fixed. These fields give the focussed recursive isomorphism, and the state-realization specialization follows by supplying its focus-free and hole-free certificates. □

Lean: `Linear.FocusInsertion.deepRename_host`

Lean: `Linear.FocusInsertion.deepRename_fresh`

Lean: `Linear.FocusInsertion.deepRename_split`

Lean: `Linear.FocusInsertion.deepRename_iso`

Lean: `Linear.FocusInsertion.deepRename_focussed`

Lean: `Linear.TypedPosition.realize_deepRename`

29.16 Erasure under recursive renaming

Let $a : f \cong_{f,d} g$ be a focussed recursive isomorphism. Its outer vertex bijection carries the selected internal target to the selected internal target: use target preservation, rewrite both selection certificates, and cancel the internal-port constructor. Restrict this bijection to the undeleted vertices and restrict the edge bijection to the non-focus edges, using the existing complement-bijection construction.

Lean: `Linear.Focussed.DeepIso.targetVertex`

Lean: `Linear.Focussed.DeepIso.erasedVertices`

Lean: `Linear.Focussed.DeepIso.erasedEdges`

Lemma 29.34 (Retained ports and contraction)**.** *Inclusion of retained ports commutes with these restricted maps. The joined vertices correspond, and contraction commutes with recursive renaming.*

Proof. The inclusion equation holds by cases on ports. Apply the injective inclusion on the target side to compare joined vertices: their images are the focus sources, already identified by a. For contraction, if the port is the deleted target, both sides are corresponding joined vertices. Otherwise injectivity ensures that its image is also not deleted. Apply inclusion and use the contraction–inclusion identities on both sides to reduce to the inclusion commutation equation. □

Lean: `Linear.Focussed.DeepIso.erased_lift`

Lean: `Linear.Focussed.DeepIso.erased_position`

Lean: `Linear.Focussed.DeepIso.erased_contract`

The restricted maps form a recursive isomorphism of the erased graphs. Internal wire labels and retained recursive edge-label relations are inherited from a. For sources and targets, rewrite the original incidence equation and commute contraction with the port map pointwise along its ordered list. Nothing requires literal equality of boxed bodies.

Lean: `Linear.Focussed.DeepIso.erased`

29.17 State search modulo recursive endpoint renaming

Definition 29.35 (Recursively renamed state search)**.** Write $s \longrightarrow_d t$ when a typed coordinate q on the erased underlying graph of s satisfies the coordinate search rule from the extracted coordinate of s, and the realization of q is recursively focus-preservingly isomorphic to t.

Taking the actual realization as target gives a step by the identity isomorphism. Composing the supplied endpoint isomorphism with another proves closure under recursive target renaming.

Lean: `Linear.State.DeepSearch`

Lean: `Linear.State.deepSearch_realize`

Lean: `Linear.State.deepSearch_target_iso`

Theorem 29.36 (Recursive search invariance, determinism, and boundaries)**.** *Two recursive-search successors of the same state are focussed recursively isomorphic. Recursive search respects recursive isomorphisms in both arguments. Final states have no such search step, and no such step reaches an initial state.*

Proof. For determinism, compare the two witnessing coordinate steps on the identical erased source graph. Coordinate determinism identifies their results; subtype extensionality identifies their typed coordinates. Compose the inverse of one endpoint isomorphism with the other.

For source invariance, erasure naturality identifies the extracted coordinates: the mode equation gives the first component and the joined-position equation the second. Transport the successor coordinate and step through the erased recursive isomorphism. Naturality of realization identifies the transported realization with the old one, so compose its inverse with the old endpoint isomorphism. Combine this with target closure; apply inverse isomorphisms to obtain the converse and hence invariance in both arguments.

For a final source, its extracted coordinate is backtrack at an input, which has no coordinate search successor. For an initial target, transport its initiality backwards through the endpoint isomorphism and then through the realization boundary law. The successor coordinate would be search at an input, excluded by the coordinate no-initial-target theorem. □

Lean: `Linear.State.deepSearch_deterministic`

Lean: `Linear.State.coordinate_deepIso`

Lean: `Linear.State.deepSearch_source_iso`

Lean: `Linear.State.deepSearch_iso_iff`

Lean: `Linear.State.final_no_deepSearch`

Lean: `Linear.State.no_deepSearch_initial`

Theorem 29.37 (Exact relation to literal-label state search)**.** *Every literal-label state-search step is a recursive step. More precisely,*

$$s \longrightarrow_d t \quad \Longleftrightarrow \quad \exists u,\ s \longrightarrow u \ \wedge\ u \cong_{f,d} t.$$

Proof. A literal endpoint isomorphism embeds into a recursive one, giving the first claim. For the forward implication of the equivalence, choose u to be the actual realization of the witnessing successor coordinate. This is a literal state-search successor by identity, and the supplied recursive endpoint isomorphism identifies it with t. Conversely, embed the literal step and compose its endpoint witness with the supplied recursive isomorphism. □

Lean: `Linear.State.search_deepSearch`

Lean: `Linear.State.deepSearch_iff`

Thus the new relation changes the permitted endpoint representatives; it does not introduce another local search rule. It is not a claim that literal-label search between two fixed representatives is itself invariant under arbitrary changes of box-body labels.

29.18 Recursive state classes and their search relation

Definition 29.38 (Recursive state class)**.** Quotient states by existence of a focussed recursive isomorphism. The identity, inverse, and composition constructions prove this is an equivalence relation. Write $[s]_d$ for the class of s. Descend recursive search to pairs of classes using its proved two-argument invariance. Descend initiality and finality using their recursive-isomorphism invariance.

Lean: `Linear.deepStateSetoid`

Lean: `Linear.DeepStateClass`

Lean: `Linear.DeepStateClass.ofState`

Lean: `Linear.DeepStateClass.Search`

Lean: `Linear.DeepStateClass.Initial`

Lean: `Linear.DeepStateClass.Final`

Each descended proposition reduces to its representative proposition by definition, giving the three representative equivalences.

Lean: `Linear.DeepStateClass.search_iff`

Lean: `Linear.DeepStateClass.initial_iff`

Lean: `Linear.DeepStateClass.final_iff`

Theorem 29.39 (Deterministic search on recursive classes)**.** *Recursive quotient search has at most one successor class. Final classes have no search successor, and no search step reaches an initial class.*

Proof. Eliminate the quotient arguments to representatives. Two steps from a common representative have recursively isomorphic targets by state-level determinism; quotient soundness equates their classes. Each boundary assertion reduces to the corresponding representative-level boundary theorem. □

Lean: `Linear.DeepStateClass.search_deterministic`

Lean: `Linear.DeepStateClass.final_no_search`

Lean: `Linear.DeepStateClass.no_search_initial`

29.19 Projection and lifting between the two state quotients

Definition 29.40 (Forgetting literal body representatives)**.** Define $\pi([s]) = [s]_d$, mapping literal-label state classes to recursive state classes. This is well-defined because every literal focussed isomorphism embeds into a recursive one. On a specified representative the equation is reflexive. The map is surjective: choose the literal class of any representative of a recursive class.

Lean: `Linear.StateClass.toDeep`

Lean: `Linear.StateClass.toDeep_ofState`

Lean: `Linear.StateClass.toDeep_surjective`

Theorem 29.41 (Search projection and lifting)**.** *A literal-class step $S \longrightarrow U$ gives $\pi(S) \longrightarrow_d \pi(U)$. Conversely, for every specified literal class S, a step $\pi(S) \longrightarrow_d T$ lifts to some step $S \longrightarrow U$ with $\pi(U) = T$. Consequently,*

$$\pi(S) \longrightarrow_d T \quad \Longleftrightarrow \quad \exists U,\ S \longrightarrow U \ \wedge\ \pi(U) = T.$$

Proof. For projection, eliminate both literal classes to representatives and embed the literal state-search step. For lifting, eliminate the source literal class and target recursive class. The exact endpoint characterization supplies a literal successor u of the fixed source representative together with a recursive isomorphism to the chosen target representative. Its literal class is the desired U, and quotient soundness proves the projection equation. The displayed equivalence combines projection and lifting. □

Lean: `Linear.StateClass.search_toDeep`

Lean: `Linear.StateClass.search_lift`

Lean: `Linear.StateClass.search_toDeep_iff`

Initiality and finality agree before and after projection, by quotient elimination and their representative definitions. Absence of search successors also agrees: a literal successor projects

to a recursive successor, and a recursive successor lifts to a literal successor. Taking the contrapositive in each direction proves the normal-form equivalence.

Lean: `Linear.StateClass.initial_toDeep`

Lean: `Linear.StateClass.final_toDeep`

Lean: `Linear.StateClass.no_search_toDeep`

29.20 Exact-length executions across the quotients

Definition 29.42 (Search-only machines)**.** On each state quotient, form a `Machine` whose step relation is its search relation and whose final predicate is its boundary finality predicate. The search boundary theorem proves the required final-normal law. Its search-determinism theorem proves determinism of this machine.

These machines deliberately contain only search. They are not the full UAM: substitution and computation are absent. Their finite executions use the already defined `Machine.Steps` relation, so the length counts search steps exactly, without introducing a second execution datatype.

Lean: `Linear.StateClass.searchMachine`

Lean: `Linear.DeepStateClass.searchMachine`

Lean: `Linear.StateClass.searchMachine_deterministic`

Lean: `Linear.DeepStateClass.searchMachine_deterministic`

Theorem 29.43 (Exact-length projection and lifting)**.** *For every $n \in \mathbb{N}$, literal executions project with length unchanged, and every recursive execution of length n from $\pi(S)$ lifts from the specified literal class S with length unchanged. Thus*

$$\pi(S) \longrightarrow_d^n T \quad \Longleftrightarrow \quad \exists U,\ S \longrightarrow^n U \ \wedge\ \pi(U) = T.$$

Proof. Projection is induction on the execution: the zero-step execution remains zero, and a step followed by a shorter run maps to its projected step followed by the projected shorter run.

For lifting, induct on n, allowing arbitrary source and target classes. At zero, the execution identifies the target with $\pi(S)$, so take $U = S$. At $n + 1$, lift the first recursive step from the specified source to a literal successor V. Rewrite the recursive tail's source using $\pi(V)$, apply the induction hypothesis from V, and prepend the lifted first step. The endpoint equality and the exact length are retained. The equivalence combines both constructions. □

Lean: `Linear.StateClass.steps_toDeep`

Lean: `Linear.StateClass.steps_lift`

Lean: `Linear.StateClass.steps_toDeep_iff`

Theorem 29.44 (Exact-length search termination)**.** *For each prescribed n, a successful search run of length n exists from S if and only if one exists from $\pi(S)$.*

Proof. Project or lift the run using the previous theorem. In the lifting direction, rewrite finality along the endpoint projection equality and use reflection of finality. In the projection direction, use preservation of finality. □

Lean: `Linear.StateClass.final_run_toDeep_iff`

29.21 Recursive-renaming and execution regression proofs

The concrete two-hole box is globally focus-free because every edge in its body is a hole rather than a focus. The established recursive isomorphism to the box with its body's edge names exchanged transports this focus freedom. These two boxed graphs were already proved to admit no literal-label isomorphism, so the test exercises genuinely recursive label comparison.

Lean: `Linear.Examples.boxedTwoHoles_focusFree`

Lean: `Linear.Examples.renamedBoxedTwoHoles_focusFree`

The earlier concrete value step embeds into recursive search. Prepending that step to the zero-step execution gives an execution of exactly one step from its initial recursive class to its final recursive class. To test lifting from a reconstructed source, identify its literal class with the original initial class, use this recursive execution, and apply the exact-length termination equivalence backwards. The result is a successful literal search run of exactly one step from the reconstructed source. Finally, the final value's boundary finality and the recursive no-search-from-final theorem exclude every recursive search successor.

Lean: `Linear.Examples.value_deep_run`

Lean: `Linear.Examples.recovered_value_lift`

Lean: `Linear.Examples.final_value_deep_normal`

29.22 Canonical roots of concrete states

This milestone connects the existing search semantics to Appendix C. We construct the paper's rooted predicate, prove that search preserves it, and establish concrete versions of Lemmas 23 and 26(1). These results use our erasure-based search semantics on recursive state classes; identifying that semantics with the paper's contextual local rewrites remains an adequacy obligation. The proofs below do not assume the abstract specimen locality laws.

Definition 29.45 (Canonical initial attachment)**.** For a concrete state s, erase its focus, attach a searching focus at the sole input of the resulting graph, and call the resulting state $r(s)$. Focus freedom and hole freedom of erasure supply the certificates required for realization. Its focus is initial by the initial-realization theorem.

Lean: `Linear.State.root`

Lean: `Linear.State.root_initial`

Lemma 29.46 (Roots respect renaming and initiality)**.** *Recursively isomorphic underlying graphs give recursively focus-preservingly isomorphic roots. In particular, a focussed recursive isomorphism between states gives one between their roots. If s is already initial, $r(s)$ is recursively focus-preservingly isomorphic to s.*

Proof. The coordinate "search at input zero" is fixed by every boundary-preserving port map. Apply recursive naturality of realization to the given underlying isomorphism. For focussed state isomorphisms, first use erasure naturality.

For an initial state, the extracted mode is search and its extracted vertex is an input. The state interface has exactly one input, so that vertex is input zero. Thus the extracted typed coordinate equals the initial coordinate, by component equality and subtype extensionality. Rewrite the general recovery isomorphism using this coordinate equality, and embed the literal-label isomorphism into a recursive one. □

Lean: `Linear.State.root_of_underlying`

Lean: `Linear.State.root_iso`

Lean: `Linear.State.coordinate_initial`

Lean: `Linear.State.root_initial_iso`

Lemma 29.47 (Search preserves the underlying graph)**.** *If $s \longrightarrow_d t$, their erased underlying graphs are recursively isomorphic.*

Proof. Let q be the witnessing successor coordinate on the erased graph of s. Insertion–erasure cancellation identifies the underlying graph of realize(q) with the underlying graph of s. Invert that isomorphism and compose it with erasure of the supplied recursive endpoint isomorphism from the realization to t. □

Lean: `Linear.State.deepSearch_underlying`

Descend r to recursive state classes using root isomorphism invariance. Its value on a representative is its representative root class, by definition. Every root class is initial. An initial class is fixed by r, using quotient soundness and the initial-state recovery lemma. In

particular $r(r(S)) = r(S)$. Search preserves $r(S)$ because it preserves the underlying graph; induction on a search execution then proves preservation of the root along the whole run.

Lean: `Linear.DeepStateClass.root`

Lean: `Linear.DeepStateClass.root_ofState`

Lean: `Linear.DeepStateClass.root_initial`

Lean: `Linear.DeepStateClass.root_eq_of_initial`

Lean: `Linear.DeepStateClass.root_idempotent`

Lean: `Linear.DeepStateClass.search_root`

Lean: `Linear.DeepStateClass.steps_root`

29.23 Concrete rootedness and its search invariant

Definition 29.48 (Concrete rooted state)**.** A recursive state class S is rooted when there is a finite search-only execution from $r(S)$ to S:

$$\mathsf{Rooted}(S) \quad \Longleftrightarrow \quad \exists n \in \mathbb{N},\ r(S) \longrightarrow_d^n S.$$

A named state is rooted when its recursive isomorphism class is rooted. This definition depends on the operation classification used by search.

Lean: `Linear.DeepStateClass.Rooted`

Lean: `Linear.State.Rooted`

Theorem 29.49 (Initial states are rooted; search preserves rootedness)**.** *Every initial class is rooted, every canonical root is rooted, and search steps and finite search executions preserve rootedness.*

Proof. An initial class equals its root, so the zero-step run witnesses rootedness. Every root is initial, giving the second claim. For a step $S \longrightarrow_d T$, append that step to the witnessing run from $r(S)$ to S. Search preservation of roots identifies $r(S)$ with $r(T)$, so this is the required run to T. Induction on a finite execution gives the final claim. □

Lean: `Linear.DeepStateClass.initial_rooted`

Lean: `Linear.DeepStateClass.root_rooted`

Lean: `Linear.DeepStateClass.rooted_search`

Lean: `Linear.DeepStateClass.rooted_steps`

Theorem 29.50 (Equivalent initial-run characterization)**.** *A class S is rooted exactly when some initial class has a finite search run to S. The initial class in any such run must equal $r(S)$.*

Proof. The forward direction chooses $r(S)$, which is initial. Conversely, suppose I is initial and reaches S. Initiality gives $I = r(I)$, and preservation of roots along the run gives $r(I) = r(S)$. Rewrite the run's source by these equalities to obtain the rootedness witness. □

Lean: `Linear.DeepStateClass.rooted_iff_initial_run`

29.24 Operation reachability under coordinate search

Definition 29.51 (Operation-reachable vertex)**.** A vertex v is operation-reachable if there are an input index i and an edge list E with an operation trace from v backwards to input i. Equivalently, reversing E gives an operation path from that input to v. An input is reachable using the empty trace. Each certified operation view has an actual operation label, by its label certificate and the definition of primitive operation labels at each depth.

Lean: `Linear.OperationReachable`

Lean: `Linear.OperationReachable.input`

Lean: `Linear.SearchOperation.isOperation`

Lemma 29.52 (Argument and source reachability)**.** *If the source of an operation is reachable, every term argument is reachable. Conversely, if one of its arguments is reachable, its source is reachable.*

Proof. Forwards, prepend the operation edge to the reverse trace from its source; its certified argument membership supplies the target incidence. Backwards, inspect a witnessing trace from the argument to an input. It cannot be empty, since an edge cannot target an input. Its first edge equals the given operation edge by uniqueness of incoming incidence. The operation's singleton source then identifies the next vertex in the trace with its source. The remaining trace proves source reachability. □

Lean: `Linear.OperationReachable.argument`

Lean: `Linear.OperationReachable.source`

Theorem 29.53 (Search preserves operation reachability)**.** *Every coordinate search rule preserves operation reachability of its vertex. Recursive renaming preserves and reflects this property.*

Proof. A variable trigger keeps the vertex. Operation entry either keeps its source, for no term arguments, or moves to the first argument, using forward argument reachability. Return from an argument first gives reachability of the operation source by the converse lemma. It then either stays there on completion or moves to the next argument by the forward lemma.

For renaming, transport the reverse operation trace and its edge list through the recursive graph isomorphism. The boundary input is fixed. Apply the same construction to the inverse isomorphism and cancel the port maps for reflection. □

Lean: `Linear.SearchStep.operationReachable`

Lean: `Linear.OperationReachable.rename`

Lean: `Linear.OperationReachable.rename_iff`

29.25 Incoming paths and insertion of the focus

Definition 29.54 (Incoming incidence trace)**.** An incoming trace is a reverse path with explicit vertex incidences but no label restriction. It is either empty at a vertex or consists of an edge whose target contains the current vertex, a chosen source vertex of that edge, and a remaining incoming trace from that source.

Lean: `Linear.IncomingTrace`

Lemma 29.55 (Incoming paths to reachable vertices)**.** *An edge targeting an operation-reachable vertex is an operation, and each of its sources is operation-reachable. Consequently, every incoming trace ending at a reachable vertex is an operation trace with the same vertices and edge list.*

Proof. Compare the incoming edge with the first edge of a witnessing reverse operation trace to an input. The trace cannot be empty because inputs have no incoming edge. Target-incidence uniqueness identifies the two edges, proving its operation label. An operation has exactly one source, so every chosen source is the next vertex in the witnessing trace and is reachable by its tail. Induct on the arbitrary incoming trace and apply this argument at each step. □

Lean: `Linear.OperationReachable.predecessor`

Lean: `Linear.IncomingTrace.operations`

Lean: `Linear.OperationReachable.incoming_operation`

Lemma 29.56 (A path never departs from its final vertex earlier)**.** *Suppose an operation trace from v to an input has length k. In any operation trace to an input with length at most k, no edge can have v as a source. In particular the original trace never has v as a source of one of its edges.*

Proof. Induct on the shorter trace's edge list. If its first edge has v as source, its singleton-source property identifies the next vertex with v. The tail and the original length-k trace therefore both run from v to inputs. Uniqueness of operation traces to inputs equates their edge lists and hence their lengths. But the tail is shorter than the trace of length at most k, a contradiction. For a later edge, apply the induction hypothesis to the tail, whose length is still at most k. □

Lean: `Linear.operationTrace_sources_avoid`

Theorem 29.57 (Lifting the operation path through focus insertion)**.** *An operation path from an input to v survives insertion of a focus at v: it becomes a path from the same input to the hosted copy of v, using the retained copies of its original edges.*

Proof. The previous lemma says no source occurrence on this path equals v. Insertion therefore maps all its source occurrences by hosting rather than to the fresh focus target. Targets are always hosted. Induct on the reverse trace, retaining each operation label and mapping each source and target membership into its incidence list. The empty trace and the boundary input are fixed. This proves a path in the actual inserted graph, not just in its focus erasure. □

Lean: `Linear.operationTrace_insert`

29.26 Rooted states have actual operation paths: Lemma 23

Define the intermediate invariant $U(s)$ to mean that the extracted coordinate of s is operation-reachable in its erased graph. It is invariant under focussed recursive isomorphisms: erasure naturality transports the graph, coordinate naturality identifies the vertex, and reachability is invariant under this transport.

Lean: `Linear.State.UnderlyingReachable`

Lean: `Linear.State.underlyingReachable_iso`

Lemma 29.58 (Realization and search preserve the intermediate invariant)**.** *Realizing a reachable typed coordinate satisfies U. Every canonical root satisfies U, and recursive state search preserves U.*

Proof. Coordinate recovery provides one isomorphism that restores both the original graph and the chosen coordinate after realizing and erasing. Reflect operation reachability along this isomorphism to prove the realization assertion. The initial coordinate is an input, hence reachable, which proves the root assertion. For a search step, coordinate search preserves reachability; apply the realization assertion to its successor coordinate and transport the result through its supplied recursive endpoint isomorphism. □

Lean: `Linear.State.underlyingReachable_realize`

Lean: `Linear.State.root_underlyingReachable`

Lean: `Linear.State.deepSearch_underlyingReachable`

The invariant descends to recursive classes using its isomorphism invariance. Quotient elimination proves the root and one-step preservation laws there. Induction on the witnessing root run proves that every rooted class satisfies U. This is the search-sequence induction used for the path argument in Appendix C, expressed through an invariant of erased coordinates.

Lean: `Linear.DeepStateClass.UnderlyingReachable`

Lean: `Linear.DeepStateClass.root_underlyingReachable`

Lean: `Linear.DeepStateClass.search_underlyingReachable`

Lean: `Linear.DeepStateClass.rooted_underlyingReachable`

Theorem 29.59 (Concrete rooted-focus operation path)**.** *If a concrete state is rooted, its actual focus source is reachable from its unique input by an operation path. If the focus source is not an input, this path is nonempty.*

Proof. First consider any state satisfying U. Lift its witnessing erased operation trace through insertion at its extracted coordinate, by the insertion theorem. The general recovery isomorphism carries this inserted graph to the actual state and carries the inserted focus source to its actual focus source. Transport the lifted operation trace through that isomorphism. Boundary preservation fixes the input. This gives operation reachability in the actual state graph.

A rooted state satisfies U by the invariant proof, so the construction applies. The interface has one input, identifying its index with zero. Reverse the trace list to obtain the existing forward operation-path representation. If this list were empty, the zero-step reverse trace would identify the focus source with the input; hence the path is nonempty whenever the source is not an input. □

Lean: `Linear.State.focus_operationReachable`

Lean: `Linear.State.rooted_focus_operationReachable`

Lean: `Linear.State.rooted_focus_path`

This establishes the graph-path conclusion of Lemma 23 for the present concrete search model. Unlike the paper's presentation, the proof does not first require a separate uniqueness theorem for whole search runs: it uses the operation-path uniqueness already proved and a direct search invariant.

29.27 The non-operation-path obstruction: Lemma 26(1)

Theorem 29.60 (Incoming-path obstruction)**.** *Every incoming incidence trace to a rooted focus source is an operation trace. Thus an incoming trace that is not an operation trace proves that the state is not rooted.*

Proof. The preceding rooted-focus theorem gives reachability of the actual source. Apply the incoming-trace theorem at that vertex. The obstruction is its contrapositive. □

Lean: `Linear.State.rooted_incoming_operations`

Lean: `Linear.State.not_rooted_of_nonoperation_path`

Lemma 29.61 (The existing edge-path representation)**.** *For any nonempty edge path whose last edge targets a reachable vertex, every edge of the path is an operation, and every source of each edge is reachable. Repeated edges are allowed in the supplied path datatype.*

Proof. Induct on the path. For a single edge, use the incoming-operation and source reachability lemmas at its target. For a path beginning with adjacent edges e, f, apply the induction hypothesis to the tail beginning at f. The adjacency witness is a source of f, hence reachable, and a target of e. The incoming-edge lemmas then prove the required claims for e, while the induction hypothesis proves them for all remaining edges. □

Lean: `Linear.path_operations_of_reachable`

Theorem 29.62 (Concrete Lemma 26(1))**.** *If a nonempty graph path ends at the focus source of a rooted state, all of its edges are operations. Therefore a path ending there and containing any non-operation edge is a certificate that the state is not rooted.*

Proof. Apply the edge-path lemma to the actual focus-source reachability supplied by rootedness. For the obstruction, select the non-operation edge from the path and contradict the resulting operation-label assertion. □

Lean: `Linear.State.rooted_path_operations`

Lean: `Linear.State.not_rooted_of_path`

This is the first part of the paper's obstruction lemma in our concrete path and search representations. The one-way replacement argument in Lemma 26(2) is proved separately in Section 29.33. The general output-closed specimen argument in Lemma 26(3) is proved for shallow specimens in Section 29.40; its deep extension remains open.

29.28 Concrete refocusing and the remaining rewrite obligation

Definition 29.63 (Concrete refocusing)**.** A transition relation on recursive state classes is refocusing when each transition from a concretely rooted state leads to a concretely rooted state. The rooted predicate is the one constructed from actual search above; it is not the abstract specimen interface's supplied rooted predicate.

Lean: `Linear.DeepStateClass.Refocusing`

Theorem 29.64 (Search refocusing and reduction to rewriting)**.** *Search is refocusing. For any supplied rewrite relation R, the relation* Search $\cup R$ *is refocusing if and only if R preserves rootedness. For any machine on these concrete classes whose step relation is refocusing, every execution from an initial class ends in a rooted class.*

Proof. Search refocusing is the proved one-step rootedness invariant. If the union is refocusing, apply its law to transitions from its R branch. Conversely, case-split a union transition: use search preservation in the search branch and the supplied preservation law in the rewrite branch. Finally induct on a machine execution, starting with the theorem that initial classes are rooted and applying its refocusing hypothesis at each step. □

Lean: `Linear.DeepStateClass.search_refocusing`

Lean: `Linear.DeepStateClass.refocusing_with_search`

Lean: `Linear.DeepStateClass.refocusing_execution`

The last two statements are conditional on the indicated rewrite or machine preservation hypothesis. They do not construct substitution or compute transitions, and do not assert that arbitrary rewrite relations preserve roots.

29.29 Rooted and non-rooted concrete examples

The initial value state is rooted for any operation classification, by initiality. Its successful passive search execution gives a rooted final value state by preservation along search runs.

Lean: `Linear.Examples.initial_value_rooted`

Lean: `Linear.Examples.final_value_rooted`

The previously constructed interior-focus state is valid and hole-free but is not rooted, for any operation classification. Its underlying closed chain has a variable before the focus position and a value after it. In the actual inserted graph, the retained variable edge targets the focus source. The single-edge path consisting of that variable therefore ends at the source, and its label is not an operation. The concrete Lemma 26(1) obstruction rules out rootedness. This separates graph validity from the operational invariant and exercises the obstruction on an actual named state.

Lean: `Linear.Examples.interior_state_not_rooted`

29.30 One-way hypernets and the term-boundary restriction

This milestone proves the single-hole case of Appendix C, Lemma 26(2), for actual shallow substitution and the concrete rooted-state predicate. The proof follows the paper's reason for introducing one-way hypernets: an operation path from the global input cannot pass through such a filling and emerge at a term output. We make the path-extraction argument explicit and allow the replaced edge to have a self-loop.

Lemma 29.65 (Sources of operation paths have term type)**.** *Every source vertex of an actual operation edge has type $\star$. If a nonempty reverse operation trace ends at an input, that input has type $\star$.*

Proof. The signature of every primitive operation has exactly one source, of type $\star$. Apply the source-typing equation to any member of its source list. For the second assertion, induct on the trace's edge list. If its tail is empty, its endpoint is the source of the first operation and the first assertion applies. Otherwise apply the induction hypothesis to the tail. Boxes, variables, focuses, and holes are excluded by the operation predicate. □

Lean: `Linear.operation_source_term`

Lean: `Linear.operationTrace_input_term`

Definition 29.66 (One-way hypernet)**.** A hypernet $H : I \to O$ is *one-way* if, for every term input i and term output j, there is no operation path from i to j. Paths to thunk outputs are deliberately not excluded. This is the restriction in the paper's Section 6.1, expressed using our existing operation-path datatype.

Lean: `Linear.OneWay`

Theorem 29.67 (Reachability characterization)**.** *A hypernet is one-way if and only if none of its term outputs is operation-reachable from an input.*

Proof. A reverse trace from an output to an input cannot be empty: the boundary constructors are disjoint. Its input therefore has term type by the preceding lemma. Reverse its recorded edge list to obtain the forbidden forward operation path. Conversely any forbidden forward path supplies precisely a reverse trace witnessing operation reachability of its term output. □

Lean: `Linear.oneWay_iff`

One-wayness is invariant under recursive graph isomorphism. Such an isomorphism fixes the ordered boundary and preserves and reflects operation reachability, so the characterization applies in both directions. A graph with no outputs is one-way vacuously, since its output index type is empty.

Lean: `Linear.DeepIso.oneWay_iff`

Lean: `Linear.oneWay_no_outputs`

29.31 Extracting an operation path from a filling

Fix a host C, a selected edge h, and a filling H with the interface of h. Write $P = C[H]$, ι_C for the retained-host vertex map, and ι_H for the filling map. These are the existing maps of shallow replacement; ι_H need not be injective when h has a self-loop.

The elementary decomposition used in the induction says that a nonempty reverse operation trace with edge list $e :: E$, from v to z, consists exactly of an intermediate vertex w, an operation label on e, incidences $v \in \mathrm{target}(e)$ and $w \in \mathrm{source}(e)$, and a trace with edge list E from w to z. Both directions follow directly from the trace constructor.

Lean: `Linear.operationTrace_cons_iff`

Lemma 29.68 (An inserted non-input stays a non-input)**.** *If a vertex v of H is not an input, its image $\iota_H(v)$ is not an input of P.*

Proof. Target coverage in H supplies an edge targeting v. Its inserted copy targets $\iota_H(v)$ in P. Inputs of a well-formed graph have no incoming edge, which proves the claim, including when v is a filling output. □

Lean: `Linear.OneWayReplacement.insert_not_input`

Theorem 29.69 (Extraction of a filling-local input path)**.** *If $\iota_H(v)$ is operation-reachable in P, then v is operation-reachable inside H.*

Proof. Choose a reverse operation trace from $\iota_H(v)$ to a global input and induct on its length, generalizing v. If v is already an input of H, the empty local trace is the required witness. Otherwise, in the zero-length case, $\iota_H(v)$ would be a global input, contradicting the previous lemma.

In the nonempty case, target coverage supplies an edge f of H targeting v. Its inserted copy and the first edge of the global trace target the same vertex. Unique incoming incidence identifies these two edges. The first edge is consequently a filling edge and retains its operation label. Its chosen source is the image of some source w of f. Apply the induction hypothesis

to the shorter global trace from that image; this yields a local operation trace from w to an input of H. Prepending f yields the desired local trace from v.

The induction stops on first reaching a filling input. It neither reflects an arbitrary whole edge path through ι_H nor assumes injectivity of that map. This is why the proof remains valid for self-looping holes, unlike unrestricted path reflection discussed earlier. □

Lean: `Linear.OneWayReplacement.extract`

29.32 Reflecting host reachability through a one-way filling

Theorem 29.70 (Host term reachability reflects)**.** *Suppose H is one-way and v is a term vertex of C. If $\iota_C(v)$ is operation-reachable in $C[H]$, then v is operation-reachable in C.*

Proof. Induct on the edge list of a reverse operation trace from $\iota_C(v)$ to a global input. In the empty case injectivity of the retained-host map identifies v with that same input.

For a nonempty trace, inspect its first edge. If it is a surviving host edge, reflect its target incidence by injectivity of ι_C and express its source as $\iota_C(w)$. The corresponding host edge is an operation, so w has term type. Apply the induction hypothesis to the shorter trace and prepend the host edge.

If the first edge is inserted from H, its target is some $\iota_H(x) = \iota_C(v)$. A filling target cannot be an input. The boundary classification of replacement therefore forces x to be an output of H. Both replacement maps preserve vertex labels, so this output has term type. Extraction applied to the original trace now makes it operation-reachable inside H, contradicting one-wayness. Thus only the retained-host case can occur. □

Lean: `Linear.OneWayReplacement.reflect`

Theorem 29.71 (Incoming host paths consist of operations)**.** *Under the same hypotheses, every edge of any host path ending at v is an operation. The given host path is allowed to contain the replaced edge.*

Proof. Reflect operation reachability to C, then apply the earlier theorem that all edges of any path ending at an operation-reachable vertex are operations. That theorem uses the existing nonempty edge-path representation and allows repeated edges. No assertion that the supplied host path itself survives filling is needed. □

Lean: `Linear.OneWayReplacement.path_operations`

Theorem 29.72 (One-way filling closure)**.** *If both C and H are one-way, then $C[H]$ is one-way.*

Proof. Suppose a term output of $C[H]$ were operation-reachable. It is the retained copy of the corresponding term output of C. Reflection would make that output operation-reachable in C, contradicting the host's one-wayness. □

Lean: `Linear.OneWayReplacement.oneWay_plug`

29.33 The concrete one-way obstruction: Lemma 26(2)

For a focussed host $C : \star \to \epsilon$, replace an edge other than its focus by a globally focus-free filling H. Once the resulting graph is proved globally hole-free, package it with its retained focus as an actual state. The constructor requires these focus-freedom and hole-freedom proofs; it does not infer them merely from the interfaces. Its focus source is $\iota_C(s)$, where s is the host's focus source.

Lean: `Linear.Focussed.filledState`

Theorem 29.73 (Concrete Lemma 26(2))**.** *Suppose the focussed host has a path ending at its focus source and that path contains a non-operation edge. If the filling H is one-way, the resulting actual state is not rooted.*

Proof. Assume the filled state is rooted. The concrete Lemma 23 gives operation reachability of its focus source $\iota_C(s)$ in the actual filled graph. The source s of the host focus has term type. Apply the preceding host-path theorem: every edge of the supplied host path must be an operation. This contradicts its specified non-operation edge. □

Lean: `Linear.Focussed.oneWay_not_rooted`

This proves the single-hole one-way obstruction for the current concrete search semantics. It follows the uniqueness-and-extraction argument of Proof C.8, packaged as a reflection lemma rather than a split on whether the supplied path contains the hole. In fact the Lean result permits replacement at any non-focus edge with the right interface, so it includes the paper's hole-labelled context case. The example below uses a genuine hole label.

Theorem 29.74 (Paired single-hole obstruction)**.** *If H and K are two admissible fillings of the same focussed host and at least one is one-way, then at least one of the two filled states is not rooted, provided the host has the same non-operation path to its focus source.*

Proof. Split the hypothesis that H or K is one-way. Apply the preceding theorem to that filling and introduce the corresponding side of the conclusion. □

Lean: `Linear.Focussed.oneWay_pair_not_rooted`

This is a single-hole instance of the output-closure argument. Section 29.40 defines concrete pre-templates, output-closure, and shallow multi-hole specimens and proves their Lemma 26(3) obstruction. Deep specimen substitution remains open.

29.34 One-way examples and a filled-state obstruction

The variable corolla is one-way. Its term output is targeted by a variable edge. If that output were operation-reachable, the incoming-edge theorem would force this variable edge to be an operation, a contradiction. By contrast, the existing operation corolla with two term arguments is not one-way: its sole edge is a one-step operation path from the input to the first term output.

Lean: `Linear.Examples.variable_oneWay`

Lean: `Linear.Examples.operation_not_oneWay`

Define an operation signature with no term arguments and one thunk argument, and take its one-edge corolla. This graph has a term input and only a thunk output. It is one-way because no output has term type, yet its sole operation edge explicitly witnesses reachability of its thunk output. Together these examples verify both the nontriviality of the predicate and the importance of retaining the paper's term-output restriction.

Lean: `Linear.Examples.thunkOnlyOperations`

Lean: `Linear.Examples.thunkOnlyGraph`

Lean: `Linear.Examples.thunk_only_oneWay`

Lean: `Linear.Examples.thunk_output_reachable`

For a nonvacuous state-level application, start with the earlier closed chain whose interior focus follows a variable and precedes a value. Relabel that variable by a hole with one term input and one term output, preserving all incidences. The other labels remain the focus and the value operation. Their signatures are unchanged, so the relabelled graph is well formed. Retain the same distinguished focus, mode, and endpoints. The only other edges are a hole and an operation, both focus-free; the focus therefore remains globally unique and satisfies its original endpoint conditions.

Lean: `Linear.Examples.oneWayContext`

Lean: `Linear.Examples.oneWayContext_focus`

Fill the genuine hole by the variable corolla. The exact hole inventory of shallow substitution shows the resulting graph is hole-free: the filling is hole-free and no hole survives in the host. Indeed the only host edge that could carry a hole is exactly the removed edge. Global focus freedom of the variable filling was already checked.

Lean: `Linear.Examples.oneWayContext_filled_closed`

Finally, take the host path consisting only of its hole edge. Its target is the focus source, and its label is not an operation. The variable filling is one-way, so the concrete Lemma 26(2) theorem proves that this actual filled state is not rooted, for every operation classification. The offending host edge is removed by substitution: this exercises the new extraction argument, not merely preservation of a path of retained host edges.

Lean: `Linear.Examples.oneWay_filled_not_rooted`

29.35 Simultaneous shallow substitution: carriers and incidence

The next milestone proves the multi-hole obstruction for a concrete *shallow* specimen representation. This needs a simultaneous filling construction: the one-way side of a related pair may differ at different holes, so one cannot simply assume that all left fillings, or all right fillings, are one-way. The construction below works for arbitrary finite typed graphs; only the later path arguments specialize to hypernets.

Definition 29.75 (Simultaneous expansion)**.** Let $C : I \to O$ be a finite typed graph and, for each edge e of C, choose a graph H_e with exactly the source and target interface of e. The expansion $C[H]$ replaces every edge at once. Its internal vertices and edges have carriers

$$V_C + \sum_{e \in E_C} V_{H_e}, \qquad \sum_{e \in E_C} E_{H_e}.$$

An unchanged edge will be represented by its corolla. The vertex carrier adds the same ordered external input and output constructors as before. The host map ι_C retains each original vertex. For each e, the map ι_e sends a filling internal vertex to its tagged fresh copy, its ith input to $\iota_C(\mathrm{sourcePort}(e, i))$, and its jth output to $\iota_C(\mathrm{targetPort}(e, j))$.

Lean: `Graph.Net.Expansion.Internal`

Lean: `Graph.Net.Expansion.Vertex`

Lean: `Graph.Net.Expansion.Edge`

Lean: `Graph.Net.Expansion.host`

Lean: `Graph.Net.Expansion.insert`

The host map is injective and its image is disjoint from every fresh internal copy. Both facts follow by cases on the boundary/internal constructors and on the disjoint-sum tag.

Lean: `Graph.Net.Expansion.host_injective`

Lean: `Graph.Net.Expansion.host_ne_internal`

Lemma 29.76 (Uniqueness of a filling's owner)**.** *If two non-output filling vertices have equal images, their owning host edges are equal. The same holds for two non-input filling vertices. Within one filling, the insertion map is injective on non-output vertices, and separately on non-input vertices.*

Proof. For the first assertion, two fresh internal images identify their owner tags. An internal image cannot equal a boundary attachment in the host image. The only remaining case compares two filling inputs. Their images are source ports of their owning host edges; injectivity of the host map and unique outgoing incidence identify those edges. For non-input vertices the same argument uses output ports and unique incoming incidence.

Once the owner is fixed, two internal images identify their internal names; two input images identify their input indices by distinctness of the host source ports. The output argument uses target-port distinctness. No injectivity is asserted between a filling input and output: a self-looping host edge can identify them. □

Lean: Graph.Net.Expansion.source_owner

Lean: Graph.Net.Expansion.target_owner

Lean: Graph.Net.Expansion.insert_source_injective

Lean: Graph.Net.Expansion.insert_target_injective

29.36 Well-formedness of simultaneous expansion

The source and target lists of a new edge (e, x) are those of x in H_e, mapped by ι_e. Each list remains duplicate-free: source vertices are never outputs, target vertices are never inputs, and the respective insertion injectivity lemmas therefore apply to every member of the list.

Lean: Graph.Net.Expansion.source

Lean: Graph.Net.Expansion.target

Lean: Graph.Net.Expansion.source_nodup

Lean: Graph.Net.Expansion.target_nodup

Lemma 29.77 (Unique incidence in the expanded graph)**.** *If two expanded edges share a source vertex, they are equal; the analogous statement holds for a shared target vertex.*

Proof. Pull the common incidence back along each mapped list. The owner lemma first identifies the two original host edges. Insertion injectivity then identifies the two filling vertices. Unique incidence within that one filling identifies the two filling edges, hence the tagged expanded edges. □

Lean: Graph.Net.Expansion.source_unique

Lean: Graph.Net.Expansion.target_unique

Lemma 29.78 (Boundary exclusion and coverage)**.** *No expanded source is an output and no expanded target is an input. Every vertex that is not an output has a source incidence, and every vertex that is not an input has a target incidence.*

Proof. For exclusion, inspect the local vertex witnessing a mapped incidence. A fresh internal image has neither boundary status. A local input used as a source maps to a host source port, which is not an output; a local output cannot be a source. The target argument is dual.

For a retained host vertex requiring a source incidence, use host coverage to find an original edge e and its corresponding source-port index. Input coverage in H_e supplies an edge sourced at that filling input, whose image is the original host vertex. Target coverage uses a filling output instead. For a fresh internal vertex in a copy of H_e, apply coverage directly inside H_e and insert the resulting incidence. These cases exhaust the expanded vertex datatype. □

Lean: Graph.Net.Expansion.source_not_output

Lean: Graph.Net.Expansion.target_not_input

Lean: Graph.Net.Expansion.source_host_covered

Lean: Graph.Net.Expansion.target_host_covered

Lean: Graph.Net.Expansion.source_coverage

Lean: Graph.Net.Expansion.target_coverage

Internal labels come from the indicated host or filling summand, and an edge (e, x) has the label of x in H_e. The host map preserves vertex labels by inspection. Each insertion map preserves them because its boundary cases are exactly the host source-port and target-port typing equations.

Lean: Graph.Net.Expansion.internalLabel

Lean: Graph.Net.Expansion.edgeLabel

Lean: Graph.Net.Expansion.host_label

Lean: Graph.Net.Expansion.insert_label

Applying these label equations to each mapped incidence list reduces its typing equation to the filling's original typing equation. Finite enumeration is obtained by a dependent sum over the finite host-edge enumeration, and a disjoint sum for the internal carriers. The constructor therefore proves all fields of `Net`; it does not postulate the incidence invariants of the simultaneous substitution.

Lean: Graph.Net.Expansion.source_typed

Lean: Graph.Net.Expansion.target_typed

Lean: Graph.Net.expand

29.37 Retained corollas, identity expansion, and hole freedom

Definition 29.79 (Corolla copy)**.** A corolla copy of an original edge e has one distinguished edge with the same label as e, source list equal to the complete ordered input boundary, and target list equal to the complete ordered output boundary. Every edge of the copy equals the distinguished edge. The existing corolla constructor supplies such a copy directly; its sole edge index is 0.

Lean: Linear.ExpansionPaths.CorollaCopy

Lean: Linear.ExpansionPaths.corollaCopy

Every source or target incidence of e becomes the corresponding incidence of its expanded corolla copy. Choose its port index in the host list; that boundary vertex occurs in the corolla's full boundary list and maps to the retained host vertex. More precisely, the copy's expanded incidence lists are the original host incidence lists mapped by ι_C. Mapping the full port-index enumeration reconstructs the original ordered list, proving these equalities. If the original edge label is globally focus-free, so is its corolla copy: every edge of the copy is the distinguished one and has that same label.

Lean: Linear.ExpansionPaths.copy_source

Lean: Linear.ExpansionPaths.copy_target

Lean: Linear.ExpansionPaths.copy_source_eq

Lean: Linear.ExpansionPaths.copy_target_eq

Lean: Linear.ExpansionPaths.copy_focusFree

Theorem 29.80 (Identity expansion)**.** *Expanding every edge by its original corolla gives a graph isomorphic to C.*

Proof. Forget the empty internal carrier of every corolla and the unique edge index inside each corolla. These give explicit vertex and edge bijections; their inverses retain a host vertex or attach edge index 0. Internal and edge labels are unchanged. For incidence, the exact copy-list equations reduce the result to a host inclusion followed by its inverse projection, which is the identity on every internal or boundary vertex. □

Lean: `Linear.ExpansionPaths.identityVertices`

Lean: `Linear.ExpansionPaths.identityEdges`

Lean: `Linear.ExpansionPaths.identityIso`

Lemma 29.81 (Hole-free expansion)**.** *If every H_e is globally hole-free, then $C[H]$ is globally hole-free.*

Proof. A hole occurrence in the expansion lies in the label of some tagged edge (e, x), possibly within its nested box contents. That label is exactly the label of x in H_e. The same occurrence in that label would be a hole occurrence in H_e, contradicting its global hole freedom. □

Lean: `Linear.ExpansionPaths.holeFree`

29.38 Paired path reflection for many fillings

A non-input vertex of H_e cannot be inserted as a global input: target coverage in H_e supplies an incoming edge whose expanded copy still targets its image. Global inputs have no incoming edge.

Lean: `Linear.ExpansionPaths.insert_not_input`

Theorem 29.82 (Local extraction under simultaneous filling)**.** *If $\iota_e(v)$ is operation-reachable in $C[H]$, then v is operation-reachable in H_e, independently of all other fillings.*

Proof. Induct on a reverse operation trace to the global input, generalizing v. If v is a filling input, take the empty local trace. Otherwise an empty global trace is impossible by the previous lemma. In the nonempty case, target coverage in H_e provides an edge at v. Unique incoming incidence in the expanded graph forces the first trace edge to be this inserted edge. Pull its chosen source back into H_e, extract a local trace from the shorter global trace by induction, and prepend that edge. This is the single-filling extraction proof with the simultaneous incidence theorem providing uniqueness. □

Lean: `Linear.ExpansionPaths.extract`

Consequently, if H_e is one-way, no term-typed host target of e can be operation-reachable in $C[H]$. Write that target as a target port of e. Its corresponding filling output has the same term type and the same image. Extraction would make this output operation-reachable in H_e, contradicting one-wayness.

Lean: `Linear.ExpansionPaths.oneWay_target`

Definition 29.83 (Paired output-closure for expansion)**.** Two filling families H, K satisfy the paired edge condition if, for every original edge e, either both fillings are corolla copies of e, or H_e is one-way, or K_e is one-way. The disjunction is chosen separately for each e; there need not be one side that is one-way at all replaced edges.

Lean: `Linear.ExpansionPaths.OutputClosedPair`

Lemma 29.84 (Paired predecessor argument)**.** *Suppose a term vertex v of the host is the target of e and its retained copies are operation-reachable in both expansions. Under the paired edge condition, e is an operation, and every source of e is a term vertex whose copies are operation-reachable on both sides.*

Proof. Either one-way branch contradicts reachability on that side by the previous result. In the remaining branch both sides retain corolla copies. Their expanded edges target the corresponding copies of v. The incoming-edge theorem forces their label, hence the original label of e, to be an operation. Every source of e is therefore term-typed. Its mapped incidences in the two copies and the predecessor reachability theorem give reachability on each side. □

Lean: `Linear.ExpansionPaths.predecessor`

Theorem 29.85 (Paired host-path reflection)**.** *If a host path ends at a term vertex reachable on both sides, all its edges are operations. In addition, every source of every edge on the path is term-typed and reachable on both sides.*

Proof. Induct on the nonempty host path. For a singleton path use the predecessor lemma. For a path $e :: f :: E$, apply the induction hypothesis to $f :: E$. An adjacency witness is both a target of e and a source of f; the strengthened induction conclusion supplies its type and reachability on both sides. Apply the predecessor lemma to e, then combine its conclusion with that for the tail. Repeated edges and any number of replaced holes are allowed. □

Lean: `Linear.ExpansionPaths.path_operations`

29.39 Actual focussed expansions and their obstruction

Retain the host focus by a corolla copy and require every other filling to be globally focus-free. The expanded focus is the tagged distinguished edge of that copy, with the same mode and the retained copies of the original endpoints. The copy-list equalities give its singleton incidences. Host-map injectivity preserves distinct endpoints, and the unchanged boundary constructors preserve the prohibition on an exposed input-to-output focus. For uniqueness, an edge from the focus copy equals its distinguished edge; any edge from another filling is globally focus-free.

Lean: `Linear.ExpansionPaths.focussed`

Initiality and finality are unchanged because both the mode and the input status of the focus source are unchanged. Once global hole freedom of the expanded graph is supplied, the construction packages the graph and its proved focus as an actual state.

Lean: `Linear.ExpansionPaths.focussed_boundary`

Lean: `Linear.ExpansionPaths.state`

Theorem 29.86 (Actual-state paired obstruction)**.** *If the host has a path containing a non-operation edge and ending at its focus source, and the two filling families satisfy the paired edge condition, at least one of the two expanded states is not rooted.*

Proof. If both states were rooted, Lemma 23 would make both actual focus sources operation-reachable. They are the retained copies of the host focus source, which has term type. Paired host-path reflection would therefore make every edge of the given path an operation, contradicting its specified non-operation edge. Classical case distinction turns this incompatibility into the stated disjunction of non-rootedness. □

Lean: `Linear.ExpansionPaths.pair_not_rooted`

29.40 Concrete pre-templates and shallow specimens

Definition 29.87 (Concrete pre-template)**.** At a fixed depth bound, a concrete pre-template is an interface-indexed binary relation on hypernets, with a proof that both members of each related pair are globally focus-free. The relation and its domain condition are defining data, not a law asserting operational behavior.

Lean: `Linear.ConcretePreTemplate`

Definition 29.88 (Concrete output-closure)**.** A concrete pre-template is output-closed when every related pair has a one-way member on at least one side. This is the paper's output-closure predicate, using the concrete one-way definition.

Lean: `Linear.ConcretePreTemplate.OutputClosed`

For each original edge, the shallow specimen records one of two kinds of data: retained corolla copies on both sides, or an actual hole label on that edge together with a related pair of fillings. Thus arbitrary changes to non-hole edges are not permitted by this specimen definition.

Lean: `Linear.SpecimenEdge`

Definition 29.89 (Concrete shallow specimen)**.** A shallow specimen consists of a globally unique-hole context of interface $\star \to \epsilon$, a distinguished focus, and two simultaneous filling families with the preceding edge-wise data. The erasure of the focussed context must belong to the supplied allowed-context predicate. Both expanded graphs must be globally hole-free. These fields express actual typing, context, relation-membership, and closedness restrictions; they do not include rootedness, path reflection, or a context-traversal law as assumptions.

Lean: `Linear.ShallowSpecimen`

This is the shallow restriction of the paper's specimen construction. It permits arbitrarily many outer holes and arbitrarily deep boxed contents in the fillings. It does not replace holes inside the original host's boxes. Compatibility with arbitrary addressed substitution and extension to deep hole occurrences remain required before identifying it with the paper's full specimen notion.

A focus label cannot equal a hole label, by disjointness of primitive labels at either zero or successor depth. Therefore the focus edge must be in the retained branch of the specimen data.

This proves existence of its two corolla copies, which are chosen to construct the focussed expansions.

Lean: `Linear.focusLabel_ne_hole`

Lean: `Linear.ShallowSpecimen.focus_copies`

Lean: `Linear.ShallowSpecimen.focusCopies`

Every other filling is globally focus-free. For a retained copy, use the original focus's uniqueness and the copy-label theorem. For a filled hole, use the pre-template's focus-free domain. These derived facts and the specimen's proved graph closedness construct its two actual states.

Lean: `Linear.ShallowSpecimen.other_free`

Lean: `Linear.ShallowSpecimen.leftState`

Lean: `Linear.ShallowSpecimen.rightState`

29.41 Lemma 26(3) and the boundary bridge for shallow specimens

Theorem 29.90 (Output-closure supplies paired reflection)**.** *An output-closed concrete pre-template supplies the paired edge condition for every one of its shallow specimens.*

Proof. At a retained edge, take the two recorded copies. At a filled hole, apply output-closure to the recorded related pair to obtain a one-way side. The side may change at the next hole. □

Lean: `Linear.ShallowSpecimen.outputClosedPair`

Theorem 29.91 (Concrete shallow Lemma 26(3))**.** *For a shallow specimen of an output-closed pre-template, any context path to its focus source containing a non-operation edge implies that at least one of its two actual states is not rooted.*

Proof. The preceding theorem supplies the paired edge condition. The focus-copy and focus-freedom lemmas supply actual expanded states. Apply the actual-state paired obstruction to those states and the given context path. □

Lean: `Linear.ShallowSpecimen.not_both_rooted`

Equivalently, when both specimen states are rooted, every edge of every context path to the focus source is an operation. The formal proof applies paired host-path reflection directly to the two operation-reachability witnesses supplied by Lemma 23. This is the path consequence used in the paper's Lemma 39(2), here proved for shallow specimens; it is not yet the full entering/exiting classification or context-traversal theorem.

Lean: `Linear.ShallowSpecimen.rooted_context_path`

The proof uses the same incoming-incidence uniqueness and one-way extraction as Proof C.9. Rather than selecting a last hole and changing substitution order, it reasons backwards simultaneously on both sides along the supplied path. The paired predecessor lemma isolates exactly the contradiction that the paper obtains at its last hole. This proof organization avoids assuming an unproved interchange theorem.

Theorem 29.92 (Concrete shallow Lemma 39(1))**.** *The two actual states of a shallow specimen agree on initiality and finality.*

Proof. Each expanded focus has the same mode and source-input status as the original context focus. Compose the left and right equivalences supplied by the focussed-expansion boundary theorem. Output-closure and rootedness are not needed for this boundary assertion. □

Lean: `Linear.ShallowSpecimen.boundary_agreement`

29.42 A constructed two-hole focussed specimen

The earlier one-hole context has exactly one hole occurrence: inspection of its labels rules out the focus and value edges, leaving its selected hole edge. This proves global hole-label uniqueness and the absence of any retained hole when that edge is removed.

Lean: `Linear.Examples.oneWayContext_hole_only`

Lean: `Linear.Examples.oneWayContext_unique`

Lean: `Linear.Examples.oneWayContext_no_retained`

Replace that hole by the existing chain of two distinct holes. The exact freshness theorem gives a globally unique-hole context, since the host retains no hole and the inserted chain has distinct hole labels. Its focus is retained by the previously checked shallow focus-free plugging operation.

Lean: `Linear.Examples.twoHoleFocusedContext`

Lean: `Linear.Examples.twoHoleFocusedContext_unique`

Lean: `Linear.Examples.twoHoleFocus`

The simultaneous filling family replaces both new holes by variable corollas and retains the focus and value by their original corollas. Define a pre-template relating a graph to itself when it is focus-free and one-way. This relation has the required focus-free domain, and is output-closed by its one-way premise. At each hole the pair of variable corollas belongs to this relation; at every other edge the corolla-copy fields hold by construction.

Lean: `Linear.Examples.twoHoleFilling`

Lean: `Linear.Examples.oneWayEquality`

Lean: `Linear.Examples.oneWayEquality_closed`

Lean: `Linear.Examples.twoHoleFilling_edges`

Every filling is globally hole-free. A variable corolla has no hole, while a hole in a retained corolla would supply a retained hole occurrence in the earlier one-hole context, already excluded. The expansion hole-freedom lemma therefore proves closedness. These data assemble an actual two-hole specimen, with the unrestricted allowed-context predicate for this example.

Lean: `Linear.Examples.twoHoleFilling_closed`

Lean: `Linear.Examples.twoHoleSpecimen`

The second hole by itself is a path to the focus source and is not an operation. The concrete shallow Lemma 26(3) proves the required disjunction of non-rootedness for the two constructed

states, for every operation classification. Their initial/final agreement follows from the concrete specimen boundary theorem.

Lean: `Linear.Examples.twoHoleSpecimen_not_rooted`

Lean: `Linear.Examples.twoHoleSpecimen_boundary`

29.43 Alternating one-way choices: a necessary regression

Use an operation signature with one unary operation, and form both its operation corolla and a variable corolla. Relabel the existing two-hole chain to this signature, leaving its hole interfaces and incidences unchanged. On the left fill the first hole with a variable and the second with an operation; on the right reverse these choices.

Lean: `Linear.AlternatingFillings.ops`

Lean: `Linear.AlternatingFillings.variableWire`

Lean: `Linear.AlternatingFillings.operation`

Lean: `Linear.AlternatingFillings.context`

Lean: `Linear.AlternatingFillings.left`

Lean: `Linear.AlternatingFillings.right`

The variable corolla is one-way: reachability of its output would force its incoming variable edge to be an operation. The operation corolla is not one-way, since its unique edge explicitly supplies an operation path between its term input and output.

Lean: `Linear.AlternatingFillings.variable_oneWay`

Lean: `Linear.AlternatingFillings.operation_not_oneWay`

The paired edge condition holds: use the left one-way filling at the first hole and the right one-way filling at the second. Neither entire family is one-way edge by edge, as evaluation at the second left hole or the first right hole yields the non-one-way operation corolla. Finally, the two-edge host path cannot end at an output reachable on both sides. Otherwise paired path reflection would make its hole labels operations, a contradiction. This regression rules out an accidental strengthening of pointwise output-closure to a single global one-way-side choice.

Lean: `Linear.AlternatingFillings.output_closed`

Lean: `Linear.AlternatingFillings.no_global_oneWay_side`

Lean: `Linear.AlternatingFillings.not_both_reachable`

30 Proof checking and remaining obligations

30.1 The two axiom audits

The file `Audit.lean` contains six `#print axioms` commands. They inspect counting-simulation soundness, the forward and converse simulation constructions, the two contextual sufficiency theorems, and the zero-step converse regression theorem. These diagnostic commands introduce no new definitions or proofs.

Lean: `Audit: abstract theorem dependencies`

The graph audit inspects shallow plugging, host and inserted path equivalence, plugging congruence, exact trace uniqueness, typed operation-path uniqueness, finite deep enumeration, unique holes under boxing, deep plugging and its other-label locality, and the self-loop and deep-replacement regression proofs. Twelve further commands audit occurrence-coordinate observation preservation, exact hole inventories, sharp freshness and consumption, isomorphism invariance of contexts, deep site provenance, deep context closure, and the new reuse and collision regressions.

Lean: `Graph.Audit: concrete theorem dependencies`

Lean: `Graph.Audit: context calculus dependencies`

Nine further commands audit the exact deep inventory, both versions of sharp context closure, hole-freeness, selected-label consumption, certified deep plugging, and the deep reuse and collision regressions.

Lean: `Graph.Audit: exact deep inventory dependencies`

Eleven further commands audit recursive label transitivity, operation-path invariance, occurrence embeddings, hole and context invariance, deep congruence, quotient plugging and context validity, and the strict-extension regressions.

Lean: `Graph.Audit: recursive isomorphism dependencies`

Ten further commands audit shallow host congruence, site transport and its specification, simultaneous congruence, quotient invariance, retained holes, freshness, result validity, and the moved-address and wrong-address regressions.

Lean: `Graph.Audit: site transport dependencies`

Sixteen further commands inspect shallow associativity, both corolla units, deep associativity, quotient composition, composed retained inventories, sharp composite context closure and its certified constructor, three-stage coherence, deep restoration, quotient identity, and the self-loop, repair, and mixed-depth regressions.

Lean: `Graph.Audit: composition and unit dependencies`

Six further commands inspect global focus-free substitution, focussed shallow and deep substitution, finite-filling boundary agreement, the initiality regression, and rejection of a boxed focus.

Lean: `Graph.Audit: concrete focus dependencies`

Eight further commands inspect operation-view existence, search determinism, search boundary laws, shallow filling preservation, and the thunk, value, and variable regressions.

Lean: `Graph.Audit: search rule dependencies`

Ten further commands inspect explicit focus insertion and certification, both incidence-recovery

laws, hole freedom, term-typing preservation, initial attachment, search endpoint realization, and the interior and value regressions.

```
Lean: Graph.Audit:  focus realization dependencies
```

Twelve further commands inspect general erasure and its invariants, the reinsertion isomorphism and focus endpoints, arbitrary-state recovery, state search determinism and boundary laws, and the execution/interior regressions.

```
Lean: Graph.Audit:  erasure and state search dependencies
```

Fifteen further commands inspect insertion–erasure cancellation, coordinate transport and realization, erasure naturality, source invariance, quotient search determinism and boundaries, and the concrete quotient regressions.

```
Lean: Graph.Audit:  round trips and quotient search dependencies
```

Fourteen further commands inspect recursive focus freedom, coordinate search, insertion and erasure, recursive state search and quotient determinism, the quotient projection and lifting laws, exact-length executions, and regressions.

```
Lean: Graph.Audit:  recursive state search and execution dependencies
```

Sixteen further commands inspect canonical roots, search preservation of rootedness, operation reachability, insertion of rooted paths, the concrete path obstruction, refocusing, and the positive and negative state examples.

```
Lean: Graph.Audit:  rooted-state and path-obstruction dependencies
```

Sixteen further commands inspect operation-source typing, one-wayness and recursive renaming, path extraction and host reflection through fillings, one-way filling closure, the concrete Lemma 26(2) obstruction, its paired single-hole corollary, and the positive and negative examples.

```
Lean: Graph.Audit:  one-way filling dependencies
```

Twenty-five further commands inspect simultaneous expansion incidence, local path extraction, paired path reflection, focussed expansion, its identity isomorphism and hole freedom, the concrete shallow specimen bridges, and the two-hole and alternating-side regressions.

```
Lean: Graph.Audit:  simultaneous shallow specimen dependencies
```

In the pinned snapshot, counting-simulation soundness, finite deep enumeration, unique holes under boxing, occurrence-coordinate observation preservation, isomorphism invariance of unique holes, and uniqueness of the two-hole example report only `propext` and `Quot.sound`. Recursive label transitivity, recursive operation-path invariance, the nonexistence of a literal-label isomorphism between the example boxes, and their equality as quotient classes likewise report only these two axioms. Existence of transported sites also reports only these two axioms; the chosen transport and substitution results additionally use classical choice. All audited composition and unit results report these three standard axioms. The focus results do too, except the boxed-focus rejection, which uses only `propext` and `Quot.sound`. The search audits use only `propext` and `Quot.sound`, except shallow filling preservation, which also uses classical choice. The new realization audits use all three standard axioms, except term-typing preservation, which uses only propositional extensionality and quotient soundness. All new general-erasure and state-search audit entries use those three standard axioms. The round-trip and quotient-search audits do too, except coordinate search renaming, which uses only `propext` and `Quot.sound`. The recursive-search audits use all three axioms, except label and graph focus-freedom invariance, coordinate search transport, and the renamed-box

focus-freedom example, which use only `propext` and `Quot.sound`. The rooted-state audit entries use all three axioms, except coordinate operation-reachability preservation and the incoming-trace operation theorem, which use only `propext` and `Quot.sound`. In the one-way audit group, source typing, the input-type lemma, the reachability characterization, recursive renaming, and the four basic corolla checks use only `propext` and `Quot.sound`. The extraction, reflection, filling-closure, obstruction, and filled-state example entries also use classical choice. In the simultaneous-expansion group, the six incidence checks, identity isomorphism, hole freedom, and the two alternating-side premise checks use only `propext` and `Quot.sound`; the other entries also use classical choice. These are standard Lean logical axioms: propositional extensionality, quotient soundness, and classical choice, respectively. Neither audit reports `sorryAx` or a project-specific axiom.

Absence of project-specific axioms does not discharge a theorem's hypotheses. Specimen laws and initial lifting remain parameters of the abstract sufficiency results. They do not appear as named global axioms in the report. In contrast, the concrete graph constructions explicitly prove the invariant fields needed to construct their results. Neither fact supplies the missing bridge to a concrete UAM.

30.2 Reproducing the Lean and document checks

The snapshot pins Lean `4.35.0` in `lean-toolchain` and uses only the bundled `Std`. With that toolchain available, run

```
lake build
lake env lean Hypernets/Audit.lean
lake env lean Hypernets/Graph/Audit.lean
```

The script `scripts/check.sh` runs the build and both audits. Run it using `bash` with the script path as its argument. It selects the workspace-local macOS ARM compiler if present, and otherwise uses `lake` from the environment. The audits are checked separately from the root import.

The commentary's source checker recursively scans all Lean modules under `Hypernets/`, including nested graph modules, attributed declarations, noncomputable definitions, abbreviations, and anonymous examples. It verifies that each declaration has a source-fragment link, all linked ranges are valid, and each working source file equals the version at the pinned commit. Diagnostic audit ranges are linked separately. Run it with

```
python3 docs/mathematical-commentary/check_sources.py
```

This syntactic coverage check complements the mathematical exposition; it does not itself check the correctness of English explanations. Source links are immutable GitHub commit links, so future edits cannot move the referenced proof.

From the commentary project directory, `make` checks the links, compiles with `latexmk`, and exports the PDF to the repository's `output/pdf/` directory. The committed PDF can therefore be read without a TeX installation, and its editable sources and reproduction instructions are included alongside it.

30.3 What remains for the paper's concrete theorem

This is a to-do list only: completed obligations are removed. The remaining work needed to claim Theorem 6 with its original hypotheses is:

1. Prove representation adequacy between the typed depth-indexed hierarchy with explicit binder metadata and the paper's set-based graph-labelled hypernets, including compatibility across different depth bounds.
2. Relate simultaneous shallow expansion to addressed substitution. Construct a quotient of host/site pairs and prove interchange of substitutions at independent sites.
3. Identify erasure-based state search with the paper's contextual local graph rewrites in both directions.
4. Define the concrete substitution and compute transitions, with their typing and context restrictions, and assemble the full UAM transition relation.
5. Prove rootedness preservation for substitution and the required full-UAM structural consequences of determinism and refocusing. Establish the extension of the checked shallow Lemma 26(3) to deep specimen substitution, and the stationary rewrite criterion where needed.
6. Extend the concrete shallow specimen representation to arbitrary deep hole occurrences and establish its compatibility with the paper's tuple substitution. Define concrete quasi-specimens, input-safety, and robustness with the paper's restrictions.
7. Construct the abstract specimen interface from these objects. Extend the checked shallow specimen boundary agreement and rooted-context path consequence to full deep specimens. Prove the remaining Lemma 39(2–3) entering/exiting classifications and context-only traversal laws. Prove the safety and robustness bridges and the concrete observation/context membership witnesses needed for initial lifting.
8. Instantiate the checked contextual sufficiency results using the concrete interface, observations, and lifting. Discharge every remaining concrete obligation to obtain the target robustness theorem.

References

[1] Dan R. Ghica, Koko Muroya, and Todd Waugh Ambridge. *A robust graph-based approach to observational equivalence*. Logical Methods in Computer Science 21(2:8), 2025. Reference version: arXiv:1907.01257v5.

[2] *Hypernets in Lean 4*. Repository at revision `b850e7a`.

A Technical terms

This appendix is alphabetical. Click a term or its page number to jump to its definition in this PDF. Related terms may share a definition. Descriptions of abstract interface predicates state what the interface supplies; they do not claim that a concrete UAM interpretation has already been constructed.

Adjacency p. 36

Edges e, f are adjacent when a vertex occurs in both the target list of e and the source list of f.

Allowed context p. 16

A context satisfying the supplied predicate Allowed in an observation interface.

Alphabet p. 42

A type of edge labels equipped with a signature assigning their input and output wire types.

Backtracking mode p. 17

The abstract focus mode backtrack, one of the three constructors of the focus datatype.

Bijection p. 29

A pair of forward and inverse functions satisfying both inverse equations.

Boundary finality p. 90

Backtracking mode with the focus source at an input; on a state this is the final-state characterization of Proof F.4.

Boundary initiality p. 90

Search mode with the focus source at an input; preserved through all certified fillings.

Boundary port p. 30

An input or output constructor carrying an index in the corresponding ordered boundary list; input and output constructors are disjoint.

Boundary transport p. 76

Reindexing boundary ports along equal ordered interfaces while preserving internal and edge carriers.

Box p. 42

A label storing bound/free counts n, m and a body of type $* \to *^{n+m}$; its outer edge has type $T^n(*) \to *^m$.

Box-label replacement p. 46

Changing the body of one selected box edge while retaining its binding metadata and all outer incidence data.

Canonical root p. 126

The initial state class obtained by erasing a state's focus and attaching a search focus at its unique input.

Certified composed plugging p. 85

A context constructor using pairwise disjoint retained and final inventories, without requiring intermediate validity.

Certified deep fresh plugging p. 66

The context constructor that packages addressed replacement with a proof of exact site freshness, permitting reuse of consumed labels.

Child occurrence p. 51

An occurrence inside a box body. Primitive labels have no child occurrences.

Compatible provenance p. 54

The condition that no hole label in the replacement occurs at a retained host origin of the provenance map.

Complement restriction p. 77

The bijection between the complements of corresponding selected elements, used for surviving host edges.

Composed site p. 83

The address of an inner selected occurrence after inserting its host at an outer site.

Concrete output-closure p. 143

The requirement that each related pair in a concrete pre-template has at least one one-way member.

Concrete pre-template p. 143

An interface-indexed relation on actual hypernets, with proofs that its related members are globally focus-free.

Concrete refocusing p. 132

Preservation of concrete rootedness by each transition of a relation on recursive state classes.

Concrete rootedness p. 127

Existence of a finite search-only run from the canonical initial attachment to the state's recursive class.

Concrete state p. 92

A globally focussed, hole-free hypernet of type $* \to \epsilon$; the concrete transition relation remains to be defined.

Concrete typed net p. 31

A finite labelled hypergraph with ordered boundaries and certified distinctness, incidence uniqueness, coverage, and signature typing.

Context p. 45

A hypernet in which each complete typed hole label occurs at most once over all nesting depths.

Context-only traversal p. 17

The specimen-interface law supplying paired context steps for a rooted, settled specimen whenever either endpoint can step.

Contextual equivalence p. 16

Mutual contextual refinement: both directions hold for every pair of observations in the same allowed context.

Contextual refinement p. 16

The requirement that observations of two fragments in each common allowed context produce states related by the chosen state refinement.

Coordinate characterization of state search p. 114

Search from a realized state is equivalent to a coordinate step on its original graph followed by a focussed endpoint isomorphism.

Corolla p. 32

A graph with one operation edge, no internal vertices, and precisely that operation's ordered input and output boundaries.

Corolla copy p. 140

A one-edge filling retaining an original label and its complete ordered input and output boundaries.

Corresponding sites p. 78

Addresses related by edge bijections at each enclosing level, with the same local replacement interface.

Cost converse p. 12

The relation Q^{-1} defined by $a\,Q^{-1}\,b$ exactly when $b\,Q\,a$.

Cost preorder p. 12

A reflexive and transitive relation on natural-number execution counts; additional algebraic properties are imposed separately.

Counted execution p. 10

The relation $s \longrightarrow^{n} t$, recording exactly n machine transitions, including the zero-step execution from a state to itself.

Counting simulation p. 14

A relation preserving finality and matching steps using related execution counts and an up-to relation, with an explicit stuck-execution alternative.

Deep address transport p. 71

The map obtained by converting a deep address to fibre coordinates, applying a hole embedding, and converting back to a target address.

Deep freshness p. 65

Disjointness between the replacement hole inventory and the host holes retained along the selected address; it is necessary and sufficient for composing two contexts.

Deep occurrence p. 44

An address that records an enclosing box edge and recursively an occurrence in its body, distinguishing equal bodies at different locations.

Deep plugging p. 46

Typed replacement at an addressed site, recursively descending through boxes and rebuilding the selected enclosing labels.

Deep retained hole p. 64

A host hole under an unselected edge at some level of the address, as defined recursively without reference to the replacement graph.

Depth bound p. 43

The index bounding permitted box nesting in a hypernet; a graph need not use all of the available depth.

Depth promotion p. 44

Raising a hypernet's available depth bound by relabelling primitives and recursively promoting box bodies, without changing outer incidence.

Determinism p. 9

The property that any two one-step successors of the same state are equal.

Disjoint hole inventories p. 53

The condition that no complete typed hole label occurs in both graphs, including occurrences inside boxes.

Edge p. 31

An element of a net's finite edge carrier, carrying an operation label and ordered source and target vertex lists.

Edge hole inventory p. 62

The hole labels observed in one edge fibre, including all descendants of its label.

Empty graph p. 32

The net with no internal vertices, no edges, and empty input and output boundaries.

Entering specimen p. 17

A specimen satisfying the interface's supplied entering predicate. Its concrete interpretation by hole incidence is not yet instantiated.

Erasure-based state search p. 109

Search between actual states, defined by a coordinate step on the erased source and a focus-preserving isomorphism to the realized target.

Exact cost order p. 12

The preorder E with $a\,E\,b$ exactly when $a = b$, requiring identical successful execution counts in refinement.

Extracted state coordinate p. 109

The original mode and joined term vertex in the generally erased underlying graph of an arbitrary closed state.

Fibre embedding p. 71

An injective map between label fibres that preserves the complete hole observation at every position.

Fibre provenance p. 57

An injective, hole-observation-preserving map from a changed label's occurrence fibre to retained old fibre addresses or replacement occurrences.

Final state p. 9

A state satisfying a machine's successful-finality predicate; the machine laws require that it has no outgoing transition.

Finite enumeration p. 29

A finite list containing every element of a carrier, together with a completeness proof; the list may contain repetitions.

Finite filling sequence p. 91

A finite sequence of focus-free addressed replacements avoiding the transported focus, preserving its boundary status.

Focus erasure p. 104

Deleting the unique focus edge and its internal target, contracting the target onto the source while proving all graph invariants.

Focus insertion p. 98

Splitting a non-output term vertex and connecting its old image to a fresh internal vertex by a new focus edge.

Focus mode p. 17

One of search, backtrack, or rewrite; this datatype alone does not specify a graph position.

Focus round trip p. 107

Erasure followed by reinsertion, with an explicit isomorphism restoring original names, the focus edge, and its endpoints.

Focus-avoiding site p. 91

A shallow site different from the focus edge, or any site strictly inside a box.

Focus-free hypernet p. 89

A graph with no focus labels at any depth, including inside every boxed body.

Focussed hypernet p. 89

A hypernet with exactly one global focus, shallow, self-acyclic, and not exposed.

Focussed isomorphism p. 108

A graph isomorphism preserving the distinguished focus edge and mode, hence its endpoints and boundary observations.

Focussed recursive isomorphism p. 118

An isomorphism permitting independent names inside boxes while preserving the distinguished focus edge and its mode.

Fragment p. 16

An element of the family F_i in an observation interface, matched with contexts of the same interface index i.

Graph signature p. 30

The assignment of an ordered input-label list and output-label list to each operation label.

Hole embedding p. 71

An injective map of graph occurrences preserving hole labels and the absence of a hole; it need not itself preserve incidence.

Hole inventory p. 53

The collection of complete hole labels h for which some shallow or deep occurrence observes h.

Hole label p. 42

A natural-number name together with input and output wire-type lists. Equality compares the complete typed label, not just its name.

Hole lookup p. 45

The recursive function returning the typed hole label at an occurrence, or no label when that occurrence is not a hole.

Hole presence p. 53

The predicate $\mathsf{HasHole}(G, h)$, witnessed by an occurrence whose hole lookup returns h.

Hole-free graph p. 53

A graph having no hole label at any occurrence or nesting depth.

Host embedding p. 33

The map retaining host vertices during shallow replacement, tagging internal names and fixing each external boundary position.

Hypernet p. 43

A typed net over a finite-depth alphabet whose labels may include boxes with bodies over the preceding alphabet.

Improvement cost order p. 12

The preorder I with $a\, I\, b$ exactly when $b \leq a$; the target successful execution takes no more steps than the source.

Incidence coverage p. 31

The net laws saying that exactly non-output vertices occur in some source list and exactly non-input vertices occur in some target list.

Incidence uniqueness p. 31

The net laws allowing a vertex to occur in the sources of at most one edge and in the targets of at most one edge.

Incoming incidence trace p. 129

A reverse path recording each target and source incidence, without restricting edge labels.

Initial focus attachment p. 102

Constructing a genuine initial state by inserting search mode at the sole input of a focus-free, hole-free graph.

Initial lifting p. 16

The contextual-lifting hypothesis placing initial observations of related fragments into the specified lifted state relation.

Input-safety p. 19

The abstract condition on rooted, entering search specimens: a stuck alternative, a cost-related execution to a settled specimen, or a quasi-specimen alternative.

Insertion map p. 33

The map tagging replacement internal vertices freshly and identifying its boundary positions with the selected host edge's indexed ports.

Insertion–erasure cancellation p. 111

Erasing an inserted focus recovers the original graph up to isomorphism and restores its insertion position.

Interface p. 31

For a concrete net, the ordered input and output wire-label lists; matching lists enforce type-correct replacement.

Internal vertex p. 30

A vertex from the internal carrier, tagged separately from all input and output boundary positions.

Isomorphism p. 39

Bijections of internal vertices and edges preserving labels and ordered incidence lists while fixing the external boundary positions.

Isomorphism class p. 74

A hypernet modulo recursive isomorphism at a fixed depth bound and ordered interface.

Lifted state relation p. 18

The relation witnessed by a remote specimen with rooted endpoints equal to the two states being compared.

Machine p. 9

A transition relation on states, a successful-finality predicate, and a proof that final states cannot step.

Non-self-loop hole p. 37

A selected edge whose source and target lists are disjoint; this condition permits reflection of inserted paths.

Normal state p. 9

A state with no outgoing transition, whether or not it is successfully final.

Observation interface p. 16

Typed fragment and context families, an allowed-context predicate, and a possibly partial, nonfunctional relation producing observed states.

Occurrence p. 44

An individual shallow edge or an addressed edge inside boxes; occurrences distinguish positions even when their labels agree.

Occurrence coordinates p. 51

A dependent pair consisting of an outer edge and an address in the occurrence fibre of that edge's label.

Occurrence fibre p. 51

The disjoint sum of the edge's own shallow occurrence and all child occurrences determined by its label.

Occurrence observation p. 51

For a fibre address, the hole lookup obtained either from its shallow label or recursively from its child address.

One-way hypernet p. 133

A hypernet with no operation path from a term input to a term output; paths to thunk outputs are permitted.

Operation kind p. 93

The active/passive classification; finishing produces rewrite/backtrack mode respectively.

Operation path p. 41

A forward, endpoint-sensitive path defined by a reverse operation trace with its edge list reversed; empty paths are permitted.

Operation specification p. 42

The assignment of a term-argument count and an ordered list of thunk binder counts to each operation.

Operation step p. 40

A step through an edge selected by the operation predicate, from a vertex in its source list to one in its target list.

Operation trace p. 41

A reverse traversal recording each actual operation edge, starting at a target vertex and continuing from that edge's source.

Operation-reachable vertex p. 128

A vertex reached from an input by a path consisting only of actual operation edges.

Opposite specimen system p. 21

The specimen system obtained by swapping its left and right endpoint maps while retaining modes and entering and rootedness predicates.

Original selected fragment p. 87

The corolla carrying a site's selected label, including the whole body when the label is a box.

Overhead cost order p. 12

The preorder H with $a\,H\,b$ exactly when $a \leq b$; the target successful execution takes at least as many steps as the source.

Paired output-closure p. 142

An edge-wise condition: retain both copies or have a one-way filling on at least one side, with the side allowed to vary.

Path p. 36

A nonempty edge list whose consecutive edges are adjacent. Repeated edges are allowed; this predicate does not specify endpoint vertices.

Plugging site p. 46

An address selecting an edge, possibly inside boxes, indexed by the required local depth bound and replacement interface.

Port p. 30

A vertex represented as an internal name, an indexed input, or an indexed output, with the three constructors kept disjoint.

Primitive label p. 42

An operation, variable, focus-mode label, or typed hole; a primitive label has no box body.

Primitive signature p. 42

The typed port assignment for primitives: one term input for non-holes, operation outputs determined by arities, and stored interfaces for holes.

Provenance certificate p. 54

An injective map assigning every result occurrence a retained host or inserted replacement origin with the same hole observation.

Quasi-specimen p. 18

The abstract relation requiring a non-rewrite specimen witness, with prescribed endpoint relations, whenever the compared states are rooted.

Quotient context validity p. 75

The unique-hole predicate on recursive isomorphism classes, justified by invariance under a change of representative.

Quotient plugging p. 75

For a fixed concrete host and site, the well-defined operation sending a replacement isomorphism class to the class of the plugged graph.

Quotient search p. 116

Erasure-based search descended to focus-preserving literal-label state isomorphism classes; it has unique successor classes.

Reasonable triple p. 12

Three cost preorders satisfying the addition, decrease, inclusion, and sandwich conditions used by counting-simulation soundness.

Reassociation bijections p. 81

Carrier bijections that change the sum tags of host, middle, and final replacement vertices and edges.

Recursive isomorphism p. 69

Carrier bijections preserving incidence and primitive labels, with corresponding box bodies recursively isomorphic and all binder metadata preserved.

Recursive label equivalence p. 69

Equality of primitives, or matching box metadata with recursively isomorphic bodies; primitive and box labels are never equivalent.

Recursive state class p. 122

A state modulo focussed recursive isomorphism, including independent renaming at every nested box depth.

Recursive state search p. 121

Coordinate search after erasure, allowing a focussed recursive isomorphism at the realized target.

Reindexing p. 38

Renaming a net's internal and edge carriers along bijections, transporting labels and incidence while preserving boundary order.

Relabelling p. 42

Changing operation labels along a signature-preserving map, leaving the graph's carriers, vertex labels, and incidence unchanged.

Related boxes p. 69

Boxes with the same bound/free metadata and bodies satisfying a specified relation at that exact interface.

Relational isomorphism p. 68

Boundary-preserving carrier bijections preserving ordered incidence and vertex labels, with corresponding edge labels related by a supplied relation.

Relational plugging congruence p. 73

The construction extending a replacement isomorphism for a reflexive label relation by identity on the surviving host.

Remote specimen p. 18

A specimen whose mode is not rewrite or which is not entering; only an entering rewrite focus is excluded.

Replacement carriers p. 33

The disjoint union of host and replacement internal names, and the union of surviving host edges with inserted replacement edges.

Retained hole label p. 55

A label observed below a surviving host edge, excluding the selected removed edge and all of its descendants.

Reverse operation traversal p. 40

The machine traversing selected operation edges backwards, taking boundary inputs as successful final states.

Rewrite mode p. 17

The abstract focus mode rewrite, used in the robustness premise for a remote specimen.

Robustness p. 19

The abstract condition requiring a rooted, remote rewrite specimen's source step to lead to stuckness or a cost-related quasi-specimen match.

Rooted state p. 17

A state satisfying the specimen interface's supplied predicate Root, which its laws require to be preserved by transitions.

Search mode p. 17

The abstract focus mode search, used with entering specimens in the input-safety premise.

Search operation view p. 93

A certified decomposition of an actual operation edge into its singleton source, ordered argument prefix, and remaining thunk targets.

Search outcome p. 94

Entry selects the first argument or finishes; return selects the next argument or finishes after the last.

Search position p. 94

A mode and outer vertex of the underlying graph; the bridge to explicit focus edges remains open.

Search step p. 94

The coordinate relation implementing the five rules of Fig. 9(a–e), with no substitution or compute transitions.

Search-only machine p. 124

A machine on state classes containing search transitions and boundary finality, but no substitution or computation transitions.

Selected hole label p. 66

The partial observation of the selected edge itself after following a site address; selecting a box returns no hole label.

Settled specimen p. 18

A specimen in backtracking mode, or in search mode without entering; every settled specimen is remote.

Shallow occurrence p. 44

The occurrence of an edge at the current graph layer, without descending into a box body.

Shallow plugging p. 36

Replacing one outer edge by a graph with its exact signature interface, gluing corresponding boundary ports and retaining the outer interface.

Shallow specimen p. 143

A focussed unique-hole context with two simultaneous shallow filling families, typed relation witnesses, allowed erasure, and hole-free resulting graphs.

Sharp plugging freshness p. 55

The requirement that no retained host hole label occurs in the replacement; labels confined to the removed region may be reused.

Simultaneous expansion p. 138

Replacement of every shallow host edge by a graph of its interface, with disjoint tagged internal carriers; corollas retain unchanged edges.

Site interface transport p. 78

Equality transport of the interface indices of an address, leaving its selected occurrence unchanged.

Site transport p. 79

A classically chosen corresponding address in a recursively isomorphic host, with unchanged local depth and interface.

Source port p. 31

A source-list vertex selected by a finite index in the edge label's input signature; the arity equation justifies the lookup.

Specimen p. 17

An element of the abstract specimen type, equipped with left and right state endpoints, a mode, and an entering predicate.

Specimen system p. 17

The interface packaging specimens, endpoints, modes and predicates, with rootedness preservation, final agreement, and context-traversal laws.

State equivalence p. 13

Mutual state refinement for the same cost preorder; it compares successful termination and execution counts in both directions.

State isomorphism class p. 116

A class of states identified by focus-preserving isomorphisms that preserve full edge labels literally.

State quotient projection p. 123

The surjective map from literal-label state classes to recursive state classes; search executions project and lift with exact lengths.

State refinement p. 13

The requirement that each successful source execution has a successful target execution with counts related by the chosen cost preorder.

Stuck state p. 9

A normal state that is not successfully final; it cannot step and does not count as successful termination.

Substitution unit p. 85

A selected one-edge corolla: filling it returns the inserted graph, while restoring an edge's own corolla returns its host, up to isomorphism.

Target port p. 31

A target-list vertex selected by a finite index in the edge label's output signature.

Term wire p. 42

The wire type $*$, distinguished from a thunk wire and used for ordinary operation inputs and term arguments.

Thunk wire p. 42

The wire type $T^n(*)$, carrying the number n of variables bound by a corresponding box.

Typed search position p. 101

A mode and term-labelled vertex of a graph of type $* \to \epsilon$, realizable as an explicit focussed state under focus and hole freedom.

Unary operations p. 40

The condition that every selected operation edge has a source list of length one; no one-output condition is imposed.

Unique holes p. 45

The property that any two occurrences observing the same complete typed hole label are equal, at every nesting depth.

Uniqueness away from an edge p. 63

Uniqueness of hole occurrences whose outer edges differ from one selected edge, with no restriction on the selected fibre.

Universal cost order p. 12

The preorder U relating every pair of natural-number counts, so refinement ignores quantitative cost.

Up-to closure p. 14

The relation obtained by placing a candidate relation between a left auxiliary state refinement and a right auxiliary state refinement.

Vertex label p. 31

The internal vertex's assigned label, or the wire label at the indexed position in the relevant boundary list.

Wire type p. 42

Either the term type $*$ or a thunk type $T^n(*)$ with its binding count.